

NVIDIA

Senior DevOps & Infrastructure Automation

JR2017386 · Interview Prep MEGA

Personalized for: Nandakumar Athiseshan
Software Engineering Technical Leader — Cisco IQ Platform

v1.0 · May 2026

Total Rounds	11
Total Q&A	163+
Target Role	Senior DevOps for NVIDIA AI Inference Operations
Products	Dynamo, NIXL — AI inference platform
Tech Stack	K8s, Terraform, AWS, Python, CI/CD, GitOps, Observability
Target Profile	AWS SA Pro · CKA · CKAD · Terraform Associate · 12+ yrs

How to Use This PDF

Every answer here is tuned to your actual Cisco IQ Platform experience — OpenShift→EKS migration, ArgoCD multi-account rollout, Karpenter, External Secrets, GenAI pipeline buildout. The doc tells you which of YOUR stories fits which question.

The 11 Rounds — typical NVIDIA process for Senior DevOps/SRE:

1. Recruiter Screen (30-45 min) — fit + comp + work auth
2. Online Assessment (60-90 min) — HackerRank Linux + Python
3. Kubernetes Deep-Dive (60 min) — live cluster diagnosis
4. Linux / OS / Networking (60 min) — first-principles debugging
5. CI/CD & GitOps (60 min) — pipelines, GitHub Actions, Argo CD
6. Terraform & AWS (60 min) — IaC, multi-account architecture
7. Observability & SRE (45 min) — SLOs, alerts, incident response, CVE handling
8. Python Coding & DS (60-90 min) — DevOps-flavored algorithms
9. System Design (60-90 min) — infrastructure at scale
10. Scenarios / War Stories (45-60 min) — production crises
11. Behavioral / HR (45-60 min) — culture, growth, comp

Plus: Workflows section (production debug playbooks) + Cheatsheet appendix (commands, frameworks, mental models).

Study plan:

2 weeks out: Rounds 1, 5, 6, 7 — your strongest areas, build confidence.

10 days: Rounds 3, 4 — K8s + Linux deep review.

1 week: Round 2 — practice OA problems timed.

4 days: Rounds 8, 9 — coding + system design (whiteboard them).

2 days: Round 10 — practice scenarios out loud.

Day before: Round 11 — polish your STAR stories.

Morning of: Cheatsheet only. Don't learn new content.

Table of Contents

Round 1. Recruiter Screen (30-45 min)	14 Q&A
Round 2. Online Assessment / HackerRank (60-90 min)	14 Q&A
Round 3. Kubernetes & Containers Deep-Dive (60 min)	18 Q&A
Round 4. Linux, OS, and Networking Deep-Dive (60 min)	17 Q&A
Round 5. CI/CD, GitOps, GitHub Actions, Build Systems (60 min)	13 Q&A
Round 6. Terraform, IaC, AWS Architecture (60 min)	14 Q&A
Round 7. Observability, SRE, Incident Response (45 min)	12 Q&A
Round 8. Python Coding & Data Structures (60-90 min)	18 Q&A
Round 9. System Design (60-90 min)	8 Q&A
Round 10. Scenario-Based / War Stories (45-60 min)	15 Q&A
Round 11. Behavioral / HR / Hiring Manager (45-60 min)	20 Q&A
Workflows. Production debug playbooks	8 workflows
Cheatsheet. Commands, frameworks, mental models	Quick reference

Round 1: Recruiter Screen (30-45 min)

Initial phone call with an NVIDIA recruiter. Tests: basic role fit, comp expectations, work auth, why-NVIDIA, high-level resume walk-through. Not technical. The recruiter is filtering for genuine interest, level-appropriate experience, and red flags. The role is Senior DevOps for NVIDIA AI Inference Operations (Dynamo + NIXL products). Frame your Cisco IQ AI platform work, OpenShift→EKS migration, and GenAI infrastructure delivery as DIRECTLY parallel to what NVIDIA needs.

Recruiter

Q1. Tell me about yourself.

A: Keep it tight: 60-90 seconds. 'I'm a Software Engineering Technical Leader at Cisco Systems, currently working on the Cisco IQ AI platform — a unified, AI-powered customer lifecycle platform. My focus is cloud and platform engineering: I'm leading the migration of our AI/ML inference workloads from on-prem OpenShift to multi-environment AWS EKS, including the CX AI Assistant, sentiment analysis pipelines, and LangSmith evaluation infrastructure. Before this, I built the CXTM platform from scratch — VPCs, EKS clusters, Karpenter autoscaling, ArgoCD multi-account rollout, cross-account NLB-to-ALB routing across US, EU, and APAC regions. I have my AWS Solutions Architect Pro, both Kubernetes certs (CKA and CKAD), and Terraform Associate. I'm drawn to NVIDIA because the AI Inference Operations role is exactly what I've been building toward — running the platform that powers production inference at the scale only NVIDIA operates at.'

■ **Tip:** Don't list every job — lead with current role, highlight ONE marquee project, end with why-NVIDIA. The recruiter will ask follow-ups on whatever sounds interesting.

Recruiter

Q2. Why NVIDIA? Why this specific role?

A: Three angles. 'First, infrastructure scale and impact. The CX AI Assistant I help operate at Cisco serves tens of thousands of internal users. NVIDIA AI Inference Operations runs the platform behind Dynamo and NIXL — used by every major AI lab on the planet. That's a 100x leverage opportunity for the same skills. Second, the technology overlap is exact: Kubernetes-at-scale, Ansible/Terraform, CI/CD, observability with Prometheus and OTEL, security including CVE handling and Wiz integrations — that's literally my current daily stack. Third, NVIDIA's culture of engineering autonomy and ownership — taking full responsibility for systems end-to-end — matches how I work. I've owned the OpenShift-to-EKS migration end-to-end at Cisco; I'd bring the same ownership to NVIDIA's inference infrastructure.'

■ **Tip:** Read the JD line by line and map every requirement to something in your resume. Mention 'Dynamo' or 'NIXL' if appropriate — shows you read the JD.

Recruiter

Q3. Walk me through your resume.

A: Reverse chronological, with depth on Cisco IQ. 'I've been at Cisco Systems as a Software Engineering Technical Leader since November 2023. I'm in the Cloud and Platform Engineering team for Cisco IQ — that's our unified AI-powered customer lifecycle platform. My biggest current initiative is OpenShift-to-EKS migration for the AI Assistant workloads: standardizing External Secrets with IRSA, EFS CSI drivers, APISIX ingress, kube-prometheus-stack, Kyverno, Splunk OTEL collectors, and Wiz security. We're improving operational efficiency by about 50%. Before that, I built the CXTM platform foundation — Terraform/Terragrunt for VPC, EKS, Karpenter, ArgoCD, External Secrets, multi-account routing across three regions. Earlier in the same role, I designed the GenAI platform itself — LangChain and LangSmith pipelines on Kubernetes for sentiment analysis and renewals intelligence, with secure secret management via Vault. Before Cisco I had 11 years at Cisco's customer support and release engineering tracks — release management, AppDynamics/Prometheus monitoring, incident response for high-revenue platforms with 99.99% uptime requirements. My education: Master of Computer Applications from University of Madras, plus a Post Graduate Program in AI/ML from UT. AWS SA Pro, CKA, CKAD, Terraform Associate.'

■ **Tip:** Spend 60% of the time on Cisco IQ — that's what NVIDIA cares about. Treat the earlier roles as supporting context showing reliability/SRE depth.

Recruiter

Q4. What's the most impactful thing you've shipped recently?

A: Pick ONE concrete recent win with measurable impact. 'The ArgoCD multi-account rollout for our apps-platform AWS account, Q1 2026. We needed end-to-end ArgoCD GitOps for a new EKS cluster sitting in a dedicated AWS account, separate from our shared services edge VPC. I delivered the entire sequence as a multi-PR rollout: DNS subdomain, ACM cert, cross-account security groups, Helm/IAM/Secrets integration, public ALB-to-cross-account NLB target group, ALB tag hygiene where I drove OPA warnings from 12 down to 0, A-record alias, and edge-VPC routing into the apps-platform account. The pattern is now reusable for every new cluster onboarding — that's the real win. Also Q1: I recovered a non-prod EKS cluster — diagnosed Fargate CoreDNS isolation caused by a missing node-SG to cluster-primary-SG ingress rule, restored 36 of 36 pods across 11 nodes, and produced a 29-section resource inventory with five supporting PRs so the next person can recover it in minutes, not hours.'

■ **Tip:** Measurable impact + reusability + senior judgment. Don't just say 'fixed a bug' — say 'fixed a class of bugs and codified the fix.'

Recruiter

Q5. What's your current compensation?

A: Three options depending on local law and your comfort. (1) Decline politely: 'I'd rather discuss what's competitive for the role than anchor on what I make today. Based on the level and location, I'd expect this role to be in the X-Y range — does that align with what NVIDIA is offering?' (2) Share total comp only: 'My current total comp is around \$X all-in including base, RSUs, and bonus. I'm looking for a meaningful step up given the scope of this role.' (3) State target only: 'I'd be targeting around \$X total comp for this level, including base, equity refresh, and signing.' In California, asking current comp is illegal — they'll ask expectations instead.

■ **Tip:** Research NVIDIA L5/L6 ranges on [levels.fyi](#) BEFORE the call. For Senior DevOps in CA, expect base \$180-240K, total \$300-450K. Anchor high but realistic.

Recruiter

Q6. What are your salary expectations?

A: 'Based on levels.fyi and Glassdoor data for Senior DevOps Engineers at NVIDIA in California, I'm seeing total comp ranges from \$300K to \$450K. Given my AWS SA Pro, both Kubernetes certs, and 12+ years of progressively senior infrastructure work, I'd target the upper half — base around \$210-230K with appropriate equity refresh and signing bonus. But I'm flexible on the split — what matters is total comp and the team. What range did NVIDIA budget for this position?'

■ **Tip:** Always pivot back to them with a question. Never name a single number — always a range. And never lowball; you can come down, you can't go up.

Recruiter

Q7. Are you authorized to work in the US? Will you need sponsorship?

A: Honest, brief. If on H-1B: 'I'm currently on H-1B and would need transfer sponsorship — NVIDIA does sponsor for senior roles. My priority date is X.' If GC/citizen: 'I'm a [permanent resident / citizen] — no sponsorship needed.' Match your actual status. Don't volunteer extra detail.

■ **Tip:** NVIDIA sponsors actively for senior engineers. Don't downplay or apologize for needing sponsorship.

Recruiter

Q8. Why are you leaving Cisco?

A: Positive framing only. Never bash current employer. 'Cisco has been a great place to grow — I've gone from customer support engineer to technical leader over 14 years, and I've owned major platform builds. But the IQ platform work has matured to the point where the most exciting frontier is no longer at Cisco — it's at NVIDIA, where every modern AI inference workload runs. I want my platform engineering to directly shape the infrastructure powering global AI. That's a step-change in scope I can only get by moving.'

■ **Tip:** *Pull-toward-NVIDIA framing > push-away-from-Cisco framing. Hiring managers smell complainers a mile away.*

Recruiter

Q9. Are you interviewing elsewhere?

A: 'Yes, I'm in conversations with a few other teams at top tech companies, but I'm being selective about where I apply — NVIDIA is a top priority because of the specific overlap between AI inference operations and the platform engineering work I've been doing.' If pressed: 'I'd rather keep specifics confidential out of respect for those processes.'

■ **Tip:** *Saying 'only you' weakens you. Listing competitors strengthens you but don't name them.*

Recruiter

Q10. What's your timeline for making a decision?

A: 'I'm not in a rush — I want to make the right choice. That said, I expect to have a decision-ready offer or two within 4-6 weeks. If NVIDIA moves at a similar pace, that aligns well. Standard notice from Cisco would be 2 weeks, with up to 4 if there's transition support they'd want.'

Recruiter

Q11. What kind of work environment do you thrive in?

A: 'I do my best work when I have ownership over a problem end-to-end, when the team is high-judgment and senior, and when the work has measurable customer impact. At Cisco IQ I own the migration architecture, write the Terraform, do the cluster diff analysis, lead the PR reviews — that vertical ownership is where I'm most productive. I also value teams that move fast but with discipline — code review, post-incident reviews, runbooks that actually get maintained. The autonomy described in NVIDIA's JD — taking full ownership of the DevOps landscape — is the work I want.'

Recruiter

Q12. Are you open to on-site, hybrid, or remote?

A: Match the JD — JR2017386 is listed as remote-eligible. 'I'm based in California so I'm flexible. The JD says remote — I work effectively remotely; I have a home office set up and 4+ years of remote/hybrid track record. I'm also open to hybrid if there's a Bay Area office option, especially for first 90 days of ramp-up.' Adjust based on your actual constraints.

Recruiter

Q13. Tell me about your certifications.

A: 'Four active that map directly to this role: AWS Certified Solutions Architect Professional — that's the senior-level AWS cert covering multi-account, multi-region architecture, security, networking, cost optimization. Certified Kubernetes Administrator and CKAD — cluster operations plus application deployment patterns; I use both daily. HashiCorp Terraform Associate — IaC and provider patterns. Earlier in my career I also got the VMware VCP6-DCV, VMware VCA-Cloud, ITIL, and Sun Certified Java Programmer. The recent four are the ones I use; the older ones show I treat certification as ongoing skill-validation, not just resume garnish.'

■ **Tip:** *Bringing up multiple certs is fine but don't list ALL of them in detail — pick the most relevant 2-3 and breeze past the rest.*

Q14. What questions do you have for me?

A: Always have 3-5 ready. Recruiter-appropriate questions: (1) 'What does the AI Inference Operations team's quarterly priority look like — is it new platform buildout, scaling existing systems, or migration?' (2) 'Who would I be reporting to? Engineering manager, principal engineer, or director?' (3) 'How does NVIDIA structure the technical-track vs management-track at the Senior level?' (4) 'What does success in the first 6 months look like for this role?' (5) 'What's the interview process from here — how many rounds, what topics?'

■ **Tip:** NEVER say 'I have no questions.' Saying that is the single biggest red flag in recruiter screens.

Round 2: Online Assessment / HackerRank (60-90 min)

NVIDIA OA usually combines: Linux command-line operations, Python coding (1-2 medium problems), and sometimes basic DevOps / system administration scenarios. For a Senior DevOps role, expect MORE Linux/scripting weight and LESS pure algorithmic DSA than for SDE roles. The bar is correctness AND production-quality code: handle edge cases, write readable code, comment intentionally. NVIDIA HackerRank typically gives ~60-90 minutes for 2-4 problems.

OA - Linux

Q1. Find all files larger than 100MB in /var that were modified in the last 7 days, and print their sizes in human-readable format sorted descending.

A: Combine find with -size and -mtime, then format. Multiple correct approaches — show you understand find flags.

```
# One-liner
find /var -type f -size +100M -mtime -7 -exec du -h {} + 2>/dev/null | sort -rh
```

```
# More portable / safer for spaces in filenames
find /var -type f -size +100M -mtime -7 -print0 2>/dev/null | \
  xargs -0 du -h | sort -rh
```

```
# Why each flag:
# -type f      : regular files only (exclude directories)
# -size +100M  : larger than 100 megabytes
# -mtime -7    : modified within last 7 days (negative = within)
# -print0/xargs : handle filenames with spaces/newlines safely
# sort -rh     : reverse, human-numeric (so 1G > 100M correctly)
```

■ **Tip:** On NVIDIA's actual platform, /var can have files owned by root — pipe stderr to /dev/null to avoid permission noise.

OA - Linux

Q2. Count the number of HTTP 5xx errors per minute from an nginx access log, output top 10 minutes with most errors.

A: Awk for parsing, sort + uniq for counting.

```
# Assume nginx default log format:
# 192.168.1.1 - - [16/May/2026:10:23:45 +0000] "GET /api HTTP/1.1" 503 0 "-" "curl"
```

```
awk '$9 ~ /^[0-9][0-9]$/' {
  # Extract [16/May/2026:10:23 — strip seconds
  split($4, t, ":")
  print substr($4, 2) ":" t[2] ":" t[3]
}' /var/log/nginx/access.log \
  | sort | uniq -c | sort -rn | head -10
```

```
# Alternative with grep + cut for simpler logs:
grep -E ' 5[0-9]{2}' access.log \
  | awk '{print $4}' \
  | cut -d: -f1-3 \
  | sort | uniq -c \
  | sort -rn | head -10
```

■ **Tip:** \$9 is the status code in nginx default. If asked about apache, log format differs slightly. Mention the assumption out loud.

OA - Linux

Q3. Show me the top 5 processes by memory usage, with PID, user, %mem, and command.

A: ps with the right format flags, sorted, head.

```
ps -eo pid,user,%mem,%cpu,comm --sort=-%mem | head -6
# head -6 because line 1 is the header
```



```
# More detailed with full command line:
ps -eo pid,user,%mem,rss,vsz,comm,args --sort=-%mem | head -6

# To find by hostname/container in K8s:
kubectl top pods --all-namespaces --sort-by=memory | head -10
```

■ **Tip:** *RSS = resident set size (actual RAM). VSZ = virtual size (includes swapped/mapped). For OOM debugging, RSS matters.*

OA - Linux

Q4. Find which process is using port 443.

A: Multiple tools; know all of them.

```
# Modern (recommended): ss
sudo ss -lntp | grep ':443'

# Traditional: netstat (may not be installed on newer systems)
sudo netstat -lntp | grep ':443'

# Definitive: lsof
sudo lsof -i :443
sudo lsof -nP -iTCP:443 -sTCP:LISTEN

# Just the PID:
sudo fuser 443/tcp

# Then: ps -fp <PID> to see the full process
```

■ **Tip:** *On containers/locked-down systems, ss is most reliable. fuser is fastest for 'just give me the PID.'*

OA - Linux

Q5. A disk is full but du shows much less than df. What's happening?

A: Classic interview scenario. Caused by deleted-but-still-open files. Process holds an open file descriptor to a file that was unlinked — the inode survives until the FD closes.

```
# Diagnose:
df -h /var/log
# /dev/xvda1: 100G used / 100G total

du -sh /var/log
# /var/log: 5G

# 95G missing! Find deleted-open files:
lsof +L1
# Shows files with link count == 0 (deleted) still open

# Or filter to a specific FS:
lsof +L1 /var

# Fix options:
# 1. Restart the process holding the FD (most common, but service downtime)
# 2. Truncate via /proc:
#   > /proc/<PID>/fd/<FDNUM>
# 3. If process is OK to kill: kill -HUP <pid> (graceful)

# Common culprits: log files rotated but app didn't reopen,
# Java processes with verbose logging, syslog
```

■ **Tip:** *Mention this pattern as a postmortem story: 'I've seen this in production with Splunk forwarder, fixed with logrotate copytruncate + service HUP.'*

OA - Linux

Q6. Write a one-liner to find which file changed most recently in /etc, going down 3 levels.

```
find /etc -maxdepth 3 -type f -printf '%T@ %p\n' 2>/dev/null \
| sort -rn | head -1 | cut -d' ' -f2-

# Breakdown:
# -maxdepth 3      : limit recursion
```

```
# -type f          : files only
# -printf '%T@ %p\n' : epoch-mtime followed by path
# sort -rn         : reverse numeric (most recent first)
# head -1          : take the newest
# cut -d' ' -f2-    : strip the timestamp, keep path

# Alt with explicit timestamp display:
find /etc -maxdepth 3 -type f -printf '%TY-%Tm-%Td %TH:%TM %p\n' \
  2>/dev/null | sort -r | head -5
```

OA - Python

Q7. Given a list of log lines, each formatted as 'TIMESTAMP LEVEL MESSAGE', return the top N most frequent ERROR messages.

A: Counter + filter. The classic.

```
from collections import Counter
from typing import List, Tuple

def top_errors(log_lines: List[str], n: int = 10) -> List[Tuple[str, int]]:
    """
    Returns the top-N most frequent ERROR messages.
    Each log line: 'TIMESTAMP LEVEL MESSAGE'
    LEVEL is the second whitespace-separated token.
    MESSAGE is everything after the level.
    """
    errors = Counter()
    for line in log_lines:
        parts = line.split(maxsplit=2)
        if len(parts) < 3:
            continue # malformed
        _, level, message = parts
        if level == 'ERROR':
            errors[message.strip()] += 1
    return errors.most_common(n)

# Test
sample = [
    '2026-05-16T10:00:00 ERROR Database connection timeout',
    '2026-05-16T10:00:01 INFO User logged in',
    '2026-05-16T10:00:02 ERROR Database connection timeout',
    '2026-05-16T10:00:03 ERROR Out of memory',
]
print(top_errors(sample, n=5))
# [('Database connection timeout', 2), ('Out of memory', 1)]
```

■ **Tip:** Always add type hints in NVIDIA OA — they grade for production-quality code, not just correctness.

OA - Python

Q8. Parse a Kubernetes pod YAML manifest and return a list of (container_name, image) tuples. Use only stdlib if no PyYAML.

A: If PyYAML available, use `yaml.safe_load`. Otherwise, do a defensive parser. Most NVIDIA environments have PyYAML.

```
import yaml
from typing import List, Tuple

def extract_containers(yaml_text: str) -> List[Tuple[str, str]]:
    """Extract (container_name, image) from a Pod or Deployment manifest."""
    doc = yaml.safe_load(yaml_text)
    if doc is None:
        return []

    # Navigate to spec.containers – varies by kind
    kind = doc.get('kind', '')
    if kind == 'Pod':
        containers = doc.get('spec', {}).get('containers', [])
    elif kind in ('Deployment', 'StatefulSet', 'DaemonSet', 'Job'):
        containers = (doc.get('spec', {}).get('template', {}).get('spec', {}).get('containers', []))

    return [(c['name'], c['image']) for c in containers]
```

```

else:
    # Try both paths defensively
    containers = (doc.get('spec', {}).get('containers')
                  or doc.get('spec', {}).get('template', {}))
                  .get('spec', {}).get('containers', []))

    return [(c.get('name', ''), c.get('image', '')) for c in containers]

# Multi-document YAML (--- separator):
def extract_from_multidoc(yaml_text: str) -> List[Tuple[str, str]]:
    results = []
    for doc in yaml.safe_load_all(yaml_text):
        if doc:
            results.extend(extract_containers(yaml.dump(doc)))
    return results

```

■ **Tip:** *K8s manifests come in many forms (Pod, Deployment, multi-document). Senior expectation: handle all of them.*

OA - Python

Q9. Implement a thread-safe LRU cache with capacity N.

A: `OrderedDict` + `threading.Lock`. Don't reinvent; understand the pattern.

```

from collections import OrderedDict
from threading import Lock
from typing import Any, Optional, Hashable

class LRUCache:
    def __init__(self, capacity: int):
        if capacity <= 0:
            raise ValueError('capacity must be positive')
        self.capacity = capacity
        self._cache: OrderedDict = OrderedDict()
        self._lock = Lock()

    def get(self, key: Hashable) -> Optional[Any]:
        with self._lock:
            if key not in self._cache:
                return None
            self._cache.move_to_end(key)
            return self._cache[key]

    def put(self, key: Hashable, value: Any) -> None:
        with self._lock:
            if key in self._cache:
                self._cache.move_to_end(key)
            self._cache[key] = value
            if len(self._cache) > self.capacity:
                self._cache.popitem(last=False)

    def __len__(self):
        with self._lock:
            return len(self._cache)

# Alternative: functools.lru_cache is NOT thread-safe for arbitrary
# objects across threads – it has a CPython-level lock but only for
# the dict operations. For true production: use cachetools.LRUCache
# or roll your own as above.

```

■ **Tip:** *If asked about thread-safety in Python, mention the GIL (CPython): single-byte-code operations are atomic, but multi-step operations (check-then-set) need explicit locks. NVIDIA tests this.*

OA - Python

Q10. Given a directory tree, return the total size of all .log files, but skip any directory named 'tmp' or 'cache'.

A: `os.walk` with pruning via the `dirnames` list.

```

import os
from pathlib import Path

def total_log_size(root: str) -> int:
    """Sum bytes of all .log files under root, skipping tmp/cache dirs."""

```

```

EXCLUDE = {'tmp', 'cache', '.git', '__pycache__'}
total = 0

for dirpath, dirnames, filenames in os.walk(root):
    # Prune in-place – os.walk uses this for next iteration
    dirnames[:] = [d for d in dirnames if d not in EXCLUDE]

    for f in filenames:
        if f.endswith('.log'):
            full = os.path.join(dirpath, f)
            try:
                total += os.path.getsize(full)
            except OSError:
                continue # broken symlink, race condition
    return total

# pathlib version (more pythonic, slightly slower):
def total_log_size_pathlib(root: str) -> int:
    EXCLUDE = {'tmp', 'cache'}
    total = 0
    for p in Path(root).rglob('*.log'):
        if any(part in EXCLUDE for part in p.parts):
            continue
        try:
            total += p.stat().st_size
        except OSError:
            continue
    return total

# rglob is simpler but doesn't prune – it walks tmp/ then filters.
# For huge trees, os.walk with dirnames[:] pruning is much faster.

```

■ **Tip:** *Mutating `dirnames[:]` = ... in-place is the canonical trick. Plain `dirnames` = ... won't work — `os.walk` reads its own reference.*

OA - Python

Q11. Write a function that retries a callable with exponential backoff. Configurable retries, base delay, max delay, and an optional list of retryable exceptions.

A: Production pattern. Show jitter awareness.

```

import time
import random
from typing import Callable, Type, Tuple, Any

def retry_with_backoff(
    func: Callable[[], Any],
    *,
    max_attempts: int = 5,
    base_delay: float = 1.0,
    max_delay: float = 60.0,
    retryable: Tuple[Type[BaseException], ...] = (Exception,),
    jitter: bool = True,
) -> Any:
    """
    Call func() with exponential backoff on retryable exceptions.

    Delay schedule: base * 2^(attempt-1), capped at max_delay.
    Optional jitter spreads load (avoid thundering herd).
    Raises the last exception if all attempts exhausted.
    """
    last_exc = None
    for attempt in range(1, max_attempts + 1):
        try:
            return func()
        except retryable as e:
            last_exc = e
            if attempt == max_attempts:
                break
            delay = min(base_delay * (2 ** (attempt - 1)), max_delay)
            if jitter:
                delay = delay * (0.5 + random.random()) # 0.5x-1.5x
    raise last_exc

```

```

        time.sleep(delay)
    raise last_exc

# Decorator version (more idiomatic for production):
from functools import wraps

def retry(max_attempts=5, base_delay=1.0, max_delay=60.0,
         retrieable=(Exception,), jitter=True):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            return retry_with_backoff(
                lambda: fn(*args, **kwargs),
                max_attempts=max_attempts, base_delay=base_delay,
                max_delay=max_delay, retrieable=retrieable, jitter=jitter
            )
        return wrapper
    return decorator

# Usage:
@retry(max_attempts=3, retrieable=(ConnectionError, TimeoutError))
def fetch(url): ...

```

■ **Tip:** Mention jitter unprompted — shows production maturity. Without jitter, all clients retry at the same time = thundering herd = makes the outage worse.

OA - Python

Q12. Parse a structured JSON log stream from stdin, line-by-line, and emit a summary of error rates per service over the last 5 minutes.

A: Streaming pattern; don't load everything into memory.

```

import sys
import json
from collections import defaultdict, deque
from datetime import datetime, timedelta

WINDOW = timedelta(minutes=5)

def main():
    # service -> deque of (timestamp, is_error)
    events = defaultdict(deque)

    for line in sys.stdin:
        line = line.strip()
        if not line:
            continue
        try:
            rec = json.loads(line)
        except json.JSONDecodeError:
            print(f'skipping malformed line: {line[:80]}', file=sys.stderr)
            continue

        service = rec.get('service', 'unknown')
        ts_str = rec.get('timestamp')
        if not ts_str:
            continue

        try:
            ts = datetime.fromisoformat(ts_str.replace('Z', '+00:00'))
        except ValueError:
            continue

        is_error = rec.get('level', '').upper() in ('ERROR', 'CRITICAL')
        events[service].append((ts, is_error))

        # Drop old events outside the 5-min window
        cutoff = ts - WINDOW
        while events[service] and events[service][0][0] < cutoff:
            events[service].popleft()

    # Emit summary
    for service in sorted(events):
        total = len(events[service])

```

```

        errors = sum(1 for _, e in events[service] if e)
        rate = (errors / total * 100) if total else 0.0
        print(f'{service}: {errors}/{total} errors ({rate:.1f}%)')

if __name__ == '__main__':
    main()

# Test:
# cat << 'EOF' | python3 monitor.py
# {"timestamp": "2026-05-16T10:00:00Z", "service": "api", "level": "INFO"}
# {"timestamp": "2026-05-16T10:00:05Z", "service": "api", "level": "ERROR"}
# EOF

```

■ **Tip:** Streaming + deque pruning = memory-bounded regardless of input size. NVIDIA OA might pipe a million log lines in.

OA - DevOps

Q13. Write a Dockerfile for a Python Flask app that follows best practices.

A: Show: multi-stage build, non-root user, pinned versions, slim base, healthcheck, no secrets, layer caching.

```

# syntax=docker/dockerfile:1.7

# Stage 1: Build dependencies
FROM python:3.12-slim AS builder

WORKDIR /build

# Copy only requirements first for layer caching
COPY requirements.txt .

# Install to a venv we'll copy to final stage
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir -r requirements.txt

# Stage 2: Runtime
FROM python:3.12-slim AS runtime

# Create non-root user
RUN groupadd --gid 1000 app && \
    useradd --uid 1000 --gid app --no-create-home --shell /bin/false app

WORKDIR /app

# Copy venv from builder
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copy app code (do this LAST so source changes don't bust dep cache)
COPY --chown=app:app . /app

# Drop privileges
USER app

# Document the port (doesn't actually expose it)
EXPOSE 8000

# Healthcheck for orchestrator
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')" || exit 1

# Use exec form, not shell form (so signals propagate)
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "4", "app:app"]

```

■ **Tip:** Things NVIDIA scores: non-root, multi-stage, no secrets in image, signals propagate (exec form), pinned base tag (3.12-slim not latest), healthcheck, layer ordering for cache.

OA - DevOps

Q14. Given a running Kubernetes pod that's in CrashLoopBackOff, list the kubectl commands you'd run in order to diagnose.

```

# 1. Confirm the state and get basic info
kubectl get pod <pod-name> -n <namespace>
kubectl describe pod <pod-name> -n <namespace>
# Look at: Events (bottom), Restart Count, Last State, Exit Code

# 2. Logs – current AND previous (the crash is in --previous)
kubectl logs <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace> --previous

# For multi-container:
kubectl logs <pod-name> -n <namespace> -c <container-name> --previous

# 3. If it's a startup issue, check the entry point/command
kubectl get pod <pod-name> -n <namespace> -o yaml | grep -A 5 'command\|args\|image:'

# 4. Check resources – is it OOMKilled?
kubectl get pod <pod-name> -n <namespace> -o jsonpath='{.status.containerStatuses[*].lastState}'
# Look for "reason": "OOMKilled"

# 5. Check secrets/configmaps it depends on
kubectl get pod <pod-name> -o yaml | grep -A 3 'secretKeyRef\|configMapKeyRef'
kubectl get secret <secret-name> -n <namespace>
kubectl get cm <cm-name> -n <namespace>

# 6. Probe configuration – bad liveness probe is a classic cause
kubectl get pod <pod-name> -o yaml | grep -A 10 'livenessProbe\|readinessProbe'

# 7. Resource limits
kubectl get pod <pod-name> -o yaml | grep -A 5 'resources:'

# 8. Try to exec in if it's running just long enough
# (rare but useful – sometimes container starts then dies after a few seconds)
kubectl exec -it <pod-name> -- sh
# Or use 'kubectl debug' for ephemeral debug container (1.25+):
kubectl debug -it <pod-name> --image=busybox --target=<container>

# 9. Check node health
kubectl describe node <node-name>
kubectl top node

# 10. Wider context
kubectl get events -n <namespace> --sort-by=.lastTimestamp | tail -20

```

■ **Tip:** The order matters in an interview. Senior answer: describe → logs (both!) → resources → probes → secrets → exec/debug. NOT: kubectl get pods, kubectl get pods, kubectl get pods.

Round 3: Kubernetes & Containers Deep-Dive (60 min)

This is the heaviest technical round for an NVIDIA SRE/DevOps role. The interviewer often shares their screen with a live Kubernetes cluster and asks you to diagnose failures, design network policies, configure scheduling. Hands-on. They want to see you reason out loud while typing `kubectl` commands. Bring up your Cisco IQ EKS experience whenever relevant — multi-account ArgoCD, Karpenter, External Secrets, APISIX ingress.

K8s - Core

Q1. Walk me through what happens when you run ``kubectl apply -f deployment.yaml``.

A: Trace the full lifecycle: client → API server → etcd → controllers → scheduler → kubelet → container runtime. (1) `kubectl` serializes the YAML, sets last-applied-configuration annotation, and POSTs to API server. (2) API server authenticates the request (TLS cert / SA token / OIDC), authorizes via RBAC, runs admission controllers (mutating then validating, including any Kyverno/OPA policies), then validates against the OpenAPI schema. (3) Stores the Deployment object in etcd. (4) Deployment controller (in kube-controller-manager) watches Deployments. Sees the new spec. Creates a ReplicaSet object (sent through the API server, stored in etcd). (5) ReplicaSet controller sees the new RS, creates Pod objects matching replicas count. (6) Scheduler watches for unscheduled pods. For each, evaluates filtering (node selectors, taints/tolerations, resource requests, affinity rules, topology spread) and scoring (least-loaded, preferred affinity). Binds pod to node by updating Pod.spec.nodeName via API server. (7) Kubelet on that node watches for pods bound to it. Pulls the image via the container runtime (containerd in modern K8s). Creates the pod sandbox (pause container for network namespace). Starts container(s). Reports status back to API server. (8) Pod Status field flips to Running once kubelet reports.

■ **Tip:** If interviewer asks 'where does CNI fit?' — kubelet asks the CNI plugin to set up pod network namespace BEFORE starting containers. CSI fits the same way for volumes.

K8s - Core

Q2. What's the difference between a Deployment, StatefulSet, DaemonSet, and Job? When use each?

A: Deployment: stateless replicated workloads. Pods are interchangeable; replica count drives scale. Use for web servers, API services, stateless workers. Rolling updates with `maxSurge/maxUnavailable`. StatefulSet: workloads needing stable identity and stable storage. Each pod gets a stable name (app-0, app-1), stable DNS (app-0.headless-svc), and stable PVC (each pod has its own). Use for databases, Kafka, Elasticsearch, anything with replication topology. Pods come up in order; scale down in reverse order. DaemonSet: one pod per matching node. Use for node-level agents — log collectors (Fluent Bit), metrics agents (node-exporter), CNI/CSI plugins, GPU device plugins (nvidia-device-plugin runs as DaemonSet on every GPU node). Job: run-to-completion. Use for batch processing, migrations, one-off tasks. CronJob = scheduled Jobs. For NVIDIA inference workloads, Jobs are common for training pre-checks and model conversion pipelines.

■ **Tip:** For NVIDIA: `nvidia-device-plugin` DaemonSet is critical for exposing GPUs as schedulable resources. The MIG (Multi-Instance GPU) configurations also use DaemonSets.

K8s - Hands-on

Q3. [Live cluster] A pod is stuck in Pending. Walk me through diagnosis.

A: Pending = scheduler can't place the pod. Systematic checklist.

```
# Step 1: Get the full picture
kubectl get pod <pod> -n <ns> -o wide
kubectl describe pod <pod> -n <ns>
# Scroll to Events section – scheduler decisions are logged there

# Common Pending reasons and how to verify each:
# REASON 1: Insufficient resources
```



```
# Events show: "0/N nodes available: insufficient cpu/memory"
kubectl describe nodes | grep -A 5 'Allocated resources'
kubectl top nodes
# Fix: Reduce requests, scale cluster, or wait for autoscaler/Karpenter

# REASON 2: Taints without tolerations
# Events: "node(s) had untolerated taint"
kubectl describe nodes | grep -i taint
kubectl get pod <pod> -o yaml | grep -A 5 tolerations
# Fix: Add toleration or remove taint

# REASON 3: NodeSelector / Affinity mismatch
# Events: "node(s) didn't match Pod's node affinity/selector"
kubectl get pod <pod> -o yaml | grep -A 10 -E 'nodeSelector|affinity'
kubectl get nodes --show-labels | grep <selector-label>
# Fix: Label nodes correctly or adjust selector

# REASON 4: PVC not bound
# Events: "0/N nodes available: pod has unbound immediate PersistentVolumeClaims"
kubectl get pvc -n <ns>
kubectl describe pvc <pvc-name> -n <ns>
# Common: storage class doesn't exist, or no PV available, or wrong access mode
# Fix: Create PV, fix storage class, check EBS CSI / EFS CSI driver health

# REASON 5: Image pull issues (technically ImagePullBackOff, but can manifest as Pending if init)
kubectl describe pod <pod> | grep -A 3 'Events'

# REASON 6: Quota exceeded
kubectl describe quota -n <ns>
# ResourceQuota or LimitRange blocking creation

# REASON 7: PodDisruptionBudget blocking (rare for Pending – usually shows on eviction)

# REASON 8: Karpenter / Cluster Autoscaler stuck
kubectl logs -n karpenter -l app.kubernetes.io/name=karpenter
# Look for: failures to launch nodes, AWS API errors, IRSA misconfig
```

■ **Tip:** Senior answer mentions Karpenter specifically — that's what NVIDIA uses or wants candidates to know. Mention your Cisco IQ Karpenter setup.

K8s - Hands-on

Q4. A pod is in CrashLoopBackOff. Diagnose end-to-end.

```
# Step 1: How many restarts? How fast is the loop?
kubectl get pod <pod> -n <ns>
# RESTARTS column shows count

kubectl describe pod <pod> -n <ns>
# 'Last State': what happened on the last crash
# 'Reason': OOMKilled, Error, Completed
# 'Exit Code':
# 0 = clean exit (probably misconfigured – app shouldn't exit)
# 1 = generic error (check logs)
# 137 = SIGKILL (often OOMKilled, look for that reason)
# 139 = SIGSEGV (segfault – corrupt binary or bad lib)
# 143 = SIGTERM (graceful shutdown – but if looping, something IS killing it)

# Step 2: Logs from THIS attempt and the PREVIOUS attempt
kubectl logs <pod> -n <ns>
kubectl logs <pod> -n <ns> --previous # critical for crash debugging
kubectl logs <pod> -n <ns> -c <container> --previous # multi-container

# Step 3: Common root causes by symptom

# CASE A: OOMKilled (Exit 137 + Reason OOMKilled)
# Container exceeded its memory limit
kubectl get pod <pod> -o yaml | grep -A 4 resources
# Fix: raise memory limit, fix the leak, profile heap

# CASE B: App can't reach a dependency
# Logs show connection timeout to DB / Redis / external API
# Check: Network Policy too restrictive, DNS broken, target down, IRSA missing
kubectl exec -it <pod> -- nslookup <dependency-service>
kubectl get netpol -n <ns>
```

```
# CASE C: Missing config/secret
# Logs: "key not found" or "env var unset" or "config file not readable"
kubectl get pod <pod> -o yaml | grep -E 'configMap|secret'
kubectl describe configmap <cm> -n <ns>
kubectl describe secret <secret> -n <ns>

# CASE D: Bad liveness probe
# Pod starts OK, then liveness fails → kubelet restarts it
kubectl get pod <pod> -o yaml | grep -A 10 livenessProbe
# Common: probe too aggressive, wrong path/port, slow-starting app
# Fix: add startupProbe, increase initialDelaySeconds, fix probe target

# CASE E: Permission denied / mount issues
# Logs show "permission denied" on file write
# Check: securityContext, runAsUser, volume permissions, PSP/PSA
kubectl get pod <pod> -o yaml | grep -A 5 securityContext

# CASE F: Application bug
# Logs show stack trace
# This is application's problem – file a bug, but as SRE you should
# add a circuit breaker / rate-limit the restart loop

# Step 4: Get a shell IF the pod stays up long enough
# Use ephemeral debug container (works even on crashing pods):
kubectl debug -it <pod> --image=busybox:1.28 --target=<container> -n <ns>
```

■ **Tip:** Mention exit codes specifically. Senior interviewers love hearing '137 = OOMKilled' without prompting.

K8s - Hands-on

Q5. [Live] Set up a NetworkPolicy that allows only the 'frontend' namespace to talk to pods labeled app=api in the 'backend' namespace on port 8080.

```
# Step 1: Confirm the CNI enforces NetworkPolicy
# Calico, Cilium, AWS VPC CNI w/ tigera, weave-net – most do
# Plain Flannel does NOT enforce. EKS VPC CNI requires enabling.
kubectl get pods -n kube-system | grep -E 'calico|cilium|aws-node'

# Step 2: Label the frontend namespace (so we can match it)
kubectl label namespace frontend purpose=app

# Step 3: Write the policy
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-api
  namespace: backend
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
    - Ingress
  ingress:
    - from:
      - namespaceSelector:
          matchLabels:
            purpose: app    # matches the label we set on frontend ns
      # Optionally also restrict by pod label inside frontend:
      # - podSelector:
      #   matchLabels:
      #     role: web
  ports:
    - protocol: TCP
      port: 8080
EOF

# Important gotcha #1: Without any NetworkPolicy, traffic is ALLOWED.
# As soon as ONE policy applies to a pod, that pod becomes default-DENY
# for the policyType (Ingress here). All allowed traffic must be listed.

# Gotcha #2: namespaceSelector matches NAMESPACE labels, not pod labels.
# Common bug: forgetting to label the source namespace.
```

```
# Gotcha #3: If you want to allow DNS (and you probably do):
# Need a separate Egress policy or add egress rule to coredns
# This is why teams often use Cilium with its global Network Policies.

# Step 4: Test
# From a pod in frontend ns:
kubectl run -it --rm test --image=nicolaka/netshoot -n frontend -- \
  curl -sf http://api.backend.svc.cluster.local:8080/

# From a pod in random other ns (should be blocked):
kubectl create ns other
kubectl run -it --rm test --image=nicolaka/netshoot -n other -- \
  curl -m 5 -sf http://api.backend.svc.cluster.local:8080/
# Should timeout
```

■ **Tip:** Mention 'as soon as one policy applies, the pod becomes default-deny' — this is the most common bug. Also mention you'd test with netshoot from both allowed AND denied sources.

K8s - Hands-on

Q6. Apply a taint to dedicate a node to GPU workloads, then make a Deployment tolerate it.

```
# This is exactly the pattern NVIDIA uses for GPU node pools.

# Step 1: Taint the GPU node so only GPU workloads land there
kubectl taint nodes gpu-node-1 dedicated=gpu:NoSchedule
# Effects:
#   NoSchedule       : prevents new pods (existing pods stay)
#   PreferNoSchedule : soft – try to avoid, but allowed if needed
#   NoExecute        : evicts existing pods that don't tolerate

# Step 2: Verify
kubectl describe node gpu-node-1 | grep -A 5 Taints

# Step 3: Add tolerations + nodeSelector to your workload
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: triton-inference
spec:
  replicas: 2
  selector:
    matchLabels:
      app: triton
  template:
    metadata:
      labels:
        app: triton
    spec:
      # Toleration: allows scheduling onto tainted node
      tolerations:
        - key: dedicated
          operator: Equal
          value: gpu
          effect: NoSchedule

      # nodeSelector: prefers/requires GPU node
      nodeSelector:
        nvidia.com/gpu.present: "true"
      # Or use nodeAffinity for soft prefs

      containers:
        - name: triton
          image: nvcr.io/nvidia/tritonserver:24.05-py3
          resources:
            limits:
              nvidia.com/gpu: 1    # request 1 GPU from device plugin
EOF

# Why both taint AND nodeSelector?
# - Taint REPELS unwanted pods from GPU nodes
# - nodeSelector ATTRACTS GPU pods TO GPU nodes
```

```
# Without taint: non-GPU pods can land on expensive GPU nodes
# Without nodeSelector: GPU pods might land on non-GPU nodes (if they tolerate the taint)

# Karpenter equivalent uses a NodePool with taints in the spec
# (Karpenter creates nodes pre-tainted)
```

■ **Tip:** Mention 'taint repels, nodeSelector attracts — you need both' — this distinction is often missed. For NVIDIA bring up nvidia-device-plugin specifically.

K8s - Hands-on

Q7. Implement a Horizontal Pod Autoscaler that scales on custom CloudWatch metric (requests per pod).

```
# Step 1: HPA needs metrics. Three sources:
#   1. metrics-server (CPU/memory)           - built-in
#   2. custom.metrics.k8s.io API             - needs adapter
#   3. external.metrics.k8s.io API           - needs adapter

# For CloudWatch metrics, install the CloudWatch metric adapter
# (or use Keda which is more flexible)

# Step 2: Install Keda (recommended for cloud metrics)
helm repo add keda https://kedacore.github.io/charts
helm install keda kedacore/keda -n keda --create-namespace

# Step 3: ScaledObject pointing to CloudWatch
cat <<EOF | kubectl apply -f -
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: inference-scaler
  namespace: default
spec:
  scaleTargetRef:
    name: inference-deployment
  minReplicaCount: 2
  maxReplicaCount: 100
  pollingInterval: 30          # seconds
  cooldownPeriod: 300         # don't scale down too fast
  triggers:
    - type: aws-cloudwatch
      metadata:
        namespace: AWS/ApplicationELB
        metricName: RequestCountPerTarget
        dimensionName: TargetGroup
        dimensionValue: targetgroup/inference-tg/abc123
        targetMetricValue: "100"    # scale when avg > 100 req/min/pod
        minMetricValue: "0"
        awsRegion: us-west-2
        identityOwner: pod          # use pod's IRSA role
EOF

# Step 4: IRSA so Keda can read CloudWatch
# Annotate the keda-operator service account with the role ARN
# Role policy: cloudwatch:GetMetricData, GetMetricStatistics

# Plain HPA equivalent (if you don't want Keda):
cat <<EOF | kubectl apply -f -
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: inference-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: inference-deployment
  minReplicas: 2
  maxReplicas: 100
  metrics:
    - type: External
      external:
        metric:
```

```

        name: aws_alb_request_count_per_target
        target:
          type: AverageValue
          averageValue: "100"
      behavior:
        scaleDown:
          stabilizationWindowSeconds: 300 # avoid flapping
          policies:
            - type: Percent
              value: 50
              periodSeconds: 60
        scaleUp:
          stabilizationWindowSeconds: 0 # scale up immediately
          policies:
            - type: Percent
              value: 100
              periodSeconds: 30
EOF

```

■ **Tip:** Keda is the modern answer. Mention `behavior.scaleDown.stabilizationWindowSeconds` — that's the senior detail that prevents flapping during quiet periods.

K8s - Hands-on

Q8. How does port mapping work in Docker? Compare bridge, host, and overlay networks.

```

# Bridge network (default for `docker run`)
# Container gets a private IP on docker0 bridge
# -p host_port:container_port creates an iptables DNAT rule

docker run -d -p 8080:80 nginx

# Behind the scenes:
# 1. Docker creates a veth pair: one end in container netns, other on docker0
# 2. Container gets IP like 172.17.0.2
# 3. iptables PREROUTING rule:
#    DNAT host:8080 -> 172.17.0.2:80
# 4. iptables MASQUERADE rule for outbound from container

# Verify:
docker inspect <container> | grep IPAddress
sudo iptables -t nat -L DOCKER -n
# Shows: DNAT tcp dpt:8080 to:172.17.0.2:80

# Host network — no isolation, container uses host's network stack
docker run -d --network host nginx
# Container IS on host network. -p is ignored.
# nginx binds directly to host:80
# Faster (no NAT), but loses port isolation. Used for daemons.

# Overlay network — multi-host, used by Swarm/K8s
docker network create -d overlay myoverlay
# Encapsulates packets in VXLAN, lets containers across hosts communicate.
# K8s doesn't use Docker's overlay — it uses CNI plugins (Calico, Cilium, AWS VPC CNI).

# Comparison:
# bridge   : default, isolated, NAT'd, port mapping needed.   Pros: secure, simple. Cons: NAT overhead.
# host     : no isolation, no NAT.                             Pros: fastest. Cons: port conflicts.
# overlay  : multi-host, VXLAN.                                Pros: distributed. Cons: complex, slower.
# macvlan  : container gets MAC on physical network.          Pros: looks like real host. Cons: switch config.
# none     : no networking at all.                             Pros: max isolation. Cons: no network!

```

■ **Tip:** If asked 'why does K8s not use Docker bridge?' — K8s pod networking is flat (every pod can reach every other), not NAT'd. That's the CNI plugin's job (Calico, Cilium, etc.).

K8s - Concepts

Q9. Difference between readiness, liveness, and startup probes.

A: Readiness: should this pod receive traffic? Failure removes pod from Service endpoints (doesn't restart). Use for: 'still warming up,' 'temporarily can't serve' (dependency unreachable). Liveness: is this container alive? Failure restarts the container. Use ONLY for deadlocks — situations where restart is the actual fix. Bad liveness probes cause outages: if your DB is slow, liveness fails, restarts pod, new pod also fails, infinite restart loop.

Startup: protects slow-starting apps. Liveness AND readiness are suspended until startup succeeds. Use for: Java apps that take 60s to JIT, ML model loading (huge for NVIDIA — model load can take minutes). Without startupProbe, slow apps hit liveness failures during boot and never come up. Production rule: readiness aggressive, liveness conservative (only for true deadlock), startup generous.

```
containers:
- name: model-server
  image: triton:24.05
  startupProbe:
    httpGet:
      path: /v2/health/ready
      port: 8000
    failureThreshold: 30      # 30 * 10s = 5 minutes to start
    periodSeconds: 10
  readinessProbe:
    httpGet:
      path: /v2/health/ready
      port: 8000
    periodSeconds: 5
    failureThreshold: 3      # ~15s out of rotation before traffic gone
  livenessProbe:
    httpGet:
      path: /v2/health/live  # different endpoint — only fails for deadlock
      port: 8000
    periodSeconds: 30
    failureThreshold: 3
    initialDelaySeconds: 60  # backup for startupProbe
```

■ **Tip:** Senior insight: 'My rule is — only use livenessProbe if you can describe the deadlock it's catching. If you can't, leave it off.' That separates seniors from juniors.

K8s - Concepts

Q10. Explain requests vs limits. Why is setting CPU limit often a bad idea?

A: Requests: reserved for scheduling. Sum of requests on a node must fit node allocatable. Affects QoS class. Limits: hard cap on usage. CPU limit > usage = throttled (CFS bandwidth control). Memory limit > usage = OOMKilled. The trap: CPU limits cause throttling EVEN WHEN NODE HAS FREE CPU. Your app sits idle most of the time, then suddenly needs a burst — limit kicks in, latency spikes. Production observation at large scale (Netflix, Google): many teams set CPU requests but no CPU limit. They guarantee the floor (requests) but let bursts happen. Memory limits are different: leaks need a cap. So pattern: requests = guaranteed, no CPU limit, memory limit = realistic+headroom. NVIDIA case: GPU resources are different — nvidia.com/gpu is integer-only, no fractional, no overcommit. Request = limit = exactly N GPUs.

■ **Tip:** Bring up the CPU CFS throttling research — it's a known anti-pattern at scale. NVIDIA engineers will know this.

K8s - Concepts

Q11. QoS classes: Guaranteed, Burstable, BestEffort.

A: Guaranteed: requests == limits for ALL resources on ALL containers. Highest priority — last to be evicted under node pressure. Use for production critical workloads. Burstable: at least one resource has request, but request != limit (or no limit). Medium priority. Most production workloads. BestEffort: no requests, no limits. Lowest priority — first evicted. Use only for ephemeral, non-critical work. Karpenter and the scheduler use QoS to decide eviction order during node pressure.

```
# Guaranteed
resources:
  requests:
    cpu: "1"
    memory: 2Gi
  limits:
    cpu: "1"
    memory: 2Gi

# Burstable (most common)
resources:
  requests:
    cpu: "500m"
    memory: 1Gi
```

```
limits:
  memory: 2Gi    # CPU limit omitted intentionally

# BestEffort (no requests/limits set anywhere)
```

K8s - Concepts

Q12. How does the Kubernetes scheduler decide where to place a pod?

A: Two phases: filtering then scoring. **FILTERING:** rules out unsuitable nodes. Checks include: PodFitsResources (CPU/mem/GPU requests), PodFitsHostPorts (port conflicts), MatchNodeSelector (labels), TaintToleration, VolumeZone (PV must be in same zone as node), PodAffinity/Anti-Affinity (must/should be near/far from other pods), NodeUnschedulable, PodTopologySpread. After filtering, you have a list of **FEASIBLE** nodes. **SCORING:** ranks feasible nodes by priority. Priorities include: LeastRequestedPriority (prefer empty nodes), BalancedResourceAllocation (avoid CPU-heavy with low mem), ImageLocalityPriority (prefer nodes with image cached), InterPodAffinityPriority, NodeAffinityPriority, TaintTolerationPriority. The highest-scored node wins. Bound by API call: pod.spec.nodeName = chosen-node. Kubelet on that node sees a pod assigned to it, starts it.

■ **Tip:** Mention 'PodTopologySpread' specifically — it's the modern way to spread replicas across AZs. NVIDIA uses this for GPU node pools.

K8s - Concepts

Q13. Explain Service types: ClusterIP, NodePort, LoadBalancer, ExternalName.

A: **ClusterIP:** virtual IP only reachable inside cluster. Default. iptables/IPVS rules on every node DNAT clusterIP:port to pod IPs. Used internally for service discovery. **NodePort:** ClusterIP + opens a high port (30000-32767) on every node. External traffic to any node:nodePort gets DNAT'd to pod. Rarely used directly in prod — usually via LoadBalancer. **LoadBalancer:** NodePort + cloud provider provisions external LB (ALB/NLB on AWS). For Kubernetes services on AWS, AWS Load Balancer Controller creates NLB/ALB with target group pointing to pods (via IP target type — bypasses NodePort entirely). **ExternalName:** CNAME record. Service.spec.externalName = 'external.example.com'. CoreDNS returns CNAME, pod resolves to external endpoint. Useful for: routing internal traffic to AWS RDS, an external SaaS, or a service in another cluster.

```
# ClusterIP — internal only
apiVersion: v1
kind: Service
metadata: {name: api}
spec:
  type: ClusterIP
  selector: {app: api}
  ports: [{port: 80, targetPort: 8000}]

# LoadBalancer with AWS NLB
apiVersion: v1
kind: Service
metadata:
  name: api-public
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: "external"
    service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: "ip"
    service.beta.kubernetes.io/aws-load-balancer-scheme: "internet-facing"
spec:
  type: LoadBalancer
  selector: {app: api}
  ports: [{port: 443, targetPort: 8000}]
```

K8s - Hands-on

Q14. How do you give a pod permissions to AWS resources without leaking long-lived keys?

A: **IRSA** — IAM Roles for Service Accounts. Maps a Kubernetes ServiceAccount to an AWS IAM role via OIDC.

```
# Step 1: Enable OIDC provider on EKS cluster (once per cluster)
eksctl utils associate-iam-oidc-provider --cluster=my-cluster --approve
```

```
# Step 2: Create IAM policy with the permissions you need
cat > policy.json <<EOF
```



```
{
  "Version": "2012-10-17",
  "Statement": [
    {"Effect": "Allow", "Action": "s3:GetObject", "Resource": "arn:aws:s3::my-bucket/*"}
  ]
}
EOF
aws iam create-policy --policy-name PodS3Read --policy-document file://policy.json
```

```
# Step 3: Create IAM role trusted by the SA
# eksctl simplifies this:
eksctl create iamserviceaccount \
  --cluster=my-cluster \
  --namespace=default \
  --name=s3-reader \
  --attach-policy-arn=arn:aws:iam::123:policy/PodS3Read \
  --approve
```

```
# This creates BOTH:
# - IAM role with trust policy:
#   "Principal": {"Federated": "arn:...oidc-provider/..."},
#   "Action": "sts:AssumeRoleWithWebIdentity",
#   "Condition": {"StringEquals": {""...:sub": "system:serviceaccount:default:s3-reader"}}
# - Kubernetes ServiceAccount with annotation:
#   eks.amazonaws.com/role-arn: arn:aws:iam::123:role/eksctl-...
```

```
# Step 4: Pod uses that ServiceAccount
apiVersion: v1
kind: Pod
spec:
  serviceAccountName: s3-reader # ← key line
  containers:
    - name: app
      image: amazon/aws-cli
      command: ["aws", "s3", "ls", "s3://my-bucket"]
```

```
# How it works at runtime:
# 1. EKS pod identity webhook injects env vars:
#   AWS_WEB_IDENTITY_TOKEN_FILE=/var/run/secrets/eks.amazonaws.com/serviceaccount/token
#   AWS_ROLE_ARN=arn:aws:iam::123:role/...
# 2. AWS SDK reads these env vars, calls sts:AssumeRoleWithWebIdentity
# 3. Returns temporary credentials, SDK uses them transparently
```

■ **Tip:** At Cisco IQ you set up IRSA — talk about that exact experience. Also mention EKS Pod Identity (the newer, simpler alternative to IRSA, GA since 2023).

K8s - Operations

Q15. How do you safely upgrade an EKS cluster from 1.27 to 1.30?

A: Multi-version skip not allowed — must go 1.27 → 1.28 → 1.29 → 1.30. Each step: control plane first, then nodes, validate.

```
# For EACH minor version step:
```

```
# Step 1: Pre-flight — scan for deprecated APIs
# Use pluto or kubent
pluto detect-files -d ./manifests
kubent --target-version v1.30.0
```

```
# Step 2: Read release notes
# Especially for: deprecations, behavioral changes, addon compatibility
```

```
# Step 3: Snapshot state
kubectl get all -A -o yaml > backup-pre-1.28.yaml
# Better: velero backup of full cluster state + PVs
```

```
# Step 4: Upgrade control plane
aws eks update-cluster-version --name my-cluster --kubernetes-version 1.28
# This takes 25-45 minutes. Control plane is HA — no API server outage in practice.
```

```
# Step 5: Update managed add-ons
# CoreDNS, kube-proxy, VPC CNI MUST be compatible with new control plane
aws eks describe-addon-versions --kubernetes-version 1.28 --addon-name vpc-cni
aws eks update-addon --cluster-name my-cluster --addon-name vpc-cni \
```



```
--addon-version v1.18.0-eksbuild.1 --resolve-conflicts OVERWRITE
```

```
# Step 6: Upgrade nodes – managed node group
# AWS does a rolling update: cordon, drain, terminate, replace
aws eks update-nodegroup-version --cluster-name my-cluster --nodegroup-name ng1
# Or with Karpenter – update the AMI in NodePool spec, drift detection handles rest
```

```
# Step 7: Validate
kubectl get nodes # all Ready, new version
kubectl get pods -A | grep -v Running # nothing weird
# Run your smoke tests / synthetic checks
```

```
# Step 8: Repeat for next version (1.28 → 1.29 → 1.30)
```

```
# Critical gotchas:
# - Don't upgrade prod first. Always: dev → stage → prod
# - kubeconfig clients might break. Update kubectl to match.
# - Custom controllers may break (CRD API moves, e.g. Cert-manager Issuer API)
# - PodSecurityPolicy removed in 1.25 – must migrate to PSA before
# - Pluto/kubent BEFORE the upgrade saves you
```

■ **Tip:** Mention the version-skip rule (max 1 minor at a time). Mention your real cluster upgrade experience from Cisco IQ if you have it.

K8s - Concepts

Q16. How does kube-proxy work? iptables vs IPVS mode.

A: kube-proxy runs on every node. Watches Services and Endpoints (or EndpointSlices in newer versions). Configures the kernel networking to implement Service routing. iptables mode (default): for each Service, kube-proxy writes iptables rules in the nat table. Cluster IP traffic gets DNAT'd to a randomly chosen pod IP from the endpoints list. Random selection = round-robin-ish. With many services, the iptables rule list grows linearly — performance degrades at scale (10K+ services becomes slow). IPVS mode: uses Linux IPVS (kernel-level Layer 4 LB). Hash-based lookup, O(1) regardless of service count. Better at scale. Supports algorithms: rr (round robin), lc (least conn), dh (dest hash), sh (src hash). Modern EKS uses iptables by default but IPVS is supported. Alternative: Cilium with eBPF replaces kube-proxy entirely — even better performance.

■ **Tip:** Mention 'at NVIDIA's scale, IPVS or eBPF (Cilium) makes a measurable difference' — shows you think about scale, not just correctness.

K8s - Concepts

Q17. Explain Helm and what makes a 'good' chart.

A: Helm is a package manager for Kubernetes. Chart = templated Kubernetes manifests + values for parameterization. Anatomy: Chart.yaml (metadata, version, dependencies), values.yaml (default config), templates/ (Go template YAML), charts/ (subcharts). helm install creates a Release (named instance). helm upgrade modifies it. helm rollback reverts. Good chart practices: (1) JSON schema (values.schema.json) for input validation. (2) Sensible defaults — chart works without overriding values. (3) Labels following Kubernetes recommended labels (app.kubernetes.io/name, /instance, /version, /component, /part-of, /managed-by). (4) Templates use helper functions in _helpers.tpl for reusable name/label logic. (5) Resource hints — securityContext, resources, probes, PDB included. (6) Use {{ include }} not {{ template }} for whitespace-sensitive output. (7) NEVER ship secrets in values — reference external secret stores. (8) Avoid hooks unless lifecycle ordering is essential.

■ **Tip:** For NVIDIA: mention you'd version charts in a repository (ChartMuseum, Harbor, Artifactory OCI registry) and that you deploy via Argo CD not via `helm install` directly in production.

K8s - Hands-on

Q18. How do you build a Docker image from a Dockerfile and push to ECR?

```
# Step 1: Authenticate Docker to ECR
aws ecr get-login-password --region us-west-2 \
  | docker login --username AWS --password-stdin \
  123456789.dkr.ecr.us-west-2.amazonaws.com

# Step 2: Create the repository if it doesn't exist
aws ecr create-repository --repository-name my-app --region us-west-2
```

```
# Step 3: Build with proper tag
docker build -t my-app:v1.2.3 .

# Step 4: Tag for ECR
docker tag my-app:v1.2.3 \
  123456789.dkr.ecr.us-west-2.amazonaws.com/my-app:v1.2.3

# Step 5: Push
docker push 123456789.dkr.ecr.us-west-2.amazonaws.com/my-app:v1.2.3

# In CI (GitHub Actions with OIDC, no long-lived keys):
- uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::123:role/ecr-push-role
    aws-region: us-west-2

- uses: aws-actions/amazon-ecr-login@v2

- name: Build and push
  run: |
    docker buildx build \
      --platform linux/amd64,linux/arm64 \
      --tag $ECR_REGISTRY/$IMAGE_NAME:${GITHUB_SHA::8} \
      --tag $ECR_REGISTRY/$IMAGE_NAME:latest \
      --push .

# Multi-arch is important — NVIDIA has both x86 and Grace (ARM)
```

■ **Tip:** Mention *buildx* for multi-arch — Grace Hopper / Grace Blackwell are ARM. NVIDIA cares.

Round 4: Linux, OS, and Networking Deep-Dive (60 min)

Heavy focus on Linux internals, networking, troubleshooting from first principles. NVIDIA SRE candidates are expected to debug at the kernel-adjacent level. Strong fundamentals trump tool knowledge. Reference your AppDynamics/Prometheus monitoring experience and your 99.99% uptime track record from earlier Cisco roles.

Linux

Q1. Walk me through what happens when you run `ls -la` from boot to output.

A: (1) Shell parses command into argv. (2) Shell looks up `ls` — checks aliases, functions, builtins, then PATH. Finds /usr/bin/ls. (3) Shell calls fork() — creates child process (COW memory). (4) Child calls execve('/usr/bin/ls', ['ls', '-la'], envp). Kernel loads ELF binary, sets up new memory map, starts at _start → __libc_start_main → main(). (5) ls calls getopt() to parse args, then for each directory: opendir() → fdopendir() (translates to openat() syscall, returns FD). Kernel walks VFS → ext4/xfs → inode → returns FD. (6) ls calls readdir() in a loop — getdents64() syscall returns directory entries. (7) For each entry, lstat() (or fstatat()) gets metadata (mode, owner, size, mtime, etc.). With -l, ls formats it. (8) For -a, ls includes dot files. (9) ls writes formatted output via write(1, buf, len) to stdout (FD 1, usually a pty connected to terminal). (10) Terminal emulator (xterm, iTerm) reads from the pty, renders glyphs. (11) ls calls exit(0). Kernel reaps, sends SIGCHLD to parent shell, shell prints prompt.

■ **Tip:** Senior interviewers love this question. Drop in strace: `strace -e openat,getdents64,lstat ls -la` — shows you debug at the syscall level.`

Linux

Q2. What's the difference between hard and soft links?

A: Hard link: a second directory entry pointing to the SAME inode. The file 'is' both names — they're equal. Inode reference count tracks how many hard links exist; file data is deleted only when count == 0. Hard links can't cross filesystems (inodes are per-FS). Can't link directories (would create cycles). Soft link (symlink): a separate file containing a path string. Resolved by the kernel at access time. Can cross filesystems, can link directories, can point to non-existent paths (dangling). Slightly slower (extra path resolution). Has its own inode and metadata. Verify: `stat file` shows Links count; ls -li` shows inode number.`

```
# Create both
echo hello > original.txt
ln original.txt hard.txt          # hard link
ln -s original.txt soft.txt       # soft link

ls -li
# 12345 -rw-r--r-- 2 user user 6 May 16 10:00 original.txt
# 12345 -rw-r--r-- 2 user user 6 May 16 10:00 hard.txt          # SAME inode 12345
# 67890 lrwxrwxrwx 1 user user 12 May 16 10:00 soft.txt -> original.txt

# Delete original
rm original.txt

cat hard.txt    # works — inode still has data, ref count still > 0
cat soft.txt    # cat: soft.txt: No such file or directory (dangling)
```

Linux

Q3. Server load average is 50. Should you panic?

A: Load average = average number of processes in 'runnable + uninterruptible sleep' state. It includes BOTH CPU-waiting AND I/O-waiting processes. Critical: divide load by CPU count to interpret. Load 50 on a 64-core box = healthy. Load 50 on a 4-core = trouble. AND the second factor: is the load CPU-bound or I/O-bound?

```
# Step 1: How many cores?
nproc          # logical CPUs
lscpu | grep '^CPU(s):'
```

Load of 50 / 64 cores = ~0.8 per core = fine

```
# Step 2: Is it CPU or I/O?
```

```

uptime
# load average: 50.0, 48.0, 45.0

top
# Look at %wa (I/O wait)
# 90% wa = I/O bound (disk, network slow)
# 90% us = user CPU saturated

vmstat 1 5
# 'r' column = runnable processes
# 'b' column = blocked (uninterruptible) — usually I/O
# If b is high → disk problem

# Step 3: Drill into I/O
iostat -xz 1
# %util > 90 = disk saturated
# r_await, w_await > 10ms = slow I/O

# Step 4: Drill into CPU
top -H          # threads, not just processes
perf top
ps -eo pid,user,%cpu,cmd --sort=-%cpu | head

# Senior answer mentions load average is a LAGGING indicator
# (1, 5, 15 minute moving averages). For real-time, use pressure stall info:
cat /proc/pressure/cpu
cat /proc/pressure/io
# 'some' column = at least one task stalled
# 'full' column = ALL tasks stalled

```

■ **Tip:** Mention *PSI (Pressure Stall Information)* — modern kernel feature 5.2+, the proper way to detect resource pressure. Senior signal.

Linux

Q4. Explain virtual memory and the difference between RSS, VSZ, and PSS.

A: VSZ (Virtual Size): total virtual memory mapped into a process. Includes code, data, shared libs, mmap'd files, anonymous regions. Includes pages NOT in RAM (swapped, lazy-loaded, file-backed). Often huge and meaningless — a process with VSZ of 10GB might use 100MB of actual RAM. RSS (Resident Set Size): physical RAM currently used. Counts shared library pages multiple times if multiple processes share them. PSS (Proportional Set Size): RSS but divides shared pages by sharers. If 4 processes share 100MB of libc, each gets PSS of 25MB for that. PSS is the most honest 'how much RAM is this process really using' metric. USS (Unique Set Size): non-shared private pages only. What you'd free if you killed the process. For OOM debugging: focus on RSS and PSS, ignore VSZ.

```

# Quick view
ps -eo pid,user,rss,vsz,comm --sort=-rss | head

# Detailed per-process memory map
cat /proc/<pid>/status | grep -E 'Vm|Rss'
# VmPeak, VmSize, VmRSS, VmData, VmStk, VmExe, VmLib, RssAnon, RssFile, RssShmem

# PSS requires smaps
cat /proc/<pid>/smaps_rollup
# Shows Rss, Pss, Shared_Clean, Shared_Dirty, Private_Clean, Private_Dirty

# Top viewer with PSS
smem -r -k -t
# Sorted by USS by default; -s switches to PSS

```

Linux

Q5. What happens when you write to a file? walk through the kernel path.

A: (1) write(fd, buf, count) syscall. CPU switches to kernel mode. (2) Kernel finds the file struct via the FD table. From file → inode → superblock → mount info → filesystem driver. (3) Filesystem driver (ext4, xfs, btrfs, etc.) decides where the bytes go. For a normal write: bytes copied from user buf to the kernel's page cache (in-memory pages associated with the inode). Pages marked dirty. (4) write() returns to userspace — data is NOT on disk yet. (5) Dirty pages are written back to disk asynchronously by kernel threads (pdflush / kworker). Triggered by: periodic flush (vm.dirty_writeback_centisecs, default 5s), dirty ratio threshold (vm.dirty_ratio,

vm.dirty_background_ratio), explicit fsync(). (6) Filesystem driver computes which disk blocks to write — may need journal/log update first for crash consistency. (7) Block layer schedules disk I/O. (8) Driver (NVMe, SATA) issues hardware command. DMA copies pages to disk. (9) Disk acks. Page marked clean. Critical for durability: write() alone is NOT durable. To guarantee data on disk: fsync(fd) after write. Databases (Postgres, MySQL) call fsync on commit.

■ **Tip:** Mention *O_DIRECT* (bypass page cache — used by databases for control over caching) and *O_SYNC* (synchronous writes — slow but durable without fsync).

Linux

Q6. What's the difference between a process and a thread?

A: On Linux: not as different as you'd think. Both are 'tasks' in kernel terminology. A process is created by fork() — child gets a copy of parent's memory (COW). A thread is created by clone() with CLONE_VM | CLONE_FILES | CLONE_FS | CLONE_SIGHAND — child shares everything (memory, FDs, signal handlers) with parent. Same scheduler, same kernel data structures (task_struct). Visible in /proc as separate entries. Process: own PID, own memory space (separate address space), own FD table, own signal handlers. IPC needed to communicate (pipes, sockets, shared memory). Thread: same PID + unique TID, shared memory (so direct sharing of variables), shared FD table, shared signal handlers. Communication is just shared memory. Trade-offs: threads cheaper (no memory copy on creation), but require locking. Processes more isolated (a thread can corrupt sibling threads; processes can't corrupt each other). Python special case: GIL serializes Python bytecode execution across threads — for CPU-bound work, multiprocessing > threading.

```
# View threads of a process
ps -eLf | grep <name>          # -L = show threads
ls /proc/<pid>/task/           # one dir per thread

# Each task has a PID (process ID, same for all threads in process)
# and a TID (thread ID, unique per thread)

# In Python:
import threading
threading.get_ident()          # returns TID
```

■ **Tip:** Bring up Linux's 'everything is a task' model — it's elegant and many engineers don't know it.

Linux

Q7. How does fork() actually work? Why is it efficient?

A: fork() creates a near-identical child process. Implementation: COW (copy-on-write). Steps: (1) Kernel allocates a new task_struct for the child. (2) Copies parent's mm_struct (memory descriptor) but marks ALL pages as read-only and references parent's physical pages. Both parent and child see identical memory but neither owns it exclusively. (3) Child gets a new PID, but otherwise: same FDs, same code, same data (lazily). (4) Returns to parent with child's PID, returns to child with 0. (5) When EITHER parent or child WRITES to a page, the MMU triggers a fault. Kernel allocates a new physical page, copies content, updates ONE process's page table. The other still sees the original. Cost: just enough work to set up bookkeeping; no actual memory copy. Followed almost always by exec() in child (which discards the inherited memory entirely). vfork() is even cheaper — doesn't copy mm_struct, parent blocks until child execs.

■ **Tip:** If asked about fork bombs: while(1) fork(); exhausts PID table or per-user process limit. Defense: ulimit -u (RLIMIT_NPROC), cgroups pids.max.

Linux

Q8. Disk is full but du shows much less than df. Why and how to fix?

A: Already covered in Round 2 but expect it again at the senior level with deeper questioning.

```
# Cause: a process has an open FD to a file that was deleted/unlinked.
# Inode is alive (refcount stays > 0 while FD is open).
# Disk blocks aren't freed until the FD closes.

# Diagnosis:
sudo lsof +L1          # files with link count 0 still open
sudo lsof +L1 /var      # filter to mount
```

```
# Fix options ranked by safety:
# 1. Restart the holding process (causes service downtime)
# 2. Send signal that makes the app reopen logs (HUP for rsyslogd, nginx)
sudo kill -HUP <pid>
# 3. Truncate via /proc magic (no downtime, advanced):
sudo bash -c '> /proc/<pid>/fd/<fd-number>'
# This empties the file the FD points to without closing the FD.
# 4. Find and fix the bug: logrotate without copytruncate, app writing to
#    same file after rotation, etc.
```

```
# Prevention:
# - logrotate with copytruncate for apps that don't reopen
# - syslog-style apps: use HUP-on-rotate
# - structured logging to stdout (let container runtime handle rotation)
```

■ **Tip:** Bring up your real story if you have one. NVIDIA SRE interviewers love war stories.

Networking

Q9. What happens when you type 'curl https://api.nvidia.com' and hit enter?

A: This is the canonical interview question. Senior expectation: cover full stack. (1) curl parses URL: scheme=https, host=api.nvidia.com, path=/. (2) DNS resolution: curl calls getaddrinfo. (a) Check /etc/hosts. (b) Check nsswitch.conf order. (c) Send DNS query to resolvers in /etc/resolv.conf. (d) Resolver iterates: root server → .com nameserver → nvidia.com nameserver → A/AAAA record for api.nvidia.com. (e) curl picks an IP. (3) TCP connection: curl creates socket, calls connect(). Kernel sends SYN to target:443. Target replies SYN-ACK. Kernel sends ACK. ESTABLISHED. (4) TLS handshake: client sends ClientHello (TLS version, cipher suites, SNI=api.nvidia.com). Server replies ServerHello + cert chain. Client validates cert against /etc/ssl/certs/ trust store. ALPN negotiates HTTP/2 (or HTTP/1.1). Client and server exchange key material (ECDHE), derive session keys. (5) HTTP request: 'GET / HTTP/1.1\r\nHost: api.nvidia.com\r\n\r\n' sent over the encrypted connection. (6) Server (or front LB / CDN) routes request: WAF inspects, ALB selects target via target group health, target group sends to ASG instance or EKS pod. (7) Server responds with HTTP status + headers + body. (8) curl writes body to stdout. (9) TCP FIN closes connection (or kept open for HTTP/2 multiplexing).

■ **Tip:** Stretch goals: mention TLS 1.3 (one RTT handshake vs 1.2's two), QUIC/HTTP/3 (UDP-based, no TCP slow-start).

Networking

Q10. Explain TCP three-way handshake and four-way close.

A: Open: (1) Client sends SYN (seq=x). (2) Server replies SYN+ACK (seq=y, ack=x+1). (3) Client sends ACK (seq=x+1, ack=y+1). State: ESTABLISHED on both sides. Close: (1) Side A sends FIN (wants to close write side). State: FIN_WAIT_1. (2) Side B sends ACK (acknowledges close). State A: FIN_WAIT_2. State B: CLOSE_WAIT. Note: B can still send data — half-close. (3) B sends its own FIN when done. State B: LAST_ACK. (4) A sends final ACK. State A: TIME_WAIT (held 2*MSL, ~60-120s, to swallow late retransmissions). Then CLOSED. Why TIME_WAIT matters: heavy short-lived connections (CI/CD agents, ECR pulls) can exhaust ephemeral ports if all in TIME_WAIT. Tune: net.ipv4.tcp_tw_reuse=1 (safe to reuse TIME_WAIT sockets for outbound), shorter tcp_fin_timeout. Don't enable tcp_tw_recycle (removed, broke NAT).

■ **Tip:** Bring up TIME_WAIT — it's a real production issue. NVIDIA at scale would care.

Networking

Q11. Difference between TCP and UDP. When use each?

A: TCP: connection-oriented, reliable, ordered, flow-controlled, congestion-controlled. Handshake before data. Retransmits lost packets. Slow-start, congestion window. Adds ~20+ bytes overhead per packet. UDP: connectionless, unreliable, unordered, no flow control, no congestion control. Fire-and-forget. 8 bytes header. Use TCP when: correctness > latency. HTTP, SSH, SMTP, anything where missing data is unacceptable. Use UDP when: latency > correctness, or application handles its own reliability. DNS (single packet, retry at app), VoIP, video streaming, gaming, NTP, syslog, DHCP. Modern hybrid: QUIC (HTTP/3) is on UDP but adds reliability + multiplexing in user space. NVIDIA inference: typically TCP for control plane (gRPC), but tensor streaming research uses UDP or RDMA (RoCE/InfiniBand) for ultra-low-latency.

■ **Tip:** Mention RDMA / InfiniBand / RoCE in NVIDIA context — that's how NVLink and GPUDirect work for multi-GPU training. Senior signal.

Networking

Q12. What's the difference between HTTP, HTTPS, and HTTP/2?

A: HTTP/1.1: text-based, request/response, one in-flight request per connection (pipelining didn't take off due to head-of-line blocking). Connections kept alive (Keep-Alive header). HTTPS = HTTP over TLS. Just adds the encryption layer; the HTTP protocol underneath is the same. Defaults to port 443. Provides confidentiality (encryption), integrity (MAC), authentication (server cert; optionally client cert). HTTP/2: binary framing layer over single TCP connection. Multiplexes many requests/responses concurrently (streams) over one connection — no head-of-line blocking AT THE HTTP LAYER. Server push (deprecated in practice). Header compression (HPACK). Same HTTP semantics. Trade-off: still TCP underneath, so TCP-layer head-of-line still applies — one lost packet stalls all streams. HTTP/3 fixes this by switching to QUIC (UDP-based). Standard ports: HTTP=80, HTTPS=443. SSH=22. DNS=53 (UDP, 53 TCP for AXFR). NTP=123 (UDP). SMTP=25/465/587. IMAP=143/993.

■ **Tip:** Memorize default ports — NVIDIA SRE round always asks. 80, 443, 22, 53, 25, 110, 143, 3306 (MySQL), 5432 (PG), 6379 (Redis), 27017 (Mongo), 9092 (Kafka), 2379-2380 (etcd).

Networking

Q13. Walk through DNS resolution including caching layers.

A: (1) App calls `getaddrinfo('api.nvidia.com')`. (2) `glibc` checks `nsd` / `systemd-resolved` cache (if running). (3) Checks `/etc/hosts`. (4) Reads `/etc/resolv.conf` for nameserver IPs. (5) Sends DNS query (UDP/53) to first nameserver — usually a local recursive resolver (corp DNS, router, ISP, or 8.8.8.8). (6) Recursive resolver checks its cache. If hit and TTL valid, returns. (7) Cache miss: iterative resolution. Resolver asks root NS (.) for `.com`. Root returns NS records for `.com`. (8) Resolver asks `.com` NS for `nvidia.com`. Returns NS records for `nvidia.com`. (9) Resolver asks `nvidia.com` NS for `api.nvidia.com`. Returns A record. (10) Resolver caches and returns to client. (11) Client may cache too (`glibc`, browser, app-level). In Kubernetes: pod's `resolv.conf` points to CoreDNS service IP (or `kube-dns`). CoreDNS resolves Kubernetes service names (`mysvc.myns.svc.cluster.local`) by reading Service objects. External names forwarded to upstream resolver. Caches with configurable TTL. For NVIDIA at scale: `ndots:5` in `resolv.conf` can cause many failed lookups before success (each suffix tried). Tune `ndots:2` if internal naming allows.

■ **Tip:** `ndots:5` is a real perf issue in K8s — NVIDIA engineers will know it. `NodeLocal DNSCache` is the standard mitigation.

Networking

Q14. How do iptables and netfilter work?

A: netfilter = kernel framework that lets modules register hooks at five points in the packet flow. iptables = userspace tool that configures netfilter rules. Hook points: PREROUTING (incoming, before routing decision), INPUT (destined to local), FORWARD (passing through), OUTPUT (locally generated), POSTROUTING (outgoing, after routing). Tables organize rules by purpose: filter (accept/drop), nat (DNAT/SNAT/MASQUERADE), mangle (modify headers), raw (mark for skip-conntrack), security. Chains are lists of rules within tables at specific hooks. Rule = match + action. Match: source IP, dst port, protocol, conntrack state. Action: ACCEPT, DROP, REJECT, DNAT, MASQUERADE, LOG, jump to user chain. Modern replacement: nftables (single unified tool, atomic updates, faster). Kubernetes kube-proxy iptables mode writes thousands of rules — eBPF (Cilium) is the modern alternative.

```
# Common ops:
sudo iptables -L -n -v          # list filter table
sudo iptables -t nat -L -n      # NAT table (kube-proxy lives here)

# Block an IP
sudo iptables -I INPUT -s 1.2.3.4 -j DROP

# Allow only SSH and HTTP
sudo iptables -P INPUT DROP
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

```
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT

# DNAT (port forward)
sudo iptables -t nat -A PREROUTING -p tcp --dport 80 \
  -j DNAT --to-destination 10.0.0.5:8080
```

■ **Tip:** Bring up conntrack (connection tracking) — high-traffic K8s nodes can exhaust `nf_conntrack_max`, causing dropped connections. `/etc/sysctl: net.netfilter.nf_conntrack_max`.

Networking

Q15. Difference between ALB and NLB on AWS?

A: ALB (Application Load Balancer): Layer 7. Understands HTTP/HTTPS/gRPC. Routes based on path, host, headers, query string, source IP. Terminates TLS. Integrates with WAF. Cookie-based session stickiness. Slower than NLB (~milliseconds). Charges by LCU (Load Balancer Capacity Units). NLB (Network Load Balancer): Layer 4. TCP/UDP/TLS. Ultra-low latency (~100µs). Handles millions of req/sec. Static IPs per AZ (or BYOIP). Preserves source IP by default with target-type=ip. Pass-through TLS (no decrypt) OR terminate. Cheaper at scale. Use ALB for: web apps, HTTP APIs, host/path routing, WAF needed. Use NLB for: ultra-low-latency TCP, gRPC w/o L7 features, static IP requirement (e.g. allowlists), TLS pass-through, raw TCP services. Common pattern at Cisco IQ (your resume): external NLB → internal ALB across accounts for security separation. Mention this.

■ **Tip:** Your resume mentions 'Designed cross-account routing: NLB (network account) → ALB (application account)' — that's exactly the pattern NVIDIA might draw on the whiteboard. Be ready to explain it.

Networking

Q16. Explain VPC, subnet, route table, IGW, NAT Gateway, security group, NACL.

A: VPC: isolated virtual network in AWS. CIDR block, e.g. 10.0.0.0/16. Subnet: subset of VPC CIDR within a single AZ. Can be public or private. Route table: associated with subnets; controls where traffic goes. Default: local routes for VPC. Plus custom: 0.0.0.0/0 → IGW (public), or → NAT (private). Internet Gateway (IGW): horizontally-scaled AWS-managed gateway. One per VPC. Public subnets have route 0.0.0.0/0 → IGW. NAT Gateway: provides outbound internet for private subnets. Lives in PUBLIC subnet. SNAT translates private IPs to NAT's EIP. Stateful — return traffic allowed. \$0.045/hr + \$0.045/GB on data processed. Costs add up fast. Security Group: stateful firewall on ENIs (instance/pod-level). Allow rules only. Reference other SGs by ID. Default deny. NACL: stateless firewall at SUBNET level. Allow AND deny rules. Numbered (lower = evaluated first). Default allow all. Use ALB+SG for app-level firewall; NACL for coarse subnet defense. VPC Endpoint: private connection to AWS services without traversing public internet. Gateway endpoint (free): S3, DynamoDB. Interface endpoint (paid, PrivateLink): everything else (ECR, STS, Secrets Manager, etc.). Critical for private subnets to avoid NAT data costs.

■ **Tip:** Mention: 'For EKS in private subnets, I'd add VPC endpoints for ECR (api + dkr), STS, Secrets Manager, CloudWatch Logs, S3 — saves significant NAT charges.'

Networking

Q17. Two services in the same VPC can't talk to each other. Walk me through diagnosis.

A: Top-down checklist.

```
# Step 1: Confirm DNS
# Can A resolve B's name?
dig +short b-service.example.com
nslookup b-service.example.com

# Step 2: TCP reachability
nc -vz b-service.example.com 443
# Or:
curl -v https://b-service.example.com/health

# Step 3: Layer 4 path
# If timeout (not refused), packets are being dropped.
# Suspect: security group, NACL, route table, VPC peering.
# Step 4: Check security group on B
```



```

# B's SG must ALLOW inbound on the port from A's SG or IP.
aws ec2 describe-security-groups --group-ids sg-xxx \
  --query 'SecurityGroups[].IpPermissions'

# Step 5: Check security group on A (egress – usually open by default)
# Less common cause but possible if explicit egress rules.

# Step 6: NACL on B's subnet
# NACLs are stateless: need allow rule for INBOUND on target port,
# AND OUTBOUND on ephemeral ports for return traffic.
aws ec2 describe-network-acls --filters Name=association.subnet-id,Values=subnet-xxx

# Step 7: Route table
# If A and B are in different subnets – route exists for the destination?
# Cross-VPC? Need VPC peering route (and reciprocal route in target VPC).
# Cross-account? Need Transit Gateway or peering + accept.

# Step 8: VPC Flow Logs
# Enable VPC flow logs to confirm whether packets are being:
# - ACCEPTED (so app-level issue)
# - REJECTED (SG/NACL blocking)
# - Not arriving at all (route/peering)

# Step 9: For K8s services specifically
# Check if endpoints have ready IPs
kubectl get endpoints b-service -n <ns>
# If no endpoints: pod selector mismatch or pods not Ready

# Step 10: NetworkPolicy
kubectl get netpol -n <ns>
# Default-deny that doesn't include source?

```

■ **Tip:** Senior systematic answer. Always go OUTSIDE-IN: DNS → TCP → SG → NACL → route → application.

Round 5: CI/CD, GitOps, GitHub Actions, Build Systems (60 min)

NVIDIA's JR2017386 explicitly calls out 'administering, monitoring, and deploying systems and services on GitHub' and CI/CD ownership. Tie to your Cisco IQ work: GitHub-based CI/CD with PR-driven workflows, CircleCI-to-GitHub migration, GitHub Copilot + Claude CLI agentic loops, Atlantis-equivalent automation patterns.

CI/CD

Q1. Design a CI/CD pipeline for an AI inference service on EKS. End-to-end.

A: Structure: source → CI (build/test/scan) → artifact (image) → GitOps repo → CD (Argo CD) → smoke test → monitor.

```
# .github/workflows/build-and-deploy.yml
name: build-and-deploy

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

permissions:
  id-token: write    # for OIDC to AWS
  contents: read
  pull-requests: write

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: 'pip'
      - run: pip install -r requirements-dev.txt
      - run: pytest --cov --cov-report=xml --junit-xml=junit.xml
      - run: ruff check .
      - run: mypy src/
      - uses: actions/upload-artifact@v4
        with:
          name: coverage
          path: coverage.xml

  security-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Dependency scan
        uses: aquasecurity/trivy-action@master
        with:
          scan-type: 'fs'
          severity: 'CRITICAL,HIGH'
          exit-code: '1'
      - name: SAST
        uses: github/codeql-action/analyze@v3
      - name: Secret scan
        uses: gitleaks/gitleaks-action@v2

  build-image:
```

needs: [test, security-scan]
if: github.event_name == 'push' # only on merge to main
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4

```

- uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::ACCT:role/gh-ecr-push
    aws-region: us-west-2

- uses: aws-actions/amazon-ecr-login@v2

- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3

- name: Build and push (multi-arch)
  uses: docker/build-push-action@v5
  with:
    platforms: linux/amd64,linux/arm64
    push: true
    tags: |
      ${ steps.login-ecr.outputs.registry }/inference:${ github.sha }
      ${ steps.login-ecr.outputs.registry }/inference:latest
    cache-from: type=gha
    cache-to: type=gha,mode=max

- name: Image scan
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ${ steps.login-ecr.outputs.registry }/inference:${ github.sha }
    severity: 'CRITICAL'
    exit-code: '1'

- name: Sign image (Cosign + OIDC)
  run: |
    cosign sign --yes ${IMAGE_URI}@${IMAGE_DIGEST}

deploy-dev:
  needs: build-image
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with:
        repository: org/gitops-config
        token: ${ secrets.GITOPS_TOKEN }
    - name: Update image tag in dev values
      run: |
        yq -i '.image.tag = strenv(SHA)' \
          apps/inference/dev/values.yaml
        git config user.name "github-actions"
        git config user.email "actions@github.com"
        git add -A
        git commit -m "deploy: inference ${SHA} to dev"
        git push
      env:
        SHA: ${ github.sha }

# Argo CD picks up the GitOps repo change and syncs.
# Production deploys via separate workflow with manual approval.

```

■ **Tip:** Pull from your Cisco IQ work: 'I did this exact pattern when migrating from CircleCI to GitHub Actions — CI builds artifacts, GitOps repo PR updates image tag, Argo CD syncs.'

CI/CD

Q2. Why GitOps over push-based CD?

A: Push-based CD: CI pipeline has cluster credentials, runs kubectl apply directly. Pull-based GitOps: CI builds artifacts. A controller IN the cluster (Argo CD, Flux) pulls desired state from Git, reconciles. Advantages of GitOps: (1) Git is source of truth — desired state visible, auditable, revertable. (2) CI doesn't need cluster credentials — reduces blast radius. (3) Drift detection — controller continuously reconciles, can auto-correct manual changes. (4) Disaster recovery — rebuild cluster from Git. (5) Multi-cluster fanout — one Git push, many clusters update. (6) Rollback = git revert. (7) Separates concerns — CI pipeline doesn't know about cluster topology. Disadvantages: (1) More moving parts — need to manage Argo CD itself. (2) Initial setup complexity. (3) Image updates need automation (image updater, PR bot). (4) Secrets need external manager (External Secrets, sealed-secrets, SOPS). At Cisco IQ I migrated CircleCI push-deploys to GitHub CI + Argo CD GitOps —

that improved scalability ~20% and reduced downtime ~30%.

■ **Tip:** Cite your exact resume numbers. Senior interview = back claims with metrics.

CI/CD

Q3. How does Argo CD detect and fix drift?

A: Argo CD continuously compares two states: (1) Desired state — rendered manifests from Git (Helm template, Kustomize build, or raw YAML). (2) Live state — current Kubernetes objects. The Application controller polls (default every 3 minutes) and on events. For each managed resource, computes a normalized diff. Result: Synced (match) or OutOfSync (differ). Healthy/Degraded is orthogonal — comes from resource health checks (Deployment progressing, Pod ready, etc.). Sync policy controls actions: manual (you click sync), automated (auto-applies Git changes). Within automated: prune (delete live resources not in Git — DANGER), self-heal (revert manual cluster changes back to Git). Common issues: (1) Controller-managed fields (replicas after HPA scales, last-applied-configuration annotation drift) — use ignoreDifferences to scope out. (2) Webhook mutations (kyverno adds labels) — same fix. (3) Auto-sync with prune enabled is dangerous — accidentally removing a resource from Git deletes it from prod. Use sync windows / restricted prune for prod.

```
# Application spec showing key fields
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: inference-prod
  namespace: argocd
spec:
  project: prod
  source:
    repoURL: https://github.com/org/gitops-config
    targetRevision: main
    path: apps/inference/prod
    helm:
      valueFiles: [values.yaml, values-prod.yaml]
  destination:
    server: https://prod-cluster.example.com
    namespace: inference
  syncPolicy:
    automated:
      prune: false           # DON'T auto-delete in prod
      selfHeal: true         # but DO revert manual drift
    syncOptions:
      - CreateNamespace=true
      - PrunePropagationPolicy=foreground
      - ServerSideApply=true
    retry:
      limit: 5
      backoff:
        duration: 5s
        factor: 2
        maxDuration: 3m
  ignoreDifferences:
    - group: apps
      kind: Deployment
      jsonPointers:
        - /spec/replicas    # HPA owns this — don't fight
```

■ **Tip:** From your resume: ArgoCD multi-account rollout in apps-platform AWS account, OPA tag hygiene 12 → 0 warnings. Use this story.

CI/CD

Q4. What's the app-of-apps pattern in Argo CD?

A: An Argo CD Application that itself contains other Application manifests. Used for bootstrapping clusters declaratively. Pattern: (1) Root app installed manually once via kubectl/argocd CLI. (2) Root app's source points to a Git directory containing child Application manifests. (3) Argo CD syncs the root → creates children → children sync their own resources. Tree structure: root → platform-addons → external-secrets, cert-manager, ingress, monitoring. root → shared-services → postgres, redis, queues. root → business-apps → api, worker, frontend. ApplicationSet is the evolution: generates many Applications from a template using generators (list,

cluster, git directory, pull request, matrix). For multi-cluster/multi-env, ApplicationSet > app-of-apps. Risk: app-of-apps has huge blast radius — break root app, lose visibility into all children. Mitigate with: separate prod app/project, manual sync for root in prod, RBAC restricting who can edit root.

■ **Tip:** *Be ready to whiteboard the tree structure. NVIDIA's AI inference team almost certainly uses ApplicationSet — it's the standard at scale.*

CI/CD

Q5. How does GitHub Actions OIDC work? Why is it better than AWS keys?

A: OIDC = OpenID Connect, a thin layer on OAuth 2.0 for identity. GitHub Actions can mint short-lived JWT tokens signed by GitHub. AWS can verify these tokens against GitHub's OIDC provider and exchange for STS credentials. End-to-end: (1) Register GitHub as an OIDC identity provider in IAM (one-time per AWS account). (2) Create IAM role with trust policy: 'principal = github oidc provider, condition = subject claim matches repo:org/repo:ref:refs/heads/main'. (3) Workflow declares permissions: id-token: write. (4) Workflow step uses aws-actions/configure-aws-credentials with role-to-assume. (5) Action requests JWT from GitHub, calls sts:AssumeRoleWithWebIdentity, receives 15-60min AWS credentials. (6) Subsequent steps use those creds. Why better: (a) No long-lived secrets in GitHub. (b) Trust scoped to specific repo/branch/environment claims. (c) Short-lived — limited damage if leaked. (d) Auditable in CloudTrail (shows assumed role). (e) Easy rotation (no rotation needed — tokens are ephemeral).

```
# Trust policy example — scopes role to your repo, main branch only
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Federated": "arn:aws:iam::ACCT:oidc-provider/token.actions.githubusercontent.com"},
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
      },
      "StringLike": {
        "token.actions.githubusercontent.com:sub": "repo:my-org/my-repo:ref:refs/heads/main"
      }
    }
  }]
}

# Workflow step:
- name: AWS auth
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::ACCT:role/gh-actions-prod-deploy
    aws-region: us-west-2
```

■ **Tip:** *Mention 'environment:' in workflows — gates use claim repo:org/repo:environment:prod, adds protection rules (manual approval, secrets per env).*

CI/CD

Q6. How do you secure self-hosted GitHub Actions runners?

A: Self-hosted runners are part of your attack surface. They have whatever credentials you give them. Hardening: (1) Ephemeral runners — fresh container/VM per job, destroyed after. Use actions-runner-controller (ARC) on Kubernetes. (2) Don't run untrusted PR code on runners with secrets. pull_request_target context is dangerous. (3) Isolate by repo/environment/team. Per-team runner groups. (4) Least privilege IAM — only what the workflow needs. Distinct roles per environment. (5) Network isolation — runners in private subnets, egress through NAT, only required endpoints reachable. (6) Audit log access — record what jobs run, by whom. (7) Update runner binaries regularly — runner is software too. (8) Don't allow privileged: true in Docker-in-Docker patterns. Use kaniko/buildkit for rootless image builds. (9) Pinned action versions (use SHA, not v3 floating tag). (10) Required workflows for org-level enforcement.

■ **Tip:** *From your Cisco IQ: 'Built and maintained Build Agents to support platform services and deployment pipelines.' Tell a story about ephemeral runner pattern you set up.*

Q7. GitHub Actions matrix strategy — when and how?

A: Matrix runs the same job across combinations of variables — OS, version, region, etc. Reduces duplication. Limits: max 256 jobs per matrix; up to 10 dimensions. Combinations explode fast — be deliberate. include/exclude lets you add/remove specific combos. fail-fast: false to run all combos even after one fails (useful for diagnostic builds, dangerous for prod deploys).

```
jobs:
  test:
    runs-on: ${ matrix.os }
    strategy:
      fail-fast: false
    matrix:
      os: [ubuntu-latest, macos-latest, windows-latest]
      python: ['3.10', '3.11', '3.12']
      include:
        # Add: extra combo
        - os: ubuntu-latest
          python: '3.13-dev'
          experimental: true
      exclude:
        # Remove: macOS doesn't get 3.10
        - os: macos-latest
          python: '3.10'
    continue-on-error: ${ matrix.experimental == true }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python }
      - run: pytest
```

■ **Tip:** For NVIDIA: matrix across CUDA versions, GPU types, kernel versions — very real use case for inference image builds.

Q8. How do you handle secrets in CI/CD without leaking them?

A: (1) NEVER commit secrets to Git. Pre-commit hooks (gitleaks, detect-secrets) catch leaks at commit time. CI secret scanning (Trivy, gitleaks-action) is the second line. If you DO leak: rotate immediately, don't just delete the commit (history accessible). (2) Use OIDC for cloud access — short-lived, no static keys. (3) For non-cloud secrets (API keys, DB passwords): GitHub encrypted secrets, with environment scoping. (4) Mask secrets in logs — GitHub auto-masks but explicit add-mask is safer for derived values. (5) Don't echo secrets to logs intentionally. (6) Don't pass secrets via command-line args (visible in /proc). (7) For runtime: External Secrets Operator + AWS Secrets Manager, Vault, SOPS-encrypted in Git, sealed-secrets. (8) Audit access — who has read on secrets, who triggered which workflow. (9) Limit secret scope — repo secrets vs environment secrets vs org secrets. Use the smallest scope that works.

■ **Tip:** Bring up your External Secrets + IRSA experience from Cisco IQ. Exact pattern.

Q9. Jenkins / CloudBees declarative vs scripted pipeline?

A: Declarative: structured DSL with pipeline { agent {} stages { stage(") { steps {} } } }. Easier to lint, govern, and standardize. Required for shared libraries to enforce conventions. Recommended default. Scripted: full Groovy. More flexible — can use loops, conditionals, custom logic. Harder to audit and govern. Use when declarative blocks can't express the workflow (rare). CloudBees adds enterprise features over OSS Jenkins: managed controllers, governance (role-based access, audit), high availability, plugin curation, support. Shared libraries: versioned Groovy code reused across pipelines. Provides standardized stages (build, test, deploy) that teams call. Best practice: keep complex logic in shared libraries; pipeline calls them.

■ **Tip:** From your resume: GitHub + CloudBees migration from CircleCI. Tell the story.

Q10. How would you speed up a slow CI pipeline?

A: Diagnose first, optimize second. Measure: (1) Per-step duration in the pipeline view. (2) Queue time vs execution time. Long queue = need more runners. (3) Identify the bottleneck step — often dependencies install, slow tests, image build. Common wins: (1) Dependency caching — actions/cache, or buildx GHA cache for Docker layers. (2) Parallelize tests — pytest-xdist, jest --maxWorkers, Go's t.Parallel(). (3) Test sharding — split test suite across N parallel jobs. (4) Incremental builds — only rebuild what changed (Bazel, Nx, Turborepo). (5) Larger runners — 8-core \$0.064/min often pays off vs 2-core \$0.008/min. (6) Self-hosted runners on AWS Spot — cheaper at scale. (7) Skip unnecessary checkout depth (default fetch-depth=1, not 0). (8) Multi-stage Docker builds — only rebuild changed layers. (9) Matrix concurrent — N OS×versions in parallel, not serial. (10) Eliminate flaky retries that waste time. Be careful: too much parallelism overwhelms downstream (e.g., concurrent deploys to same env).

■ **Tip:** Mention your own metric: at Cisco IQ you reduced PR-authoring cycle time ~60% with Claude CLI agentic loops. That's a CI/CD optimization story.

CI/CD

Q11. Design an image promotion strategy across dev/staging/prod.

A: Build once, promote many. Never rebuild for prod. Pattern: (1) PR build creates image with tag like `sha-abc123` and `pr-456`. (2) Merge to main builds + pushes with `main-sha-abc123` and `latest`. (3) GitOps repo updates dev values: image tag = sha-abc123. Argo syncs dev. (4) After validation in dev, automated PR updates stage values to same SHA. (5) Manual approval for stage → prod promotion. PR updates prod values, requires CODEOWNERS approval, Argo CD syncs prod. The image bytes never change — same digest from dev through prod. Use immutable tags (SHA or semver) not floating tags (latest, main). Validate provenance: image signing with cosign, verify with Kyverno admission policy. For multi-region: same digest promoted to all regions simultaneously, with canary rollout per region.

■ **Tip:** 'Same artifact end-to-end' is the senior signal. NVIDIA cares because deploying tested-but-not-promoted bytes is a real outage cause.

CI/CD

Q12. How do you handle a deployment that fails health checks in prod?

A: Layered defense. (1) Pre-deploy: smoke tests in lower envs catch most issues. Canary deploy to 5% traffic first. (2) During deploy: rolling update with maxUnavailable=0, maxSurge=25% — new pods come up before old removed. Readiness probes prevent traffic going to broken pods. (3) Health gates: Argo CD or Argo Rollouts checks post-deploy metrics (5xx rate, latency p99). If thresholds breached, auto-abort. (4) Detect: alerts on SLO burn rate, dashboards on golden signals (latency, traffic, errors, saturation). (5) Mitigate FIRST: rollback (git revert in GitOps, or argo rollouts abort + undo) — restore service. Then diagnose. Don't try to forward-fix under fire. (6) Communicate during mitigation: incident channel, status page, exec summary. (7) Post-incident: blameless RCA. Action items: detection gap fixes, automation, test coverage. Use Argo Rollouts for canary/blue-green with automated analysis: AnalysisTemplate queries Prometheus, fails canary on threshold breach.

■ **Tip:** Your resume: 'GitOps-driven proactive CA injection and automated certificate hardening/cleanup jobs, preventing sync/admission failures' — that's a real story of preventing failed deploys at the source.

CI/CD

Q13. How does Atlantis fit into Terraform workflow?

A: Atlantis is a Terraform pull-request automation tool. Watches PRs that modify Terraform code. On PR open, runs `terraform plan` and posts result as a PR comment. On `atlantis apply` comment from authorized user, runs apply. Benefits: (1) Plan visibility in PR — reviewers see exactly what changes. (2) No engineer needs Terraform admin creds locally — Atlantis has them. (3) Audit trail in PR. (4) Locks state to prevent concurrent applies (per workspace). (5) Policy enforcement via OPA/Conftest before apply. (6) Multi-env via dir/workspace mapping. Alternative: Terraform Cloud / HCP Terraform, Spacelift, env0 — paid SaaS with similar workflow. At Cisco IQ you have Atlantis in your toolkit — be ready to talk through PR-driven IaC workflow.

■ **Tip:** Mention you 'leveraged GitHub Copilot, Claude CLI, and automation agents to generate infrastructure PRs directly from JIRA tickets' — that's modern AI-augmented IaC.

Round 6: Terraform, IaC, AWS Architecture (60 min)

Your strongest area — AWS SA Pro + Terraform Associate + Cisco IQ Terraform/Terragrunt experience with reusable modules and cross-account patterns. NVIDIA explicitly lists Ansible & Terraform in JR2017386.

Terraform

Q1. Walk me through terraform init, plan, apply, destroy.

A: init: downloads providers, initializes backend (remote state config), downloads modules. Run once per project, or after backend/providers/modules change. plan: compares state + config + real provider data, computes diff. Outputs proposed changes. -out flag writes binary plan for atomic apply. apply: executes the plan. Without -auto-approve, asks for confirmation. Without an explicit plan file, computes plan then applies (less atomic). destroy: destroys all managed resources. Almost never used in production directly — use targeted destroys via state rm + revert. Production workflow: fmt → validate → init (if needed) → plan (review in PR) → apply (after approval). Modern: Atlantis or Terraform Cloud handle this through PR comments. Never apply to prod from a developer laptop.

■ **Tip:** Mention 'I'd run `fmt` and `validate` as part of pre-commit hooks, not just CI — fail fast.'

Terraform

Q2. What is Terraform state and why does it matter?

A: State (`terraform.tfstate`) is the mapping from your configuration to real-world resources. Without it, Terraform doesn't know which AWS instance corresponds to `aws_instance.web` in your code. State tracks: resource IDs, attribute values, dependencies, metadata (provider version, etc.). Critical because: (1) Terraform diffs config vs state vs reality to compute plans. (2) State has secrets — if a resource manages a password, the password value is in state. (3) Operations like import, taint, mv, rm operate on state. Production state management: (1) Remote backend — S3 + DynamoDB lock for AWS, Terraform Cloud, GCS, Azure Storage. NEVER local state in a team setting. (2) Encryption at rest (KMS on S3). (3) Versioning enabled on S3 bucket — recovery option. (4) Lock via DynamoDB — prevents concurrent applies. (5) Backup before destructive ops. (6) Separate state per environment AND per blast-radius unit. Smaller state = faster plans, less risk.

```
# Backend configuration
terraform {
  backend "s3" {
    bucket     = "my-org-tf-state-prod"
    key        = "platform/vpc/terraform.tfstate"
    region     = "us-west-2"
    encrypt    = true
    kms_key_id = "alias/terraform-state"
    dynamodb_table = "tf-lock-prod"
  }
}

# Each env/component has its own state file:
# platform/vpc/dev/terraform.tfstate
# platform/vpc/stage/terraform.tfstate
# platform/vpc/prod/terraform.tfstate
# platform/eks/dev/terraform.tfstate ... etc
```

■ **Tip:** Your resume mentions Terragrunt — bring up how Terragrunt automates the backend key-per-environment pattern via ``terragrunt.hcl`` includes.

Terraform

Q3. Terraform modules vs workspaces — when use each?

A: Module: reusable configuration. Defines what to create. Inputs (variables) and outputs. Compose by source = `'./modules/vpc'` or remote (registry, git). Use for: repeatable patterns (VPC, EKS cluster, IAM role with policies). Workspace: a way to maintain multiple states for the same configuration. `terraform workspace new prod`, `terraform workspace select dev`. The state file is suffixed with workspace name. Use for: very-similar-environments where the config doesn't change. Avoid for production at scale because: (1) Easy to

apply to wrong workspace by mistake. (2) Can't have meaningfully different config without `count = terraform.workspace == ...` mess. (3) Separate backend per env is safer. Production pattern: modules + explicit env directories. Each env (live/dev, live/stage, live/prod) is its own root module with its own backend + separate AWS account. Terragrunt is built on this idea — DRY via hcl includes.

■ **Tip:** *Your Cisco IQ resume: 'Authored reusable Terraform modules to standardize infrastructure provisioning across services' — exactly this. Cite it.*

Terraform

Q4. How does terraform import work?

A: terraform import maps an EXISTING real resource into your state file so Terraform can manage it. Workflow: (1) Write minimal `resource` block in config matching the type (e.g. `aws_s3_bucket.imported {}`). (2) Run `terraform import aws_s3_bucket.imported my-bucket-name`. (3) State now has the resource. Run `terraform plan` — will show MANY changes because your skeletal config doesn't match the real resource's attributes. (4) Fill in config to match: bucket name, ACLs, versioning, etc. Iterate plan until it shows no changes. (5) Now terraform manages it. Modern Terraform 1.5+: import blocks in config — declarative imports, plan-able, no separate CLI command needed. Useful for migrating existing infrastructure or recovering from accidental state removal.

```
# Old way (CLI):
terraform import aws_s3_bucket.legacy my-bucket-name
```

```
# New way (1.5+): import block in config
import {
  to = aws_s3_bucket.legacy
  id = "my-bucket-name"
}
resource "aws_s3_bucket" "legacy" {
  bucket = "my-bucket-name"
  # other config to match the real resource...
}
```

```
# Run terraform plan and iterate until clean.
```

■ **Tip:** *Mention the new `import {}` block syntax — Terraform 1.5+, you've kept up with the changes.*

Terraform

Q5. What's drift and how do you handle it?

A: Drift = real infrastructure differs from Terraform's state/config. Causes: (1) Someone manually changed something in the AWS console. (2) Another tool modified it. (3) Provider defaulted a field your config didn't set. (4) Service made automatic changes (autoscaling, recovery). Detection: `terraform plan` with no config changes — pure plan shows reality vs state. AWS Config drift detection, Driftctl. Fix decisions: (1) Reality is right → update config to match, run `apply` (no-op). (2) Config is right → run `apply`, Terraform corrects reality. (3) Provider default → add `lifecycle.ignore_changes` for that field. Prevention: (1) IAM policies that prevent console writes to managed resources. (2) AWS Config rules + remediation. (3) Scheduled drift detection in CI. (4) Service Control Policies in AWS Organizations. (5) GitOps culture — every change goes through PR.

■ **Tip:** *Drift is a process problem more than a tech problem. Bring up: 'I'd add a CI workflow that runs terraform plan on schedule and alerts on diff.'*

Terraform

Q6. Difference between count and for_each?

A: `count`: creates N identical resources indexed 0..N-1. Access via `aws_instance.web[0]`. Problem: removing one shifts indexes — Terraform thinks resources moved, plans `destroy+create`. `for_each`: creates resources keyed by string (or map). Access via `aws_instance.web["frontend"]`. Removing 'frontend' destroys only that one; others untouched. Use `for_each` whenever resources have identity (named items). Use `count` for: (1) numeric scaling (N replicas), (2) conditional create (`count = var.enabled ? 1 : 0`). Modern Terraform: `for_each` with `toset()` preferred even for simple cases.

```
# Bad — removing 'us-west' shifts indexes, recreates everything
variable "regions" { default = ["us-east", "us-west", "us-south"] }
resource "aws_s3_bucket" "regional" {
```

```

count = length(var.regions)
bucket = "data-#{var.regions[count.index]}"
}

# Good – stable keys
resource "aws_s3_bucket" "regional" {
  for_each = toset(var.regions)
  bucket   = "data-${each.key}"
}
# Now removing 'us-west' destroys only data-us-west.

```

■ **Tip:** Senior must-know. Production rule of thumb: 'for_each unless I genuinely need just-count-N-things.'

Terraform

Q7. How do you prevent accidental destruction of critical resources?

A: Layered defenses: (1) Lifecycle `prevent_destroy = true` — Terraform refuses to destroy. Override only by removing the lifecycle setting (visible in diff). (2) AWS-side deletion protection: RDS, EC2 termination protection, S3 bucket versioning + MFA delete, EKS cluster deletion protection. (3) IAM scope — apply role can't delete certain resource types. Different roles for read-only vs write vs destroy. (4) Manual approval gates in CI for plans containing destroy actions. (5) Atlantis/Terraform Cloud — apply restricted to specific users/teams. (6) Use targeted operations cautiously — `-target=...` shouldn't be a habit. (7) Separate state per blast-radius unit (don't put DB and VPC in same state — losing one shouldn't blast the other). (8) State backups + S3 versioning. (9) Disable `autoApply` on prod Argo CD applications. (10) Plan output review in PR with required approvals.

```

resource "aws_db_instance" "prod" {
  identifier = "main"
  # ...
  deletion_protection      = true
  skip_final_snapshot      = false
  final_snapshot_identifier = "main-final-${formatdate("YYYY-MM-DD-hhmm", timestamp())}"

  lifecycle {
    prevent_destroy = true
    ignore_changes = [
      password, # rotated outside TF
      final_snapshot_identifier,
    ]
  }
}

```

■ **Tip:** Senior production maturity. NVIDIA's prod databases will have this exact pattern.

Terraform

Q8. How do you handle secrets in Terraform?

A: Hard rule: never commit raw secrets to Git. Multiple safer patterns: (1) Reference existing secrets — Terraform creates the secret container (`aws_secretsmanager_secret`), but value populated out-of-band (manual or separate pipeline). resource has `lifecycle.ignore_changes` on `secret_string`. (2) Read secrets via data source from external manager — `data "aws_secretsmanager_secret_version"`. Get value, pass to resource. Value still ends up in state. (3) Mark variables `sensitive = true` — hides in CLI output, doesn't hide in state. (4) Use environment variables for credentials Terraform needs (AWS keys, GH tokens) — not in code. (5) State itself is sensitive — encrypt at rest (S3 + KMS), restrict access tightly. Anyone with state has secrets. (6) For Terraform's own AWS auth: OIDC role assumption from CI, not static keys. (7) For app secrets at runtime, prefer secret manager + `ExternalSecrets` in K8s, not Terraform-managed secrets. Senior reminder: 'sensitive' flag is cosmetic — state is the real risk.

■ **Tip:** Tie to your External Secrets / AWS Secrets Manager + IRSA work at Cisco IQ. Same philosophy.

Terraform

Q9. Show me how you'd structure Terraform for a multi-account, multi-region AWS platform.

A: Pattern: modules for reusable components + environment directories for state separation.

```

# Repository structure:
infrastructure/

```

```

■■■ modules/                                # reusable building blocks
■   ■■■ vpc/
■   ■■■ eks-cluster/
■   ■■■ iam-role/
■   ■■■ argo-cd-bootstrap/
■   ■■■ external-secrets/
■■■ live/                                    # environment configurations
    ■■■ shared-services/
    ■   ■■■ account.hcl                    # account number, region
    ■   ■■■ network-vpc/
    ■       ■■■ terragrunt.hcl
    ■   ■■■ route53/
    ■       ■■■ terragrunt.hcl
    ■■■ dev/
    ■   ■■■ account.hcl
    ■   ■■■ us-west-2/
    ■       ■■■ vpc/
    ■       ■■■ eks/
    ■       ■■■ eks-addons/
    ■       ■■■ apps-platform/
    ■   ■■■ env.hcl
    ■■■ stage/
    ■   ■■■ ...
    ■■■ prod/
        ■■■ account.hcl                    # different AWS account
        ■■■ us-west-2/
        ■■■ eu-central-1/
        ■■■ ap-southeast-2/

# Each terragrunt.hcl includes parent locals for backend config:
# live/dev/us-west-2/eks/terragrunt.hcl
include "root" {
  path = find_in_parent_folders()
}
include "env" {
  path = "${get_terragrunt_dir()}/../../env.hcl"
}

terraform {
  source = "../../../modules//eks-cluster"
}

inputs = {
  cluster_name    = "dev-platform"
  vpc_id          = dependency.vpc.outputs.vpc_id
  subnet_ids      = dependency.vpc.outputs.private_subnet_ids
  kubernetes_version = "1.30"
}

dependency "vpc" {
  config_path = "../vpc"
}

# Backend automatically inherits to:
# bucket = terraform-state-dev
# key     = us-west-2/eks/terraform.tfstate
# dynamodb_table = tf-lock-dev

```

■ **Tip:** This is exactly your Cisco IQ pattern. Show this structure on whiteboard if asked. The cross-account piece (separate accounts for shared/dev/prod) is the senior signal.

Terraform

Q10. How do you handle a failed terraform apply in production?

A: First rule: don't blindly rerun. (1) Read the error carefully. Note which resource and which step. (2) Check state lock — if apply was interrupted, lock may still be held. terraform force-unlock <lock-id> only after confirming no other apply running. (3) Run terraform plan — see current state vs desired. Some resources may be partially created. (4) Decide: forward-fix or rollback? (5) Forward: fix the config issue, re-apply. Common: permissions, quota, dependency timing. (6) Rollback: revert the PR, re-apply. Manage partial resources: terraform state list shows what's tracked. terraform state rm removes from state (doesn't destroy). terraform import re-adds. (7) Verify in AWS console / API that reality matches expectations. (8) Document what happened, what you did,

lessons. (9) If state corruption: restore from S3 versioning, or rebuild via import. NEVER edit state file by hand in production.

■ **Tip:** Tell a story if you have one. 'I once had a partial apply leave a half-created NAT Gateway. I did X, Y, Z to recover.'

AWS

Q11. Design VPC architecture for a multi-AZ EKS cluster with private nodes.

Standard 3-tier across 3 AZs:

VPC 10.0.0.0/16

■■■ Public subnets (one per AZ)

■ 10.0.1.0/24 (us-west-2a)

■ 10.0.2.0/24 (us-west-2b)

■ 10.0.3.0/24 (us-west-2c)

■■■ Route: 0.0.0.0/0 → IGW

■ ■■■ Resources: ALB/NLB ENIs, NAT Gateways

■ ■■■ Kubernetes labels: kubernetes.io/role/elb=1

■

■■■ Private subnets (one per AZ)

■ 10.0.11.0/24 (us-west-2a)

■ 10.0.12.0/24 (us-west-2b)

■ 10.0.13.0/24 (us-west-2c)

■ ■■■ Route: 0.0.0.0/0 → NAT in same AZ (single AZ NAT = cost saving)

■ ■■■ Resources: EKS worker nodes, pods (AWS VPC CNI)

■ ■■■ Kubernetes labels: kubernetes.io/role/internal-elb=1

■

■■■ Database subnets (one per AZ)

10.0.21.0/24 (us-west-2a)

10.0.22.0/24 (us-west-2b)

10.0.23.0/24 (us-west-2c)

■■■ No internet route — pure internal

■■■ Resources: RDS, ElastiCache, MSK

VPC Endpoints (Interface for the private subnet to use):

- com.amazonaws.us-west-2.ecr.api (ECR API)
- com.amazonaws.us-west-2.ecr.dkr (ECR Docker)
- com.amazonaws.us-west-2.sts (IRSA)
- com.amazonaws.us-west-2.logs (CloudWatch logs)
- com.amazonaws.us-west-2.secretsmanager
- com.amazonaws.us-west-2.s3 (Gateway endpoint — free!)
- com.amazonaws.us-west-2.eks (EKS API access)

Sizing considerations:

- AWS VPC CNI allocates pod IPs from subnet → /24 = 250 pods max
- For density: use prefix delegation (16x more pods per node)
- Or non-routable secondary CIDR (100.64.0.0/16) for pods only

EKS-specific tags:

- Public subnets: kubernetes.io/cluster/<cluster-name> = shared
- Public subnets: kubernetes.io/role/elb = 1
- Private subnets: kubernetes.io/cluster/<cluster-name> = shared
- Private subnets: kubernetes.io/role/internal-elb = 1

Security Groups:

- Cluster SG (created by EKS): allow node SG → cluster SG on 443
- Node SG: allow self ingress (pod-to-pod), cluster SG ingress, egress 0.0.0.0/0
- ALB SG: egress to node SG on app ports
- DB SG: ingress from node SG on DB port

■ **Tip:** Drop the prefix delegation knowledge — NVIDIA SRE at scale absolutely needs this; default VPC CNI burns through IPs.

AWS

Q12. How do you reduce AWS costs for a Kubernetes platform?

A: Cost drivers and levers, in priority order: (1) Compute (largest): right-size requests/limits — pods using 2GB requesting 8GB waste capacity. VPA recommendations + manual review. (2) Compute: Karpenter consolidation — packs nodes efficiently, replaces when underutilized. (3) Compute: Spot instances for fault-tolerant workloads — CI runners, batch jobs, dev environments. 60-90% savings. NVIDIA inference might use Spot for non-critical

inference. (4) Compute: Graviton (ARM) — ~20% cheaper, ~40% better perf/watt. Build multi-arch images. (5) Networking: NAT data transfer is hidden but huge. Use VPC endpoints for S3, ECR, STS — free for Gateway, cheap for Interface. (6) Storage: EBS gp3 over gp2 (cheaper, better IOPS). Lifecycle S3 to IA/Glacier. Delete unused EBS volumes / snapshots. (7) Load balancers: consolidate ALBs via host/path rules; one ALB > many. (8) Observability: cardinality control — high-cardinality labels balloon Prometheus storage. Datadog/Splunk bills scale with volume. (9) Reserved/Savings Plans for predictable baseline (3-year commit, 50%+ off). (10) Tag everything; per-team cost dashboards via Cost Explorer / Kubecost.

■ **Tip:** Mention Kubecost specifically — it's the standard for K8s cost attribution. Real production tool.

AWS

Q13. What's IAM Identity Center (formerly AWS SSO)?

A: Centralized identity and access management for multi-account AWS. Integrates with external IdP (Okta, Azure AD, Google Workspace) via SAML or its own directory. Provides: (1) Single sign-on for all AWS accounts in an Organization. (2) Permission sets — reusable IAM policies attached on-demand to user×account×role combinations. (3) Temporary credentials only — no long-lived keys in CLI configs. (4) CLI/SDK integration: aws sso login command flow. (5) Centralized audit via CloudTrail. (6) Group-based access via IdP groups. Best practice for senior orgs: (1) Identity Center for human access, never IAM users. (2) IAM roles for service-to-service. (3) OIDC federation for CI/CD. (4) MFA enforced via IdP.

■ **Tip:** Bring up Identity Center as a senior signal. Junior candidates often say 'I'd create IAM users' — that's a red flag now.

AWS

Q14. How do you implement cross-account access in AWS?

A: Two parties: source account (the principal) and target account (where the resource is). Pattern: IAM role in target account with trust policy allowing the source to assume. Source uses sts:AssumeRole.

```
# In TARGET account, create role:
# Trust policy:
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"AWS": "arn:aws:iam::SOURCE-ACCT:root"},
    "Action": "sts:AssumeRole",
    "Condition": {
      "StringEquals": {"sts:ExternalId": "shared-secret-id"},
      "Bool": {"aws:MultiFactorAuthPresent": "true"}
    }
  }]
}

# Attach permissions policy with what the role can do.

# In SOURCE account, principal calls assume role:
aws sts assume-role \
  --role-arn arn:aws:iam::TARGET-ACCT:role/CrossAccountAccess \
  --role-session-name my-session \
  --external-id shared-secret-id

# Returns temporary credentials valid for up to 12 hours.

# CLI profile that automatically assumes:
# ~/.aws/config
[profile target-readonly]
role_arn      = arn:aws:iam::TARGET-ACCT:role/ReadOnly
source_profile = default
external_id   = shared-secret-id

# In Terraform with multiple providers:
provider "aws" {
  alias = "target"
  region = "us-west-2"
  assume_role {
    role_arn = "arn:aws:iam::TARGET-ACCT:role/TerraformAccess"
```



```
    external_id = var.external_id
    session_name = "terraform-${var.environment}"
  }
}

resource "aws_s3_bucket" "in_target" {
  provider = aws.target # uses the assumed role
  bucket   = "target-account-bucket"
}
```

■ **Tip:** Your resume: 'multi-account architecture, enabling granular control.' Tell this story. ExternalId is for third-party trust (vendor accessing your account) — but use it internally too for defense in depth.

Round 7: Observability, SRE, Incident Response (45 min)

NVIDIA's JR2017386 emphasizes SRE discipline and CVE handling. This round will test SLI/SLO definitions, dashboards, alerting design, and incident response. Reference your 99.99% uptime history, Prometheus/AppDynamics work, and Splunk OTEL integration at Cisco IQ.

SRE

Q1. Define SLI, SLO, SLA, and error budget. Then design SLIs for an inference API.

A: SLI (Indicator): a measured number, usually a ratio of good/total events. E.g., availability = $\text{successful_requests} / \text{total_requests}$. SLO (Objective): the target for that SLI. E.g., availability $\geq 99.9\%$ over 30 days. Internal commitment. SLA (Agreement): contractual promise to customers, usually weaker than SLO (so SLO breach doesn't immediately = SLA breach). E.g., 99.5% with refund if missed. Error budget: $100\% - \text{SLO}$ = allowed unreliability. For 99.9% SLO over 30 days, budget = 43.2 minutes downtime. If you blow the budget, freeze feature work until reliability recovers. For an inference API specifically: (1) Availability SLI: $\text{successful_inferences} / \text{total_inferences}$. 'Successful' = HTTP 2xx AND response within timeout (e.g., 5s). (2) Latency SLI: $\text{requests_with_p99_under_500ms} / \text{total_requests}$. (3) Correctness SLI: harder — could be golden-set accuracy checks, validated drift detection. (4) Throughput SLI: not a ratio, but a capacity target — sustain 1000 req/sec.

■ **Tip:** Senior signal: distinguish event-based SLIs (most common) from time-based ones. Mention burn rate alerts.

SRE

Q2. What are the 'four golden signals'?

A: From Google SRE book. (1) Latency: how long requests take. Track p50, p95, p99 (or p99.9 for senior). Differentiate latency of successful requests vs failed — a fast 500 isn't success. (2) Traffic: request rate. Requests per second. For batch: jobs in queue, items processed per minute. For NVIDIA inference: inferences/sec. (3) Errors: failed request rate. Both explicit (HTTP 5xx) and implicit (wrong content). Track by error class — 4xx separately from 5xx; 503 separately from 504. (4) Saturation: how full your service is. CPU > 80%? Memory > 90%? Queue depth growing? Connection pool exhausted? GPU memory pressure? This is the leading indicator — saturation tells you about coming problems. Companion frameworks: RED (Rate, Errors, Duration) — service-oriented. USE (Utilization, Saturation, Errors) — resource-oriented. Use RED for services, USE for resources. Both for full picture.

■ **Tip:** Memorize 'RED for services, USE for resources' — gets cited in SRE interviews. Brendan Gregg coined USE.

SRE

Q3. How do you design alerts to avoid alert fatigue?

A: (1) Alert on symptoms, not causes. Page when users hurt (errors, latency). CPU at 95% with no user impact = ticket, not page. (2) Use SLO burn rate alerts. Two-window multi-burn-rate: page when 30-min burn rate AND 5-min burn rate both exceed threshold. Catches both fast outages and slow degradation, suppresses transients. (3) Severity levels. Page = wake someone up = user impact + needs immediate action. Ticket/notification = look at when convenient. (4) Every alert needs: runbook link, ownership (team/oncall), clear action. No 'alert exists, what do I do?' (5) Inhibition — if cluster is down, don't also page for every service in the cluster. (6) Maintenance windows — silence during planned work. (7) Review alerts after every incident. Did the alert fire too late? False positive? Tune. (8) De-dupe: routing engine groups related events. (9) Test alerts regularly (chaos drills). (10) Track noise — % of alerts that lead to action. Below 70%? You're training people to ignore pages.

■ **Tip:** Your resume: 'Validated and standardized kube-prometheus-stack (Grafana, Prometheus, AlertManager)' — tell this story.

SRE

Q4. Walk me through your incident response process.

A: Tiered response. (1) Detect: alert fires, oncall pages. Detection should be alerting on SLI violation, ideally before customers notice. (2) Acknowledge: oncall ack'd within SLA (5 min usually). Starts incident timer. (3) Triage: severity assessment. SEV1 = major customer impact. SEV2 = degraded service. SEV3 = minor / single customer. Higher severity = more people involved. (4) Mitigate first: stabilize before understanding fully. Rollback, scale up, route around, fail over. Even partial mitigation buys investigation time. (5) Communicate: status page update, exec brief, customer-facing comm if needed. NOT silence. NOT speculation. 'We're investigating, next update in 15 min.' (6) Investigate: incident commander coordinates. Engineers debug, communicator manages updates, scribe captures timeline. Use a war-room Slack channel. (7) Resolve: declare resolved only when verified. Keep monitoring for 30+ min. (8) Postmortem: blameless. Within 5 business days. Timeline, impact (in dollars or affected users), root cause, contributing factors, lessons, action items. Action items have owners and dates. (9) Track to closure: action items don't die in a doc. They should be JIRA tickets with priority.

■ **Tip:** Cite your earlier role: 'Applied discipline in incident detection and remediation, resolving high-priority escalations under SLA' — make this a story.

SRE

Q5. What's a blameless postmortem and why does it matter?

A: Blameless = focus on system failures, not individual ones. Premise: every operator has the same training and intent; they made the best decision with the information they had. If a single human can take down your system, the system is the problem. Structure: (1) Timeline with timestamps. (2) Impact (customers, revenue, duration). (3) Detection (how did we find out, was it fast enough?). (4) Contributing factors (multiple — never one root cause in complex systems). (5) Mitigation (what stopped the bleeding). (6) Action items (preventive fixes with owners and dates). What it avoids: shaming, scapegoating, defensiveness. What it enables: people speak truthfully — 'I did X without realizing Y.' Without that, you never find the real causes. Critical for senior engineering culture. Encourage attendance from non-blamed engineers (the actor) — they have key context.

■ **Tip:** Mention 'we use 5 Whys but cap at where action is possible' — shows mature framing.

Observability

Q6. Difference between logs, metrics, traces, profiles.

A: Metrics: time-series numbers. Counters, gauges, histograms. High volume + low storage cost per data point. Best for: dashboards, SLO calculation, alerting. Examples: requests_total{status='200'}, gpu_utilization. Logs: timestamped text events. High storage cost. Best for: debugging specific incidents, audit. Examples: 'User foo logged in', stack traces. Structured logs (JSON) are searchable. Traces: causal chains spanning services. Each request has a trace, composed of spans (units of work). Best for: distributed debugging — 'why is this request slow?' Tools: Jaeger, Tempo, AWS X-Ray. Spans link via parent ID + sampling. Profiles: continuous CPU/memory/lock profiling. Best for: 'where is this app spending time?' Tools: Pyroscope, Parca, eBPF-based. The 'four pillars' of observability. Modern stack: Prometheus (metrics) + Loki/Splunk/ES (logs) + Tempo/Jaeger (traces) + Pyroscope (profiles), unified via Grafana. OpenTelemetry is the standard for instrumentation.

■ **Tip:** Your resume mentions Splunk OTEL collectors — that's literally the modern observability standard. Drop OTEL into the answer.

Observability

Q7. Design a Prometheus + Grafana stack for a Kubernetes cluster.

A: Standard pattern: kube-prometheus-stack Helm chart.

```
# Components (deployed via Helm chart kube-prometheus-stack):
# - Prometheus Operator (manages Prometheus + ServiceMonitor CRDs)
# - Prometheus (time-series DB; scrapes targets)
# - Alertmanager (deduplicates, routes alerts to Slack/PagerDuty)
# - Grafana (dashboards)
# - node-exporter (per-node metrics; runs as DaemonSet)
# - kube-state-metrics (cluster-level metrics from Kubernetes API)
# - kubernetes-mixin (default dashboards + alert rules)

# Critical metrics to scrape:
# 1. node-exporter: CPU, memory, disk, network per node
# 2. kube-state-metrics: pod state, deployment replicas, PVC status
```

```
# 3. cAdvisor (built into kubelet): per-container resource use
# 4. App metrics via ServiceMonitor – each service exposes /metrics
# 5. Karpenter: node provisioning latency, consolidation events
# 6. nvidia-dcgm-exporter: GPU utilization, memory, temp, ECC errors

# ServiceMonitor – declarative scrape config:
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: inference-api
spec:
  selector:
    matchLabels: {app: inference}
  endpoints:
    - port: metrics
      interval: 15s
      path: /metrics

# Alert rules via PrometheusRule:
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: inference-slo
spec:
  groups:
    - name: inference
      rules:
        - alert: InferenceErrorRateHigh
          expr: |
            sum(rate(http_requests_total{job="inference",status=~"5.."}[5m]))
            / sum(rate(http_requests_total{job="inference"}[5m])) > 0.01
          for: 5m
          labels: {severity: page}
          annotations:
            summary: "Inference 5xx rate > 1% for 5 min"
            runbook: https://wiki/runbooks/inference-errors

# Long-term storage (Prometheus is short-retention by default):
# - Thanos: federated Prometheus with object storage (S3)
# - Mimir: Grafana's horizontally-scalable Prometheus alternative
# - VictoriaMetrics: efficient, single-binary alternative

# Federation for multi-cluster:
# Each cluster has local Prometheus.
# Global Thanos Query aggregates across clusters via Sidecar pattern.
```

■ **Tip:** Mention *dcgm-exporter* (NVIDIA's GPU metric exporter) — NVIDIA cares deeply that you know this exists.

Observability

Q8. What's high cardinality and why does it matter?

A: Cardinality = number of unique time series produced by a metric. Each unique combination of label values = a new series. Prometheus stores each as a separate stream. Series count drives memory, disk, query cost. Anti-patterns: (1) `user_id` label — millions of users = millions of series. (2) `request_id` — every request is unique. (3) Unbounded paths `/api/v1/users/123` → label per path → unbounded. The trap: cardinality is multiplicative. `http_requests_total{path, method, status, region, cluster}` — if each label has 10 values, that's $10^5 = 100K$ series for ONE metric. Mitigations: (1) Don't use unique IDs as labels. Use traces for that. (2) Aggregate at instrumentation — `request_count_total` without `user_id`. (3) Use exemplars — link a trace ID from a metric data point. Trace stored separately. (4) Drop labels at scrape time via `metric_relabel_configs`. (5) Use recording rules to pre-aggregate. (6) For very-high-cardinality: ClickHouse / VictoriaMetrics handle it better than Prometheus.

■ **Tip:** Senior signal. Mention 'cardinality bombs' — when a deploy adds a high-cardinality label and balloons Prometheus.

Observability

Q9. What should be on a production dashboard?

A: Layered. Top of dashboard: the answer to 'is it broken?' visible in 5 seconds. Then drill-down. Layer 1 — Service health: golden signals (latency, traffic, errors, saturation). Status: green/yellow/red. SLO burn rate gauge.

Deployment version. Layer 2 — Per-route breakdown: error rate by endpoint, latency p99 by endpoint. Top slowest endpoints. Layer 3 — Resources: CPU, memory, network, disk per pod. Container restarts. OOM count. HPA scale events. Layer 4 — Dependencies: latency to DB, cache, downstream services. Error rate to each. Layer 5 — Infrastructure: node health, cluster autoscaler / Karpenter activity, AWS API errors. Layer 6 — Recent changes: deployment timeline, config changes, incident annotations. Best practice: dashboard answers 'is it broken?' first, 'why?' second. Don't put CPU graphs at top — they don't tell users they're affected.

■ **Tip:** Your resume: 'Designed observability dashboards and predictive analytics AI/ML workflows.' Cite this directly.

Observability

Q10. How do you handle a critical CVE in a base container image?

A: JR2017386 explicitly calls out 'performing scans and handling of security CVEs.' Speak to the playbook. (1) Triage: severity is necessary but not sufficient — also need exploitability in YOUR context. Internet-facing services with the vulnerable lib in the request path = high actual risk. Internal-only sidecar = lower. Map CVSS + exposure. (2) Identify: which images, which deployments, which environments? SBOM (Trivy, Gype, syft) gives you this. (3) Patch: upgrade base image, dependency, or kernel. If upstream fix isn't out, evaluate mitigations: WAF rules, network policy restricting affected paths, disabling vulnerable features. (4) Rebuild: trigger image rebuild in CI. Verify with scan that it's clean. (5) Roll out: dev → stage → prod through normal CI/CD. For SEV1 CVEs: expedite through approval, but maintain canary discipline. (6) Verify: post-rollout scan confirms zero affected pods running. (7) Document: incident-style postmortem. Why was vulnerable version in production? Was scanning catching it? (8) Prevention: scheduled rebuilds even without code changes — pulls in latest base patches. Distrosless or minimal base images reduce attack surface.

■ **Tip:** Bring up your Wiz integration from the resume — that's the modern scanning platform. NVIDIA uses Wiz too.

SRE

Q11. How do you measure and reduce toil?

A: Toil = manual, repetitive, automatable, tactical, devoid-of-enduring-value work. Identify: track time spent on operational tasks. If oncall spends 4 hours/week on 'restart this thing that fails every Tuesday' — that's toil. Survey engineers periodically. Reduce: (1) Eliminate at source — fix the bug that causes the manual fix. (2) Automate the response — self-healing scripts triggered by alert. (3) Automate the detection — better monitoring so you catch BEFORE it requires manual action. (4) Empower — self-service tooling so users don't file tickets. (5) Standardize — templates, paved roads reduce per-team toil. Measure: % of SRE time on toil vs project work. Google SRE book targets <50% toil; senior orgs aim for 30%. Track: tickets/week trend, alerts/oncall trend, runbook usage. Your resume: 'Reduced PR-authoring cycle time by ~60% by combining Claude CLI agentic loops with skills-driven scaffolding.' That's textbook toil reduction.

■ **Tip:** Make it a story. NVIDIA hires senior DevOps to PERMANENTLY KILL toil, not to do it more efficiently.

SRE

Q12. Graceful degradation — what is it?

A: Continuing to serve users with reduced functionality when dependencies fail, rather than full outage. Patterns: (1) Circuit breaker: open the circuit (stop calling the failing dependency) after N failures. Allow trickle of requests to detect recovery (half-open). (2) Bulkhead: isolate failure domains. Don't share thread pools across critical and best-effort calls — one runaway shouldn't take everything. (3) Fail open vs fail closed: feature flags off = fall back to safe default. For NVIDIA inference: if model server is overloaded, return cached prediction or downgraded model. (4) Cache aggressively. Stale data > no data for non-critical reads. (5) Timeout + retry + jitter for transient failures. NOT infinite retry. (6) Rate limit at edge to protect downstream. (7) Async processing for non-critical flows — queue, retry later. Real example: NVIDIA inference API losing connection to feature store. Options: serve from local cache (graceful), return 503 (degraded), block all requests (full outage). Choose based on use case.

■ **Tip:** Bring up Hystrix or resilience4j circuit breaker patterns. Senior signal.

Round 8: Python Coding & Data Structures (60-90 min)

NVIDIA HackerRank-style coding round. For Senior DevOps, expect 2 medium problems with DevOps flavoring rather than pure algorithmic LeetCode. Focus: correct + readable + handles edge cases + production-quality code. Show your work — narrate while coding.

Coding

Q1. Implement a function that parses an arbitrary log format string (Apache-style) and returns structured records.

A: Real DevOps problem — log parsing with configurable format.

```
import re
from typing import Dict, List, Optional
from dataclasses import dataclass
from datetime import datetime

# Apache common log format:
# %h %l %u %t \"%r\" %>s %b
# 192.168.1.1 - - [16/May/2026:10:00:00 +0000] "GET /api HTTP/1.1" 200 1234

LOG_FORMAT_PATTERNS = {
    '%h': r'(?P<host>\S+)',
    '%l': r'(?P<ident>\S+)',
    '%u': r'(?P<user>\S+)',
    '%t': r'\[(?P<time>[^\]]+)\]',
    '%r': r'(?P<request>[\""]*)',
    '%>s': r'(?P<status>\d+)',
    '%b': r'(?P<bytes>\S+)',
    '%{Referer}i': r'(?P<referer>[\""]*)',
    '%{User-agent}i': r'(?P<user_agent>[\""]*)',
}

def compile_format(fmt: str) -> re.Pattern:
    """Convert Apache format string to compiled regex."""
    pattern = fmt
    # Replace longer tokens first to avoid prefix collision
    for token in sorted(LOG_FORMAT_PATTERNS, key=len, reverse=True):
        pattern = pattern.replace(token, LOG_FORMAT_PATTERNS[token])
    return re.compile('^' + pattern + '$')

def parse_log_line(line: str, compiled: re.Pattern) -> Optional[Dict]:
    """Parse a single line. Returns None for malformed."""
    m = compiled.match(line.strip())
    if not m:
        return None
    return m.groupdict()

def parse_log(lines: List[str], fmt: str) -> List[Dict]:
    compiled = compile_format(fmt)
    return [r for line in lines if (r := parse_log_line(line, compiled))]

# Test
common_fmt = '%h %l %u %t \"%r\" %>s %b'
lines = [
    '192.168.1.1 - - [16/May/2026:10:00:00 +0000] "GET /api HTTP/1.1" 200 1234',
    '10.0.0.5 - alice [16/May/2026:10:00:01 +0000] "POST /login HTTP/1.1" 401 89',
]
for record in parse_log(lines, common_fmt):
    print(record)
```

■ **Tip:** Production-quality details: walrus operator for clean filtering, dataclass-able output, regex compiled once.

Coding

Q2. Write a function to rate-limit API calls. Token bucket algorithm, thread-safe.

```
import threading
import time
```

```

from typing import Optional

class TokenBucket:
    """
    Thread-safe token bucket rate limiter.

    Allows bursts up to `capacity`, refills at `refill_rate` tokens/sec.
    Useful for: API rate limiting, throttling outbound calls, smoothing traffic.
    """
    def __init__(self, capacity: int, refill_rate: float):
        if capacity <= 0:
            raise ValueError('capacity must be positive')
        if refill_rate <= 0:
            raise ValueError('refill_rate must be positive')
        self.capacity = capacity
        self.refill_rate = refill_rate
        self._tokens = float(capacity)
        self._last_refill = time.monotonic()
        self._lock = threading.Lock()

    def _refill(self) -> None:
        now = time.monotonic()
        elapsed = now - self._last_refill
        added = elapsed * self.refill_rate
        self._tokens = min(self.capacity, self._tokens + added)
        self._last_refill = now

    def acquire(self, n: int = 1, blocking: bool = True,
                timeout: Optional[float] = None) -> bool:
        """
        Try to acquire n tokens. Returns True on success.
        If blocking=True, wait until available (or timeout).
        """
        deadline = (time.monotonic() + timeout) if timeout else None
        while True:
            with self._lock:
                self._refill()
                if self._tokens >= n:
                    self._tokens -= n
                    return True
                if not blocking:
                    return False
                # Calculate how long until enough tokens
                deficit = n - self._tokens
                wait = deficit / self.refill_rate
            # Release lock while sleeping
            if deadline and time.monotonic() + wait > deadline:
                return False
            time.sleep(min(wait, 0.1)) # cap to keep responsive

    def __repr__(self):
        with self._lock:
            self._refill()
            return f'TokenBucket(tokens={self._tokens:.2f}/{self.capacity})'

# Usage
bucket = TokenBucket(capacity=10, refill_rate=2.0) # 10 burst, 2/sec sustained

# Decorator form:
from functools import wraps
def rate_limited(bucket: TokenBucket):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            bucket.acquire(blocking=True)
            return fn(*args, **kwargs)
        return wrapper
    return decorator

@rate_limited(bucket)
def call_api():
    ...

```


■ **Tip:** Mention you'd use a proven library (*limits*, *slowapi*) in real production — but knowing the algorithm matters for design.

Coding

Q3. Given access to a Kubernetes API, write a Python script to find all pods that have been restarting in the last hour.

```
from kubernetes import client, config
from datetime import datetime, timezone, timedelta
from typing import List, Tuple

def get_restarting_pods(
    threshold: int = 1,
    window_minutes: int = 60,
    namespace: str = None,
) -> List[Tuple[str, str, int, datetime]]:
    """
    Returns pods that have restarted >= threshold times within window.
    Output: list of (namespace, pod_name, restart_count, last_restart_time)
    """
    # Load config – in-cluster (pod) or local kubeconfig
    try:
        config.load_incluster_config()
    except config.ConfigException:
        config.load_kube_config()

    v1 = client.CoreV1Api()
    cutoff = datetime.now(timezone.utc) - timedelta(minutes=window_minutes)

    if namespace:
        pods = v1.list_namespaced_pod(namespace).items
    else:
        pods = v1.list_pod_for_all_namespaces().items

    restarting = []
    for pod in pods:
        if not pod.status.container_statuses:
            continue
        total_restarts = sum(c.restart_count for c in pod.status.container_statuses)
        if total_restarts < threshold:
            continue
        # Check last-restart-time
        last_restart = None
        for c in pod.status.container_statuses:
            if c.last_state and c.last_state.terminated:
                term_time = c.last_state.terminated.finished_at
                if term_time and (not last_restart or term_time > last_restart):
                    last_restart = term_time
        if last_restart and last_restart >= cutoff:
            restarting.append((
                pod.metadata.namespace,
                pod.metadata.name,
                total_restarts,
                last_restart,
            ))

    return sorted(restarting, key=lambda x: x[3], reverse=True)

if __name__ == '__main__':
    for ns, name, count, when in get_restarting_pods(threshold=2, window_minutes=60):
        print(f'{ns:20} {name:40} restarts={count:3} last={when}')
```

■ **Tip:** Mention you'd export this as a Prometheus metric or alert. Tie to your observability work.

Coding

Q4. Implement a simple in-memory cache with TTL. Thread-safe.

```
import threading
import time
from typing import Any, Optional, Hashable

class TTLCache:
```

```

"""Thread-safe cache with per-entry TTL."""

def __init__(self, default_ttl: float = 60.0):
    self._data: dict = {} # key -> (value, expires_at_monotonic)
    self._lock = threading.RLock()
    self.default_ttl = default_ttl

def get(self, key: Hashable) -> Optional[Any]:
    with self._lock:
        entry = self._data.get(key)
        if entry is None:
            return None
        value, expires = entry
        if time.monotonic() >= expires:
            del self._data[key]
            return None
        return value

def set(self, key: Hashable, value: Any, ttl: Optional[float] = None) -> None:
    ttl = ttl if ttl is not None else self.default_ttl
    with self._lock:
        self._data[key] = (value, time.monotonic() + ttl)

def delete(self, key: Hashable) -> bool:
    with self._lock:
        return self._data.pop(key, None) is not None

def cleanup(self) -> int:
    """Remove expired. Returns count removed."""
    now = time.monotonic()
    with self._lock:
        expired = [k for k, (_, exp) in self._data.items() if now >= exp]
        for k in expired:
            del self._data[k]
    return len(expired)

def __len__(self):
    with self._lock:
        return len(self._data)

def __contains__(self, key):
    return self.get(key) is not None

# Production note: cachetools.TTLCache is the standard library
# but rolling your own shows you understand the pattern.

```

■ **Tip:** If asked: *distributed TTL cache = Redis with EXPIRE. Don't reinvent at scale.*

Coding

Q5. Parse a config file (INI format), validate keys/types, and emit env vars suitable for export.

```

import configparser
import os
from typing import Dict, Any
from dataclasses import dataclass, field

@dataclass
class ConfigSchema:
    """Required keys + types for validation."""
    required: Dict[str, type] = field(default_factory=lambda: {
        'database.host': str,
        'database.port': int,
        'database.ssl': bool,
        'app.replicas': int,
        'app.log_level': str,
    })
    valid_values: Dict[str, set] = field(default_factory=lambda: {
        'app.log_level': {'DEBUG', 'INFO', 'WARN', 'ERROR'},
    })

class ConfigValidationError(Exception):
    pass

```

```

def parse_value(raw: str, expected_type: type) -> Any:
    if expected_type is bool:
        v = raw.strip().lower()
        if v in ('true', 'yes', '1', 'on'):
            return True
        if v in ('false', 'no', '0', 'off'):
            return False
        raise ValueError(f'invalid bool: {raw!r}')
    if expected_type is int:
        return int(raw)
    if expected_type is float:
        return float(raw)
    return raw.strip()

def load_config(path: str, schema: ConfigSchema = None) -> Dict[str, Any]:
    schema = schema or ConfigSchema()
    cp = configparser.ConfigParser()
    cp.read(path)

    flat: Dict[str, Any] = {}
    for section in cp.sections():
        for key, raw_value in cp.items(section):
            flat[f'{section}.{key}'] = raw_value

    # Validate + convert
    errors = []
    parsed: Dict[str, Any] = {}
    for key, expected_type in schema.required.items():
        if key not in flat:
            errors.append(f'missing required key: {key}')
            continue
        try:
            value = parse_value(flat[key], expected_type)
        except ValueError as e:
            errors.append(f'{key}: {e}')
            continue
        if key in schema.valid_values and value not in schema.valid_values[key]:
            errors.append(f'{key}: {value!r} not in {schema.valid_values[key]}')
            continue
        parsed[key] = value

    if errors:
        raise ConfigValidationError('\n'.join(errors))
    return parsed

def to_env_vars(config: Dict[str, Any], prefix: str = 'APP') -> Dict[str, str]:
    """Convert 'database.host' -> 'APP_DATABASE_HOST=value'."""
    return {
        f'{prefix}_{key.upper().replace(".", "_')}': str(v).lower() if isinstance(v, bool) else str(v)
        for key, v in config.items()
    }

# Usage:
# cfg = load_config('/etc/app/config.ini')
# env = to_env_vars(cfg)
# for k, v in env.items():
#     print(f'export {k}={v!r}')

```

■ **Tip:** *Dataclass for schema is the modern Python pattern. Show type validation, not just parsing.*

Coding

Q6. Walk through Python's hash table internals. Time complexity of dict get/put.

A: Python dict (3.6+) is an open-addressed hash table with perturbation probing. Two arrays: a dense entries array (insertion-ordered) and a sparse hash table that indexes into entries. Hash computed via `hash()` — for str, int, tuple of hashable types, etc. Lookup steps: (1) Compute `hash(key)` modulo table size — index into hash table. (2) If slot empty → `KeyError`. (3) If slot occupied, compare full hash THEN equality (fast hash check before slow eq). (4) Match → return value from entries array. Mismatch → perturb (next slot via `PERTURB_SHIFT`). Average $O(1)$ amortized for get/put. Worst case $O(n)$ if many collisions (intentionally bad hash function, or 'hash flooding' attack — Python adds hash randomization for str since 3.3). Resize: when load factor $> 2/3$, table doubles. Resize is $O(n)$ but rare — amortized $O(1)$ holds. Memory: dict in 3.6+ uses $\sim 1/2$ memory of pre-3.6 design

(sparse table is small index integers, entries array is compact). del leaves a 'dummy' marker (tombstone) — actual cleanup on resize.

■ **Tip:** If asked about thread safety: dict operations are GIL-atomic at the bytecode level for simple ops. Multi-step ops (check-then-set) still need locks.

Coding

Q7. Implement quicksort. When does it perform poorly?

```
from typing import List, TypeVar
import random

T = TypeVar('T')

def quicksort(arr: List[T]) -> List[T]:
    """Out-of-place quicksort. Average  $O(n \log n)$ , worst  $O(n^2)$ ."""
    if len(arr) <= 1:
        return arr
    # Random pivot avoids worst case for sorted input
    pivot = random.choice(arr)
    less = [x for x in arr if x < pivot]
    equal = [x for x in arr if x == pivot]
    greater = [x for x in arr if x > pivot]
    return quicksort(less) + equal + quicksort(greater)

def quicksort_inplace(arr: List, lo: int = 0, hi: int = None) -> None:
    """In-place Lomuto partition."""
    if hi is None:
        hi = len(arr) - 1
    if lo < hi:
        p = partition(arr, lo, hi)
        quicksort_inplace(arr, lo, p - 1)
        quicksort_inplace(arr, p + 1, hi)

def partition(arr: List, lo: int, hi: int) -> int:
    # Random pivot — swap to end
    rand_idx = random.randint(lo, hi)
    arr[rand_idx], arr[hi] = arr[hi], arr[rand_idx]
    pivot = arr[hi]
    i = lo
    for j in range(lo, hi):
        if arr[j] < pivot:
            arr[i], arr[j] = arr[j], arr[i]
            i += 1
    arr[i], arr[hi] = arr[hi], arr[i]
    return i

# Worst case  $O(n^2)$  when:
# 1. Always picking the smallest/largest as pivot (sorted input + first/last pivot)
# 2. Many duplicate values + naive partition (mitigate with 3-way partition)
#
# Mitigations:
# 1. Random pivot (above)
# 2. Median-of-three: pick median of first/middle/last as pivot
# 3. Introsort: switch to heapsort if recursion depth exceeds  $2 \cdot \log(n)$ 
# (Python's sorted() uses Timsort —  $O(n \log n)$  worst case, stable)
```

■ **Tip:** If asked why Python uses Timsort, not quicksort: stability matters, and Timsort exploits real-world partial-order in $O(n)$ for nearly-sorted data.

Coding

Q8. How do stacks and heaps differ? Insert/delete time complexity for a heap?

A: Stack: LIFO data structure. Last-in-first-out. push() to top, pop() from top. Both $O(1)$. In Python: list.append() + list.pop(). Used for: function call frames, parsing, undo, DFS. Heap: a tree-based structure satisfying heap property. Min-heap: parent $\leq$ children. Implemented as an array (parent at i , children at $2i+1$, $2i+2$). Operations: peek-min $O(1)$, insert $O(\log n)$, extract-min $O(\log n)$, heapify-from-array $O(n)$. Python's heapq is a min-heap. For max-heap, negate values. Memory-wise: 'stack' also refers to a thread's call stack (process stack, allocated on thread creation, typically 8MB, contains stack frames). 'Heap' also refers to the process heap (dynamically allocated memory via malloc/mmap; managed by libc allocator). Don't conflate the data structures with the

memory regions, but interviewers sometimes use either meaning.

■ **Tip:** Senior: be ready to explain both meanings if asked which one the interviewer means.

Coding

Q9. Code a function: given a list of GPU job durations and N GPUs, return the makespan if jobs are greedily assigned to the GPU with shortest current queue.

A: Heap-based scheduling — relevant to NVIDIA inference / training cluster scheduling.

```
import heapq
from typing import List

def makespan(jobs: List[int], n_gpus: int) -> int:
    """
    Returns total time to finish all jobs.
    Greedy: assign each job to the GPU finishing earliest.

    Time: O(J log G) where J=jobs, G=GPUs.
    Note: this greedy is OPTIMAL only if all jobs known in advance
    AND scheduled in any order. For online scheduling, you'd want
    longest-processing-time-first (LPT) for a 4/3-OPT approximation.
    """
    if n_gpus <= 0 or not jobs:
        return 0
    # Sort jobs longest-first (LPT heuristic) for better balance
    jobs = sorted(jobs, reverse=True)
    # Min-heap of (current_finish_time, gpu_id)
    gpus = [(0, i) for i in range(n_gpus)]
    heapq.heapify(gpus)
    for duration in jobs:
        finish, gpu_id = heapq.heappop(gpus)
        heapq.heappush(gpus, (finish + duration, gpu_id))
    # Makespan = max finish time across GPUs
    return max(f for f, _ in gpus)

# Test
print(makespan([5, 3, 4, 1, 2], n_gpus=2)) # 8 (one GPU: 5+2, other: 4+3+1)
print(makespan([10, 1, 1, 1], n_gpus=3)) # 10 (longest dominates)

# Extension: track WHICH jobs land on each GPU
def schedule(jobs: List[int], n_gpus: int):
    jobs_indexed = sorted(enumerate(jobs), key=lambda x: -x[1])
    gpus = [(0, i, []) for i in range(n_gpus)]
    heapq.heapify(gpus)
    while jobs_indexed:
        job_idx, duration = jobs_indexed.pop(0)
        finish, gpu_id, assigned = heapq.heappop(gpus)
        assigned.append(job_idx)
        heapq.heappush(gpus, (finish + duration, gpu_id, assigned))
    return sorted([(gpu_id, finish, jobs) for finish, gpu_id, jobs in gpus])
```

■ **Tip:** NVIDIA-flavor problem. Mention scheduling theory: LPT for offline ($P||C_{max}$) is 4/3 of optimal. Online is hard.

Coding

Q10. Implement a function to detect cycles in a DAG of pipeline stages (build dependencies).

```
from typing import Dict, List, Set

def has_cycle(graph: Dict[str, List[str]]) -> bool:
    """
    Detect cycle in directed graph using DFS + 3-color marking.
    graph = {node: [neighbors]}
    """
    WHITE, GRAY, BLACK = 0, 1, 2
    color = {node: WHITE for node in graph}

    def dfs(node: str) -> bool:
        color[node] = GRAY
        for nb in graph.get(node, []):
            if color.get(nb, WHITE) == GRAY:
                return True
```

```

        return True    # back edge to ancestor → cycle
    if color.get(nb, WHITE) == WHITE and dfs(nb):
        return True
    color[node] = BLACK
    return False

for node in graph:
    if color[node] == WHITE:
        if dfs(node):
            return True
return False

```

```

def topological_sort(graph: Dict[str, List[str]]) -> List[str]:
    """
    Returns nodes in topological order. Raises if cycle exists.
    Kahn's algorithm (BFS-based).
    """
    from collections import defaultdict, deque
    in_degree = defaultdict(int)
    for node in graph:
        in_degree[node]    # ensure node is in dict
        for nb in graph[node]:
            in_degree[nb] += 1

    queue = deque([n for n in in_degree if in_degree[n] == 0])
    result = []
    while queue:
        node = queue.popleft()
        result.append(node)
        for nb in graph.get(node, []):
            in_degree[nb] -= 1
            if in_degree[nb] == 0:
                queue.append(nb)

    if len(result) != len(in_degree):
        raise ValueError('cycle detected')
    return result

```

```

# Use:
pipeline = {
    'checkout': ['lint', 'unit-test'],
    'lint': ['build'],
    'unit-test': ['build'],
    'build': ['deploy'],
    'deploy': [],
}
print(topological_sort(pipeline))
# ['checkout', 'lint', 'unit-test', 'build', 'deploy']

# This is exactly how CI/CD systems compute job order from dependencies.

```

■ **Tip:** DevOps-flavored DAG question. Senior signal: mention Kahn's vs DFS-based, and that real CI systems (Argo Workflows, Tekton) use this internally.

Coding

Q11. Given a list of network connection events (open/close), find the maximum number of concurrent connections.

A: Classic sweep-line.

```

from typing import List, Tuple

def max_concurrent(events: List[Tuple[int, str]]) -> int:
    """
    events: list of (timestamp, action) where action is 'open' or 'close'
    Returns: peak concurrent count.

    O(N log N) due to sort.
    """
    # Sort by timestamp; 'close' before 'open' at same time
    # (so a close+open at same time doesn't artificially inflate)
    sorted_events = sorted(events, key=lambda e: (e[0], 0 if e[1] == 'close' else 1))

    current = 0

```

```

    peak = 0
    for _, action in sorted_events:
        if action == 'open':
            current += 1
            peak = max(peak, current)
        else:
            current -= 1
    return peak

# Test
events = [
    (1, 'open'),
    (2, 'open'),
    (3, 'close'),
    (3, 'open'),
    (4, 'open'),
    (5, 'close'),
    (5, 'close'),
    (6, 'close'),
]
print(max_concurrent(events))    # 3

# Variant: given (start, end) intervals, find peak concurrent
def max_concurrent_intervals(intervals: List[Tuple[int, int]]) -> int:
    events = []
    for s, e in intervals:
        events.append((s, 'open'))
        events.append((e, 'close'))
    return max_concurrent(events)

```

■ **Tip:** Sweep-line pattern shows up often. Production version: streaming log of connection events → real-time gauge.

Coding

Q12. Write a function that flattens a deeply nested dict for use as Prometheus labels.

```

from typing import Dict, Any, Iterator

def flatten(d: Dict, parent: str = '', sep: str = '_') -> Dict[str, Any]:
    """
    Flatten nested dict with sep-joined keys.
    Lists become indexed: items_0, items_1.
    Used for: Prometheus labels (which must be flat strings),
              log shipping, config exports.
    """
    out = {}
    for k, v in d.items():
        new_key = f'{parent}{sep}{k}' if parent else str(k)
        if isinstance(v, dict):
            out.update(flatten(v, new_key, sep))
        elif isinstance(v, list):
            for i, item in enumerate(v):
                if isinstance(item, (dict, list)):
                    out.update(flatten({str(i): item}, new_key, sep))
                else:
                    out[f'{new_key}{sep}{i}'] = item
        else:
            out[new_key] = v
    return out

# Iterative version (avoids recursion stack for deep nesting):
def flatten_iterative(d: Dict, sep: str = '_') -> Dict[str, Any]:
    out = {}
    stack = [('', d)]
    while stack:
        prefix, current = stack.pop()
        if isinstance(current, dict):
            for k, v in current.items():
                new = f'{prefix}{sep}{k}' if prefix else str(k)
                stack.append((new, v))
        elif isinstance(current, list):
            for i, item in enumerate(current):
                stack.append((f'{prefix}{sep}{i}', item))
        else:
            out[prefix] = current
    return out

```



```

        out[prefix] = current
    return out

# Test
nested = {
    'service': 'inference',
    'region': 'us-west-2',
    'metadata': {
        'owner': 'platform-team',
        'version': '1.2.3',
        'tags': ['prod', 'critical'],
    },
}
print(flatten(nested))
# {'service': 'inference', 'region': 'us-west-2',
#  'metadata_owner': 'platform-team', ...}

```

■ **Tip:** Iterative version shows depth awareness. Mention you'd cap recursion (`sys.setrecursionlimit`) for very deep input.

Coding

Q13. Implement a sliding window log aggregator: count events per minute, keep last hour.

```

from collections import deque
from datetime import datetime, timedelta
from threading import Lock
from typing import Dict, Optional

class SlidingWindowCounter:
    """
    Counts events in 1-minute buckets, retains last `window_minutes`.
    Thread-safe. Bounded memory (60 buckets for hourly window).
    """
    def __init__(self, window_minutes: int = 60):
        self.window = timedelta(minutes=window_minutes)
        # deque of (bucket_start_datetime, count)
        self._buckets: deque = deque()
        self._lock = Lock()

    def _bucket_for(self, ts: datetime) -> datetime:
        # Truncate to minute
        return ts.replace(second=0, microsecond=0)

    def add(self, ts: Optional[datetime] = None, count: int = 1) -> None:
        ts = ts or datetime.now()
        bucket = self._bucket_for(ts)
        with self._lock:
            # Add to existing bucket if last one matches
            if self._buckets and self._buckets[-1][0] == bucket:
                last_ts, last_count = self._buckets.pop()
                self._buckets.append((last_ts, last_count + count))
            else:
                self._buckets.append((bucket, count))
            self._evict(ts)

    def _evict(self, now: datetime) -> None:
        cutoff = now - self.window
        while self._buckets and self._buckets[0][0] < cutoff:
            self._buckets.popleft()

    def total(self, now: Optional[datetime] = None) -> int:
        now = now or datetime.now()
        with self._lock:
            self._evict(now)
            return sum(c for _, c in self._buckets)

    def per_minute(self, now: Optional[datetime] = None) -> Dict[datetime, int]:
        now = now or datetime.now()
        with self._lock:
            self._evict(now)
            return dict(self._buckets)

# Usage

```

```

counter = SlidingWindowCounter(window_minutes=60)
counter.add()          # +1 now
counter.add(count=5)
print(counter.total())

```

■ **Tip:** Bounded memory is key. Sliding window $O(\text{buckets})$ memory, not $O(\text{events})$. Production fit.

Coding

Q14. Given a binary tree, do iterative in-order traversal (no recursion).

```

from typing import Optional, List

class Node:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def inorder(root: Optional[Node]) -> List:
    """
    Iterative in-order: left, root, right.
    Uses explicit stack – avoids Python's recursion limit (~1000).
    """
    result = []
    stack = []
    curr = root
    while curr or stack:
        # Push all left children
        while curr:
            stack.append(curr)
            curr = curr.left
        # Pop, visit, then move to right
        curr = stack.pop()
        result.append(curr.val)
        curr = curr.right
    return result

# Test
#      1
#     / \
#    2   3
#   / \
#  4   5
tree = Node(1, Node(2, Node(4), Node(5)), Node(3))
print(inorder(tree))  # [4, 2, 5, 1, 3]

# Variations they might ask:
# - Pre-order: root, left, right → push right then left
# - Post-order: left, right, root → two-stack or reversed pre-order trick
# - Level-order: BFS with deque

```

■ **Tip:** Iterative versions show you know recursion has limits. Real-world: deep trees (deep file systems, deep XML) will blow stack.

Coding

Q15. Two-pointer problem: given a sorted array, find all pairs that sum to target.

```

from typing import List, Tuple

def two_sum_sorted(arr: List[int], target: int) -> List[Tuple[int, int]]:
    """
    Sorted array, find all pairs (i, j) where i < j and arr[i] + arr[j] == target.
    O(n) time, O(1) extra space (besides output).
    """
    pairs = []
    left, right = 0, len(arr) - 1
    while left < right:
        s = arr[left] + arr[right]
        if s == target:
            pairs.append((arr[left], arr[right]))
            left += 1
            right -= 1
        # Skip duplicates if asked for unique pairs

```

```

        while left < right and arr[left] == arr[left - 1]:
            left += 1
        while left < right and arr[right] == arr[right + 1]:
            right -= 1
    elif s < target:
        left += 1
    else:
        right -= 1
return pairs

# Unsorted version: hash table
def two_sum_hash(arr: List[int], target: int) -> List[Tuple[int, int]]:
    seen = {}
    pairs = []
    for x in arr:
        complement = target - x
        if complement in seen:
            pairs.append((complement, x))
        seen[x] = True
    return pairs

```

■ **Tip:** Two-pointer is the bread and butter of senior coding rounds. Always mention complexity.

Coding

Q16. How would you do BFS on a graph? When use BFS vs DFS?

```

from collections import deque
from typing import Dict, List, Set, Optional

def bfs(graph: Dict, start) -> List:
    """
    BFS – visits in order of distance from start.
    O(V + E) time, O(V) space.
    """
    visited: Set = set()
    order: List = []
    queue = deque([start])
    visited.add(start)
    while queue:
        node = queue.popleft()
        order.append(node)
        for nb in graph.get(node, []):
            if nb not in visited:
                visited.add(nb)
                queue.append(nb)
    return order

def bfs_shortest_path(graph: Dict, start, end) -> Optional[List]:
    """BFS gives shortest UNWEIGHTED path. For weighted, use Dijkstra."""
    if start == end:
        return [start]
    visited = {start}
    queue = deque([(start, [start])])
    while queue:
        node, path = queue.popleft()
        for nb in graph.get(node, []):
            if nb == end:
                return path + [nb]
            if nb not in visited:
                visited.add(nb)
                queue.append((nb, path + [nb]))
    return None

# When to use which:
# BFS:
# - Shortest path in unweighted graph
# - Level-order traversal
# - Find connected components by distance
# - Minimum steps in a state graph (e.g., shortest sequence of moves)
#
# DFS:
# - Topological sort
# - Cycle detection
# - Connected components by exploration

```

```
# - Memory: DFS stack is O(depth), BFS queue is O(breadth)
#           tree with 1M leaves: BFS uses 1M memory, DFS uses log(1M) = 20
```

■ **Tip:** *Memory difference is the senior point. NVIDIA infra trees can be huge — pick by traversal pattern + memory constraint.*

Coding

Q17. How are linked lists used in production systems?

A: Real uses (beyond LeetCode): (1) LRU cache — combo of doubly-linked list + hash map. Each node carries key+value, list maintains recency order, hash provides O(1) lookup. (2) Kernel scheduler — task lists per priority. (3) Garbage collector free-lists for memory allocator. (4) Express VLAN tagging in network stacks. (5) Database undo logs (linked log records). (6) Filesystem extents (inodes pointing to data block linked lists). (7) Buffer pools. Production caveats: pointer chasing is cache-unfriendly. Modern CPUs love sequential memory access (arrays). Linked lists are correct in theory but slow in practice for traversal-heavy workloads. For sequential access prefer array-backed structures (Vec in Rust, ArrayList, std::vector).

```
from typing import Optional, Any

class Node:
    __slots__ = ('val', 'prev', 'next') # smaller memory footprint
    def __init__(self, val):
        self.val = val
        self.prev: Optional['Node'] = None
        self.next: Optional['Node'] = None

class DoublyLinkedList:
    def __init__(self):
        # Sentinel head/tail simplifies edge cases
        self.head = Node(None)
        self.tail = Node(None)
        self.head.next = self.tail
        self.tail.prev = self.head
        self._size = 0

    def add_front(self, val) -> Node:
        node = Node(val)
        node.next = self.head.next
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node
        self._size += 1
        return node

    def remove(self, node: Node) -> None:
        node.prev.next = node.next
        node.next.prev = node.prev
        self._size -= 1

    def __len__(self):
        return self._size

    def __iter__(self):
        curr = self.head.next
        while curr is not self.tail:
            yield curr.val
            curr = curr.next
```

■ **Tip:** *Senior signal: '__slots__' shrinks per-instance memory by 30-40%' — production-aware Python detail.*

Coding

Q18. Implement an exponential backoff with jitter retry decorator.

A: Already covered in Round 2, but interviewers often re-ask in coding round at higher depth — async version.

```
import asyncio
import random
from functools import wraps
from typing import Callable, Type, Tuple

def async_retry(
```

```

max_attempts: int = 5,
base_delay: float = 1.0,
max_delay: float = 60.0,
factor: float = 2.0,
retriable: Tuple[Type[BaseException], ...] = (Exception,),
):
    """Async retry with exponential backoff + jitter."""
    def decorator(fn: Callable):
        @wraps(fn)
        async def wrapper(*args, **kwargs):
            last_exc = None
            for attempt in range(1, max_attempts + 1):
                try:
                    return await fn(*args, **kwargs)
                except retriable as e:
                    last_exc = e
                    if attempt == max_attempts:
                        break
                    delay = min(base_delay * (factor ** (attempt - 1)), max_delay)
                    # Full jitter (more even spread than just *0.5)
                    delay = random.uniform(0, delay)
                    await asyncio.sleep(delay)
            raise last_exc
        return wrapper
    return decorator

# Usage
@async_retry(max_attempts=4, retriable=(ConnectionError,))
async def fetch_metric(url: str):
    # ...
    pass

```

■ **Tip:** Sync AND async versions matter. NVIDIA infra is async-heavy.

Round 9: System Design (60-90 min)

For Senior DevOps, system design rounds favor INFRASTRUCTURE systems over consumer products. Examples: design a CI/CD platform, design observability stack, design multi-region failover, design secrets platform, design rate limiter. Whiteboard. Always ask clarifying questions first.

System Design

Q1. Design a CI/CD platform for an AI inference team that ships 100 services across multi-cloud.

A: Start with requirements. Multi-cloud (AWS, Azure, GCP, on-prem). 100 services. AI inference workloads. Senior-level expected scale. Walk through ARCH framework: Actors → Requirements → Components → HA → Risks.

```
# Clarifying questions to ask first:
# - How many deploys per day? (e.g., 500-1000 across services)
# - What's the rollback SLO? (e.g., 5 minutes to revert)
# - Required compliance? (SOC2, ISO27001)
# - Self-service for app teams or centrally managed?
# - Multi-region active-active or active-passive?
# - Latency budget for inference services?

# High-level architecture:

[Developers] → [GitHub (mono or polyrepo)]
      ▼
[GitHub Actions] ← OIDC → [AWS / Azure / GCP]
      ■
      ■■■ Lint, test, scan (Trivy, CodeQL, gitleaks)
      ■■■ Build images (buildx multi-arch: amd64+arm64)
      ■■■ Sign (cosign)
      ■■■ Push to registry (per-cloud: ECR / GCR / ACR + central Harbor for mirror)
      ▼
[GitOps repo] (org/gitops-config)
      ■■■ apps/<svc>/<env>/values.yaml ← image tag updated by CI bot
      ▼
[Argo CD] – one instance per cluster, or hub-spoke
      ■■■ ApplicationSet generators: per-cluster, per-env
      ■■■ Sync policies: prod = manual approval, dev/stage = auto
      ■■■ Rollout strategy: Argo Rollouts for canary
      ▼
[Multi-cloud K8s Clusters]
      ■■■ EKS (AWS) – primary
      ■■■ AKS (Azure) – secondary
      ■■■ GKE (GCP) – secondary
      ■■■ On-prem (OpenShift) – legacy/edge

# Core components:
# - Identity: IAM Identity Center + OIDC federation
# - Secrets: External Secrets Operator → cloud-native secret managers + Vault
# - Image registry: Harbor (org-wide) with replication to cloud-native registries
# - Observability: Prometheus → Mimir/Thanos (cross-cluster), Loki for logs, Tempo for traces
# - Policy: Kyverno admission policies enforced cluster-side
# - Cost: Kubecost per cluster, FinOps dashboards

# Failure modes / tradeoffs:
# - GitHub Actions outage → builds blocked. Mitigate: self-hosted runners + Jenkins backup.
# - GitOps repo as SPOF → mirror to multiple Git servers; cache rendered manifests.
# - Cross-region deploy ordering → strict promotion: dev → stage-region1 → stage-region2 → prod-region1 ...
# - Image pull explosion at deploy → registry mirroring per region, pull-through cache.

# Scale considerations:
# - 100 services × 4 envs × 3 regions = 1,200 deployment targets
# - ApplicationSet (one per service per env) scales to thousands cleanly
```

- # - Argo CD app-of-apps with project segmentation for tenancy
- # Day-2 concerns:
 - # - Onboarding new service: template repo + auto-generated GitOps PR
 - # - Deprovisioning: PR removes config; Argo prunes (with safeguards)
 - # - Compliance evidence: every deploy gets a record in audit log

■ **Tip:** Always start with clarifying questions. Drawing the boxes is 10% of the answer; reasoning about failure modes and scale is the senior 90%.

System Design

Q2. Design an observability stack for 50 EKS clusters across 3 regions.

A: Hub-and-spoke pattern. Per-cluster collection, central long-term storage.

```
# Requirements:
# - 50 clusters × 3 regions × 100s of pods = millions of metrics
# - Retention: 30 days online, 1 year archive
# - Alerts < 1 min latency
# - Logs < 5 min latency to query
# - Traces < 1 min for active service

# Architecture (per region, then federate):

# Per-cluster (lightweight):
[K8s cluster]
  ■■■ Prometheus (15-min retention, scrapes locally)
  ■■■ Promtail/Vector (ships logs)
  ■■■ OTEL collector (traces + metrics)
  ■■■ Fluent Bit (logs to central)
  ■■■ node-exporter, kube-state-metrics, dcgm-exporter (GPU)

# Per-region (central):
[Mimir cluster] – Prometheus-compatible, long-term storage
  ■■■ Storage: S3 per region (~$0.023/GB/month)
  ■■■ Distributor → Ingester → Compactor → Querier
  ■■■ Multi-tenant via X-Scope-OrgID header

[Loki cluster] – log aggregation, S3-backed
  ■■■ Distributor → Ingester → Compactor → Querier
  ■■■ Chunks in S3, indexes in BoltDB-shipper or TSDB

[Tempo cluster] – trace storage, S3-backed
  ■■■ Distributor → Ingester → Compactor → Querier
  ■■■ Sampling at OTEL collector (5-10% default, 100% on errors)

# Global:
[Grafana] – federated query across regions
  ■■■ Multiple Mimir / Loki / Tempo datasources (one per region)

[Alertmanager] – HA cluster, dedups across regions
  ■■■ Route by severity → PagerDuty / Slack / Email
  ■■■ Inhibition rules per service tier

# Critical design choices:
# - Per-cluster Prom = isolation; central Mimir = federation
# - S3 for long-term = cheap, durable
# - Sampling for traces (100% would explode cost)
# - Recording rules at cluster level to pre-aggregate before federation
# - Mimir's per-tenant rate limits stop one team's metrics from
#   crashing the cluster

# Cost levers:
# - Cardinality control (drop high-cardinality labels at scrape)
# - Sampling traces
# - Log retention tiering (hot 7d → warm 30d → cold 1y in S3 IA)
# - Recording rules reduce queries on raw data

# Failure modes:
# - Region outage → other regions still observable; cross-region
#   alerts continue
# - Mimir down → per-cluster Prom retains 15 min, scraping continues
# - S3 outage → ingest path queues briefly, alerts may delay
# - Cardinality bomb → tenant rate limit + alert to team
```



```
# Alerts to set on the stack itself:
# - Prometheus target down
# - Mimir ingester saturation
# - Loki query latency p99
# - S3 5xx rate
```

■ **Tip:** Mention OpenTelemetry as the standard collector. Vendor-neutral — NVIDIA values portability.

System Design

Q3. Design a multi-region active-active inference platform with sub-100ms latency.

```
# Requirements:
# - 3 regions: us-west-2, eu-west-1, ap-southeast-1
# - p99 latency < 100ms
# - 99.99% availability (≤ 52 min/yr downtime)
# - Active-active: traffic served from all regions simultaneously
# - Stateful component? Yes — model weights, request cache, audit log

# High-level architecture:

[Users globally]
  ▼
[Route 53 latency-based routing] ← Returns closest healthy region
  ▼
[Per-region CloudFront] ← Edge cache for static assets
  ▼
[Per-region ALB]
  ▼
[EKS cluster] (per region)
  ■■■ Inference service (Triton, vLLM)
  ■ - Autoscale with KEDA on request queue depth
  ■ - GPU node pools per model size
  ■ - Karpenter consolidation
  ■■■ Request routing/queueing (HAProxy or Envoy)
  ■■■ Model cache (mounted EBS gp3 + warm models in mem)
  ■■■ Sidecar: telemetry, audit logger
  ▼
[Region-local persistence]
  ■■■ DynamoDB Global Tables — request logs, idempotency keys
  ■■■ S3 Cross-Region Replication — model artifacts
  ■■■ ElastiCache (Redis) — local cache (NOT cross-region)

# Cross-region concerns:
# - Model deploy: same artifact version in all regions
#   - CI builds once, replicates to per-region registries
#   - GitOps deploys same image tag to all clusters atomically
# - Failover: Route 53 health checks; failed region pulled in < 60s
# - Data consistency: eventual via DynamoDB Global Tables
#   - For audit/billing: at-least-once writes, idempotent processing

# Latency budget (100ms total):
# - DNS resolution: 5ms (cached client-side after first)
# - TLS handshake (resumption): 5ms
# - ALB → pod: 1ms (in-region)
# - Inference: 70-90ms (the bulk; model + batching)
# - Network egress: 5ms

# Critical failure modes:
# - Whole region down → Route 53 health checks pull region from DNS,
#   other regions handle. Capacity must be N+1 for any region.
# - Model deploy bad in one region → canary fails health checks,
#   that region drained, fix forward, redeploy.
# - DynamoDB Global Tables conflict resolution: last-writer-wins
#   (timestamp-based). For idempotency, use conditional writes.
# - GPU shortage in one region → fallback to nearby region via
#   Route 53 health check OR explicit cross-region request routing
#   (rare, costly).
```

```
# Scale levers:
# - HPA on requests-per-pod, NOT CPU (CPU misleading for GPU work)
# - KEDA scaling on SQS depth / Kafka lag for async paths
# - Pre-warming pods at predicted peak times (cron HPA)
# - Multi-instance GPU (MIG) for memory partitioning

# Cost:
# - 3 regions × N+1 capacity = expensive (2x cheaper to run 2 regions)
# - Mitigate: smaller fleet in trough regions, surge into expected peaks
```

■ **Tip:** This is essentially Dynamo/NIXL's problem space. Show you understand the latency budget and that GPU costs dominate. Mention MIG specifically.

System Design

Q4. Design a secrets management platform for 1000 microservices across 50 K8s clusters.

```
# Architecture:

[App teams]
  ■ (rotate via UI/API)
  ▼
[Vault cluster] (per region, primary+performance replica)
  ■ AppRole / Kubernetes auth for clusters
  ■ OIDC for humans
  ▼
[External Secrets Operator] (per cluster)
  ■ Pulls secret values, creates K8s Secret in app namespace
  ▼
[Pod] consumes via env vars or mounted volume

# Layers:
# 1. Source of truth: Vault (or AWS Secrets Manager + KMS)
#   - HA cluster, audit log to S3
#   - Auto-unseal via KMS auto-unseal
#   - Secret engines: KV v2 (versioned), DB credentials (dynamic), PKI
#
# 2. Sync layer: ExternalSecrets Operator
#   - SecretStore CRD: defines source (Vault/AWS SM) + auth method
#   - ExternalSecret CRD: declares which key → which K8s Secret
#   - Refresh interval: 1 hour default, immediate via webhook
#
# 3. Consumption: pod via standard K8s Secret
#   - env: secretKeyRef
#   - volume: secret with items
#   - subPath for files

# Identity (no static credentials anywhere):
# - Cluster → Vault: K8s service account JWT exchange
# - Cluster → AWS Secrets Manager: IRSA
# - Vault → AWS APIs: instance profile or IRSA
# - Humans → Vault: OIDC via Okta/SSO

# Rotation:
# - DB credentials: Vault generates per-pod dynamic creds (15-min TTL)
# - Pod gets a fresh user/password on startup; old revoked.
# - Static secrets: rotate in Vault, ExternalSecret syncs in <1hr,
#   pod restart triggered by checksum annotation change.
# - Certs: cert-manager auto-renews via ACME or PKI engine.

# Multi-region:
# - Vault Performance Replication: writes to primary, reads from replicas
# - DR Replication: full standby cluster, manual failover
# - Per-region ExternalSecrets: each cluster talks to local Vault

# Audit and compliance:
# - Vault audit log → S3 → Splunk
# - Who accessed what secret, when, from which pod
# - SOC2 / ISO27001 evidence collection automated

# Failure modes:
# - Vault down: pods can't pull secrets at startup (BAD).
# - Mitigation: cache last-good secret values locally (with TTL),
```

```
# so existing pods keep working. New pods may fail to start.
# - ExternalSecret stuck: secret value stale.
# Mitigation: alert on lastSyncTime > threshold.
# - Rotate-without-restart: app must re-read secret.
# Pattern: file watcher in app, or sidecar that restarts on change,
# or checksum annotation forcing deployment rollout.
```

```
# Anti-patterns to avoid:
# - K8s Secret in Git plaintext (or even base64)
# - Vault root token in CI
# - Cross-team Secret access without RBAC
# - Long-lived static secrets (rotate everything)
# - Vault as the only path → cache for resilience
```

■ **Tip:** You did exactly this at Cisco IQ: External Secrets + AWS Secrets Manager + IRSA + Vault for compliance. Tell the story while answering.

System Design

Q5. Design a global rate limiter for an inference API.

```
# Requirements:
# - Limit per-API-key request rate (e.g., 1000 req/sec/key, burst 2000)
# - Globally consistent across regions
# - < 5ms p99 to check rate
# - Survive partial outage (fail-open vs fail-closed?)

# Three architectures, tradeoffs:

# OPTION 1: Local rate limiter per pod
# Pros: Zero latency, no shared state
# Cons: Each pod has independent budget – total = N × per_pod
# Use: When approximate is OK and pods don't share customers
# Implementation: Token bucket in-process (like Round 8 TokenBucket)

# OPTION 2: Centralized Redis per region
# Pros: Per-region consistency, low latency (1-2ms intra-region)
# Cons: Doesn't aggregate across regions
# Use: When regions are independent or affinity-routed
# Implementation: Redis with INCR + EXPIRE per key
#   key: rate:<api_key>:<minute>
#   on each request: INCR + check limit
#   uses sliding window log or fixed window depending on need
#   atomic via Lua script

# Lua script for sliding window:
local key = KEYS[1]
local window = tonumber(ARGV[1])
local limit = tonumber(ARGV[2])
local now = tonumber(ARGV[3])

redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
local count = redis.call('ZCARD', key)
if count < limit then
    redis.call('ZADD', key, now, now)
    redis.call('EXPIRE', key, math.ceil(window / 1000))
    return 1
end
return 0

# OPTION 3: Global state via DynamoDB Global Tables
# Pros: Cross-region consistent (eventually)
# Cons: 10-50ms per check; conflict resolution lag (~1 sec)
# Use: When per-customer global limit is hard requirement
# Implementation: ConditionalUpdateItem with version check

# Recommended hybrid:
# - Per-region Redis cluster (Elasticache or Memcached)
# - Per-region budget = global_limit / num_regions × buffer
# - Cross-region reconciliation every minute (correction)
# - Fail-open if Redis down (better to overserve than refuse legitimate traffic)
# - Async usage tracking → DynamoDB for billing/quota tracking

# Edge concerns:
# - Burst handling: token bucket > fixed window
```

```
# - Headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset
# - 429 Too Many Requests with Retry-After header
# - Don't return 429 too quickly – clients hammer retry. Use 429 with jitter advice.

# Failure modes:
# - Redis outage: fall back to per-pod local limiter (degraded but works)
# - DynamoDB lag: per-region limits may overshoot global
# - Hot key: shard by hashing api_key + bucket index

# Costs:
# - Redis hits: cheap (~$0 incremental per request)
# - DynamoDB: $1.25 per million writes – at 1M req/sec = $1.25/sec = $108K/day. ouch.
# Mitigation: aggregate writes locally, flush every minute.
```

■ **Tip:** *Fail-open vs fail-closed is the senior signal. For inference, fail-open (legitimate user gets served) > fail-closed (false 429s).*

System Design

Q6. Design a system that promotes container images from dev to prod with automated security gating.

```
# Pipeline:

[Source]
■
▼
[CI build] – produces image with metadata
■
■■■ Build (buildx, multi-arch)
■■■ SBOM generation (syft)
■■■ Vulnerability scan (trivy / grype)
■■■ License scan (FOSSA / OSS Review Toolkit)
■■■ Image signing (cosign + KMS key)
■■■ Push to ECR with immutable tag (SHA256)
■
▼
[Promotion gates]
■
▼
[Dev environment]
■■■ Pulled by tag
■■■ Functional tests
■■■ Synthetic prober
■■■ 24-48hr soak
■ (auto-promote PR if all green)
▼
[Staging environment]
■■■ Performance tests
■■■ Security re-scan (CVE DB updated?)
■■■ Compliance checks (CIS, NIST)
■■■ Canary 5% real traffic
■ (manual approval to promote)
▼
[Production environment]
■■■ Canary 5% → 25% → 50% → 100%
■■■ Auto-rollback on SLO burn
■■■ Image must be: signed, scanned, approved

# Admission gating (cluster-side enforcement):

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: verify-image-signature
spec:
  validationFailureAction: enforce
  rules:
    - name: check-image-signature
      match:
        any:
          - resources:
              kinds: [Pod]
              namespaces: [prod-*]
```

```

verifyImages:
  - imageReferences: ["*.ecr.*.amazonaws.com/*"]
  attestors:
    - entries:
      - keys:
        publicKey: |-
          -----BEGIN PUBLIC KEY-----
          ...
          -----END PUBLIC KEY-----

# Vulnerability exception process:
# - Critical CVE in image → block (Trivy exit 1 on CRITICAL)
# - Exceptions require: docs, expiration date, compensating control,
#   approval from security team. Tracked in Jira.
# - Expired exceptions trigger automated re-scan and block.

# Image immutability:
# - ECR repo with immutability=enabled (can't overwrite tags)
# - Promotion = same digest moves through envs
# - GitOps: image tag in values.yaml is the SHA256 digest

# Provenance (supply-chain):
# - SLSA Level 3: provenance attests build process
# - In-toto attestations attached to image
# - Verify provenance at admission

# Failure modes:
# - CVE published mid-promotion: scan re-runs, image blocked at next gate
# - Signing key compromise: revoke + re-sign all current prod images
# - Build env compromise: ephemeral runners + provenance verify

```

■ **Tip:** NVIDIA's JR2017386 calls out CVE handling explicitly. This entire flow is the answer to 'how do you handle CVEs at scale.'

System Design

Q7. Design a system for deploying and managing models on a GPU fleet.

A: NVIDIA-flavor question — they want to see you understand inference orchestration.

```

# Components:

[Model registry] (S3 + DynamoDB metadata)
  ■
  ▼
[Model serving layer] (Triton / vLLM / TorchServe per model)
  ■ Pod per model instance, GPU-pinned
  ▼
[Routing layer] (Envoy / KServe / Seldon)
  ■ Routes by model name + version
  ▼
[Inference clients]

# GPU fleet management:
[Karpenter NodePools]
  ■■■ h100-large: 8x H100, NVLink full, for biggest models
  ■■■ a10g-medium: 1-4x A10G, smaller models or batched inference
  ■■■ cpu-only: for CPU fallback / metadata services

# Each NodePool has:
# - Taints: nvidia.com/gpu, MIG configuration
# - Tolerations on inference pods
# - nvidia-device-plugin DaemonSet for GPU discovery
# - dcgm-exporter for GPU metrics

# MIG (Multi-Instance GPU):
# - Single H100 splits into N partitions (2x40GB, 7x10GB, etc.)
# - For small models: pack multiple onto one GPU
# - For large models: single MIG=disabled instance with full 80GB

# Scheduling:
# - HPA on request queue depth (NOT GPU%, which is misleading)
# - KEDA driving from Prometheus metrics
# - Pre-warming for known traffic patterns (cron-based scale-up)
# Model lifecycle:

```

```
# 1. Model team uploads to MLflow / model registry
# 2. CI converts to Triton format (TensorRT, ONNX optimization)
# 3. Image baked with model OR mounted from S3 via init container
# 4. Argo CD deploys ModelDeployment CR
# 5. Canary: route 5% traffic to new version
# 6. Compare metrics: latency, accuracy on golden set, drift
# 7. Promote: shift traffic 100%
# 8. Retire: old version stays in registry; can rollback

# Cost optimization:
# - GPU spot instances for non-critical (50-90% savings)
# - Auto-scale to zero for off-peak models
# - Aggressive consolidation: Karpenter drops underutilized GPU nodes
# - Mixed precision: FP16/INT8 quantization fits more on each GPU

# Observability:
# - Per-model latency p50/p99
# - GPU utilization (dcgm-exporter): SM activity, memory, temp, ECC
# - Queue depth per model
# - Token throughput for LLMs
# - Cost per inference

# Failure modes:
# - Model returns NaN: validate output at server, return 503
# - OOM on GPU: pod restart, MIG re-partition
# - GPU hardware failure: dcgm marks node unhealthy, Karpenter drains
# - Drift detection: golden-set checks fail → auto-rollback
```

■ **Tip:** This is NVIDIA's HOME TURF. Mention Triton, MIG, dcgm-exporter, TensorRT. They'll know if you're faking expertise.

System Design

Q8. How would you architect log aggregation for 10,000 pods producing 50TB/day of logs?

```
# Pipeline:

[10,000 pods]
  ■ stdout/stderr → kubelet
  ▼
[Fluent Bit DaemonSet] (one per node)
  ■■■ Reads /var/log/containers/*.log
  ■■■ Adds metadata: pod name, namespace, labels
  ■■■ Parses JSON if structured
  ■■■ Buffers to disk if downstream slow
  ■■■ Ships to:
  ■
  ▼
[Vector aggregator] (per region, optional)
  ■■■ Routing, filtering, sampling
  ■■■ Transforms (drop noisy logs, redact PII)
  ■
  ▼
[Storage tier – tiered by access pattern]
  ■
  ■■■ HOT (last 7 days): Loki or Elasticsearch (queryable in seconds)
  ■   - Loki: cheaper, S3-backed chunks, labels-based search
  ■   - ES: faster full-text, more expensive
  ■
  ■■■ WARM (8-30 days): same backend, IA storage class
  ■
  ■■■ COLD (31-365 days): S3 Glacier Deep Archive
  ■   Restore on demand for audit/forensics
  ■
  ▼
[Query layer: Grafana / Kibana]

# 50TB/day = 580 MB/sec sustained → 17 PB/year (a lot!)

# Cost levers (in order of impact):
# 1. Drop debug logs (-30%): use level filtering before shipping
# 2. Sample noisy logs (-30%): /healthz, /metrics endpoints – sample 1%
# 3. Compress in transit (-70% bandwidth): gzip/zstd
```

```
# 4. Compress at rest (-80% storage): Loki chunks compressed
# 5. Tiered storage (-60% cold cost): S3 IA → Glacier
# 6. Retention tuning: 90 days? Or 30 + archive? Per regulation
# 7. Label cardinality control: limit label values per stream

# Schema enforcement (at app level):
# - JSON structured logs only (raw text harder to query)
# - Required fields: timestamp, level, service, trace_id
# - PII scrubbing at source (regex in Fluent Bit)

# Failure modes:
# - Aggregator down: Fluent Bit buffers to disk (1GB default per node)
# - Storage 5xx: backpressure → retry → eventual drop if buffer full
# - Cardinality bomb: per-tenant limits, alert on high-cardinality labels
# - Slow query: Loki labels must be low-cardinality (no user_id labels)

# Multi-tenancy:
# - Loki tenants by team
# - Per-tenant rate limits, retention policies, dashboards
# - Cross-tenant queries restricted by RBAC

# Audit / compliance:
# - Audit log stream separated from app logs
# - Immutable storage (S3 Object Lock + versioning)
# - Encryption: KMS keys per tenant
# - Retention: 7 years for some compliance regimes
```

■ **Tip:** *Splunk OTEL collectors from your resume — bring up. Tiered storage is the cost-conscious senior signal.*

Round 10: Scenario-Based / War Stories (45-60 min)

Hiring-manager favorite. They describe a complex production scenario and want to see your judgment under uncertainty. Use TROUBLE framework: Trigger → Reproduce → Observe → Unblock → Build-fix → Learn → Eliminate recurrence. Lean heavily on your real Cisco IQ stories — EKS cluster recovery, multi-region S3 fixes, ArgoCD multi-account rollout, OpenShift-to-EKS migration.

Scenario

Q1. It's 3am. PagerDuty fires: p99 latency on inference API has spiked from 80ms to 8 seconds. What do you do, in order?

A: (1) Acknowledge in 60 seconds. Stop the clock — show I'm responding. (2) Check the obvious in 2 minutes: dashboards for the API service. Is it ALL traffic or specific routes? All regions or one? All pods or some? Recent deploys in the last hour? Drop the high-severity blast doors: shed load with rate limit, or rollback if a deploy correlates. (3) If recent deploy: rollback IMMEDIATELY via Argo CD or argo rollouts abort. Don't try to forward-fix at 3am with one engineer. Restore service first. (4) Comm out: status page update, exec on-call notified, customer support team alerted. (5) Dig in: traces for affected requests — is it inference compute, network egress, downstream dependency? GPU utilization spiked? Memory pressure causing swap? (6) Identify root cause (might take 1-2 hours). (7) Apply fix. (8) Verify recovery: p99 back to baseline, error rate normal, queue depth normal. (9) Post-incident at 9am next day: blameless RCA with timeline, action items.

■ **Tip:** Always 'mitigate first, diagnose second.' Senior interviewers HATE answers that start with 'I'd open the dashboards and investigate.' Wrong — restore service first.

Scenario

Q2. Your EKS cluster suddenly has 50% of pods Pending. Diagnose in real-time.

A: This happened to you at Cisco IQ — tell the actual story.

```
# Step 1: Confirm scope
kubectl get pods -A | grep Pending | wc -l
kubectl get nodes
# Are nodes Ready? If many NotReady, that's the cause.

# Step 2: Pick one Pending pod and trace it
kubectl describe pod <pending-pod> | tail -30
# Events tell us why scheduler is stuck

# Most likely causes for 'half the cluster Pending':

# Cause A: Karpenter / autoscaler failed
kubectl logs -n karpenter -l app.kubernetes.io/name=karpenter --tail=200
# Look for: AWS API failures, ICE (Insufficient Capacity), IAM issues
# Fix: AWS support, fallback to different instance types, broaden NodePool

# Cause B: Quota exhausted
kubectl describe quota -A
# Did someone deploy a bad spec requesting 100Gi memory per pod?
# Fix: Roll back the bad spec; resize quotas if intentional.

# Cause C: AZ went down (1 AZ = 1/3 capacity in 3-AZ cluster)
aws ec2 describe-availability-zones --region us-west-2
# Status of each AZ
# Fix: Wait for AZ recovery; ensure topology spread allows survival.

# Cause D: VPC subnet IP exhaustion (AWS VPC CNI is greedy with IPs)
kubectl get nodes -o jsonpath='{.items[*].status.capacity.pods}'
# If max-pods hits subnet limit, no more pods can land
# Diagnose: aws ec2 describe-subnets, check available IPs
# Fix: Enable prefix delegation, add secondary CIDR (100.64.0.0/16),
# or add subnets

# Cause E: Cluster autoscaler / control plane issue
kubectl get componentstatuses # deprecated but quick check
```



```
kubectl get apiservices
kubectl logs -n kube-system kube-apiserver-* 2>/dev/null
# If API server is overloaded, schedules slow down

# Story from your resume:
# 'Diagnosed EKS cluster recovery: root-caused Fargate CoreDNS
# isolation to a missing node-SG to cluster-primary-SG ingress rule;
# produced 29-section resource inventory with five supporting PRs.'
```

■ **Tip:** Tell *THIS* story. The interviewer hears it as 'this person has actually been in the trenches.'

Scenario

Q3. A new container deploy passed all CI checks but in production it's leaking memory at 100MB/hour. How do you respond?

A: Multi-phase. (1) IMMEDIATE: confirm impact. How many pods? How fast until OOM? When will customers feel it (likely soon — 12-24hr to OOM). Decide: revert now (clean) or live-fix (risky). For most production scenarios with growing leak: revert. (2) REVERT: git revert in GitOps repo, Argo CD syncs. Validate previous version stable. Buy time. (3) INVESTIGATE without prod pressure: reproduce in staging with same load profile if possible. Use memory profiler (pyroscope, py-spy, jemalloc heap profile). Look for: unbounded caches, leaked connections (DB pool, HTTP client), accumulated request state, leaked goroutines/threads. (4) FIX: identify the leak source, write a targeted patch. Add test case that catches it (e.g., 1-hour soak test with memory assertion). (5) RE-DEPLOY: canary 5% with extended bake (24hr min), then progressive rollout. Hold elevated alerting for 48 hours. (6) POSTMORTEM: why didn't CI catch it? Add memory regression tests, add 1-hour soak in stage. Update runbook.

■ **Tip:** The senior signal: 'I'd add a soak test that catches *THIS* class of bug, not just this exact bug.' That's investing in the system.

Scenario

Q4. Your CI pipeline takes 45 minutes per PR. Engineers complain. Reduce it.

A: (1) Measure first — which stages take how long? Get a flame graph of the pipeline. (2) Identify bottlenecks: dependency install (40% of CI time typical), tests (30%), build (20%), scans (10%) — usually. (3) Quick wins first: (a) Cache dependencies (actions/cache for pip, node_modules, Gradle). Single biggest impact. (b) Cache Docker layers (buildx GHA cache or remote cache). (c) Parallel test execution (pytest-xdist, jest --maxWorkers, Go's t.Parallel). (d) Test sharding across N runners. (e) Skip checks not relevant to changed files (e.g., backend tests skip on docs PR). (4) Medium effort: (a) Larger runners (4-core → 16-core often pays for itself). (b) Self-hosted runners on Spot. (c) Pre-built builder images with deps baked in. (d) Incremental builds (Bazel, Nx, Turborepo) — only rebuild changed. (5) Long-term: (a) Test pyramid review — too many slow integration tests? Move to unit. (b) Flaky test elimination — retries hide real cost. (c) Split the repo if it's a giant monorepo with unrelated services. (6) Measure again to verify wins. My target: PR feedback under 15 minutes. Your story: at Cisco IQ you reduced PR-authoring cycle time ~60% via Claude CLI agentic loops and skills-driven scaffolding — cite this.

■ **Tip:** Quantify. 'I cut 25 minutes' is weak. 'I reduced pip install from 4 min to 30 sec via cache, then parallelized tests to drop suite from 18 min to 5 min' shows you actually did it.

Scenario

Q5. AWS just announced a CVE in the EKS-managed Nitro hypervisor. All your prod clusters are affected. What's your response?

A: Real scenario — AWS does this every 6-12 months. (1) Triage in first 30 min: read AWS bulletin carefully. Severity (CVSS), exploitability (does AWS ship the fix automatically, or do we need to act?). For Nitro: AWS often patches transparently via live migration; for some, you need to recycle instances. (2) Identify scope: which clusters, which node groups, which regions. SBOM helps. (3) Decision: planned vs emergency rotation. If actively exploited → emergency: rolling cordon-drain-replace nodes within hours. If theoretical risk → planned 24-48hr window. (4) Communicate to stakeholders BEFORE acting. (5) Execute: trigger AMI rebuild + roll. Karpenter makes this clean — update node-class AMI selector, Karpenter drift-detects and replaces nodes one at a time. Each replacement is cordon → drain (respecting PDB) → terminate → new node. (6) Monitor during roll: cluster events, app SLO burn. Pause if health degrades. (7) Verify completion: every node on new AMI, manifest with

vulnerability scan. (8) Postmortem: how fast did we detect, how fast did we patch, was the runbook adequate? Update if needed.

■ **Tip:** Mention Karpenter drift handling specifically — modern senior pattern. Also mention PodDisruptionBudgets — you respect them during drain.

Scenario

Q6. Your Argo CD application is OutOfSync but you didn't change Git. What's happening?

A: Live cluster state differs from rendered Git desired state. Causes (in order of likelihood): (1) Someone made a manual change (kubectl edit, console). Check: who recently touched the cluster (audit log). (2) A controller is modifying fields Argo CD tracks. Common culprits: HPA modifying replicas, mutating webhooks (Kyverno, Istio injection), AWS Load Balancer Controller adding annotations, cert-manager adding annotations. (3) Default values changed — Kubernetes API server fills in defaults, or new field added in version upgrade. Diff often noisy around image: tag → tag@digest. (4) ServerSideApply field ownership conflicts. (5) Argo CD's diff normalizer being too strict.

```
# Investigation
argocd app diff <app-name>      # see exact diff
argocd app get <app-name> -o json | jq .status.resources

# Or via UI: 'App Details' → drill into the resource

# Fix paths:

# Fix 1: If a controller is the owner, tell Argo to ignore that field
spec:
  ignoreDifferences:
    - group: apps
      kind: Deployment
      jsonPointers: [/spec/replicas]    # HPA owns this

# Fix 2: If it's a sync wave issue (CRD not installed before CR)
metadata:
  annotations:
    argocd.argoproj.io/sync-wave: "-10"  # CRDs first

# Fix 3: If it's serverside apply ownership
spec:
  syncPolicy:
    syncOptions:
      - ServerSideApply=true
      - RespectIgnoreDifferences=true

# Fix 4: If someone made a manual change, decide:
# - Drift was correct (e.g., scaling up in emergency)
#   → backport to Git, then sync
# - Drift was wrong → sync forces Git desired state
# Self-heal=true auto-corrects (use carefully in prod)
```

■ **Tip:** Your resume: ArgoCD multi-account rollout with OPA tag hygiene 12→0 warnings. That implies you've debugged OOS many times.

Scenario

Q7. A senior engineer pushes a Terraform change that deletes a production RDS instance. The plan was approved but on apply, you realize the change is wrong. The apply is currently running. What do you do?

A: Don't panic, but move fast. (1) Cancel the apply IMMEDIATELY if possible: Ctrl-C or kill the runner. Apply has likely passed 'Plan' phase and is mid-destroy. Catch it before resources are removed. (2) Assess damage: terraform state list to see what's still there. AWS console to see RDS instance state — Available? Deleting? Already gone? (3) If RDS is mid-delete and you have a recent snapshot: take a manual snapshot RIGHT NOW if it's still 'Available' or 'Modifying'. If 'Deleting', AWS may take a final snapshot but it's not guaranteed unless skip_final_snapshot=false. (4) If gone: restore from snapshot. RTO depends on instance size (10-30+ minutes for large). (5) Block additional applies: comment 'STOP' on PR, pause Atlantis, lock state. Don't compound the

damage. (6) Communicate immediately: SEV1 incident. Database team, app team that depends on it, exec. (7) Restore service: redeploy app pointing to restored DB (probably new endpoint — update via Secrets Manager + ExternalSecret rotation). (8) Postmortem within 24 hours. Action items: lifecycle prevent_destroy on production DBs (this should have been there), required CODEOWNERS for prod TF, plan review checklist including 'any destroys?', different state files for different blast radii.

■ **Tip:** Senior signal: 'this should never happen because prevent_destroy.' Mention the long-term fix, not just the immediate response.

Scenario

Q8. Capacity planning crisis: your inference API just got featured on Hacker News and traffic is 10x in 30 minutes. Walk me through.

A: (1) Confirm reality first: dashboards show the spike, not a metric bug. Check error rate — are we still serving? (2) Scale aggressively: HPA should be scaling. Check max replicas — bump if at ceiling. Karpenter should be adding nodes — check Karpenter logs for capacity errors (ICE = insufficient capacity in AWS at this instance type). (3) If hitting AZ/instance-type capacity: broaden node pool. Allow more instance types/sizes/AZs in Karpenter NodePool. AWS will provision what's available. (4) Shed less critical load: rate-limit at edge (CloudFront, ALB), prioritize paying customers, return 503s for free tier with Retry-After header. Better to gracefully reject than serve all slowly. (5) Cache aggressively if possible: CloudFront, app-level cache, model-level cache for repeated queries. (6) Comm: status page note about elevated load, ETA on stabilization. (7) Bring up cross-region capacity if active-active. Route 53 latency routing automatically directs. (8) Cost-aware: short-term spike OK to overspend. Track for hourly billing review. (9) After the spike: postmortem. Was autoscaling fast enough? Was the rate limit set right? Do we need pre-warmed capacity for known launches?

■ **Tip:** Mention 'graceful degradation' explicitly: better to serve 80% well than 100% poorly. Junior answers say 'scale infinitely'; senior says 'shed load when needed.'

Scenario

Q9. Your engineering team wants to migrate from on-prem OpenShift to AWS EKS. You're the platform lead. Outline the migration plan.

A: Your resume LITERALLY did this. Walk through it.

```
# Phase 1: Discovery (Weeks 1-3)
# - Inventory workloads: pods, services, ingresses, CRDs, operators
# - Map OpenShift-specific features:
#   - Routes → K8s Ingress + AWS Load Balancer Controller
#   - SCCs (SecurityContextConstraints) → PodSecurityAdmission + Kyverno
#   - DeploymentConfigs → standard Deployments (Argo Rollouts for canary)
#   - Internal registry → ECR
#   - oc CLI workflows → kubectl + argocd CLI
# - Compliance/security requirements
# - Stakeholders: app teams, security, compliance, infrastructure

# Phase 2: AWS Landing Zone (Weeks 4-7)
# - Multi-account structure: shared services, dev, stage, prod
# - VPC architecture (3 AZs, private nodes, NAT, VPC endpoints)
# - IAM Identity Center for human SSO
# - Centralized logging (CloudTrail org trails, VPC flow logs to S3)
# - Networking to on-prem (Direct Connect or Site-to-Site VPN)

# Phase 3: EKS platform foundation (Weeks 8-12)
# - EKS clusters per env via Terraform/Terragrunt
# - Standard add-ons (your resume names these exactly):
#   - External Secrets Operator + IRSA
#   - EFS CSI driver
#   - APISIX ingress
#   - cert-manager + trust-manager
#   - kube-prometheus-stack (Prom + Grafana + AlertManager)
#   - Kyverno policy engine
#   - Splunk OTEL collectors
#   - Wiz security
#   - PostgreSQL operator
#   - Karpenter for node autoscaling

# Phase 4: GitOps + CI/CD (Weeks 8-12, parallel)
```

```
# - Argo CD with app-of-apps + ApplicationSet patterns
# - GitHub Actions migration from CircleCI/Jenkins
# - OIDC federation: GitHub → AWS

# Phase 5: Migration waves (Weeks 13-24)
# - Pilot: lowest-risk service first (read-only, internal, idempotent)
# - Dual-deploy: same workload running in both, traffic split via DNS
# - Validate: parity checks, performance, behavior
# - Cutover: shift traffic, monitor, rollback if needed
# - Decommission: drain OpenShift, archive runbooks

# Migration pattern per service:
# 1. Containerize if needed (some legacy used s2i)
# 2. Re-author manifests (Routes → Ingress, etc.)
# 3. Migrate secrets to AWS Secrets Manager
# 4. Deploy to EKS dev via GitOps
# 5. Validate functionally + load test
# 6. Migrate stage → prod with traffic splitting
# 7. Watch SLOs for 2 weeks
# 8. Decommission OpenShift instance

# Risks and mitigations:
# - OpenShift Routes → Ingress behavioral differences
#   Mitigation: test parity, especially TLS, redirects, sticky sessions
# - SCC permissive defaults → PSA restricted
#   Mitigation: gradual hardening, audit-mode first
# - Stateful workloads (DB, queues)
#   Mitigation: snapshot + replicate; cutover during maintenance window
# - Internal registry → ECR
#   Mitigation: dual-publish during transition
# - Network policies different defaults
#   Mitigation: rewrite based on actual traffic patterns from flow logs

# Metrics for success (your resume's numbers):
# - Operational efficiency improved 50%
# - Deployment speed: 20% faster
# - Downtime: 30% reduction

# Total: 24 weeks (6 months) for ~100 services
```

■ **Tip:** This is THE story to tell. Show you've done it. Hit the resume metrics.

Scenario

Q10. A junior engineer accidentally pushes commit secrets (AWS keys) to a public repo. Walk through your response.

A: (1) ROTATE keys IMMEDIATELY: don't wait for cleanup. AWS console or CLI: `aws iam create-access-key`, then `aws iam delete-access-key` for the exposed one. Even if removed from Git, assume scrapers grabbed it within seconds. (2) Audit usage: CloudTrail for the exposed key. Look for: any API calls from unfamiliar IPs/regions, any unusual activity, any resource creation. Even if no abuse yet, attackers may stash credentials for later. (3) Lock down: if the key had broad permissions, consider broader rotation (entire user, role chain it could assume, MFA reset for the human). (4) Cleanup Git: rebase commits to remove the secret, force-push. BFG Repo-Cleaner for full history. But: this doesn't undo the leak. Cached forks, GitHub archive search, scrapers all already saw it. The rotation is the real fix. (5) Talk with the engineer: blameless. The fault is the system, not them. (6) Strengthen prevention: pre-commit hooks (gitLeaks, detect-secrets), CI secret scanning, .gitignore for typical secret locations, env vars instead of files in code, dev-time aws-vault or aws sso for short-lived creds (no long-lived keys on laptops in the first place). (7) Postmortem: how did they get the keys? Where do all keys live? Move to OIDC federation everywhere — no long-lived secrets to leak.

■ **Tip:** Blameless. New engineers WILL make this mistake. The senior response is system hardening, not punishment.

Scenario

Q11. Your Terraform state file got corrupted. Recover.

A: (1) Stop all activity on this state. Lock it. (2) Inspect: is the file valid JSON? `terraform show -json terraform.tfstate`. If it's parseable, what's missing? Resources, outputs, lineage? (3) Check S3 versioning — recover the prior version. If you use S3 backend with versioning enabled (you should), this is a one-line `aws s3api` recovery. (4) Run `terraform plan` against the recovered state — does it match reality? Should be no-op or close.

(5) If versioning isn't available: terraform import each resource from the cloud back into a fresh state file. Tedious but works. Tooling like terraformer can help bulk-import. (6) If state has secrets and was committed somewhere — rotate those secrets. (7) Postmortem: enable S3 versioning on state buckets (this is THE prevention), enable S3 object lock (compliance), separate state per blast radius (smaller blast if it happens), backup state to a second region. Apply RBAC tightly — only Atlantis can write to state, never humans directly.

■ **Tip:** S3 versioning is THE prevention. Set it on day 1 of every state bucket. Free.

Scenario

Q12. You're on-call. A node goes NotReady. Walk me through.

```
# Step 1: Confirm and scope
kubectl get nodes
# Just this one? Or a pattern?

# Step 2: What's actually wrong?
kubectl describe node <node-name> | tail -50
# Look at Conditions: MemoryPressure, DiskPressure, PIDPressure, Ready=False
# Look at Events: recent kubelet errors, network failures

# Step 3: Can we reach the node?
# AWS console: instance state
# SSM Session Manager: aws ssm start-session --target i-xxx
# Or SSH if you have access

# Common causes:

# CASE A: kubelet died or hung
# (NotReady, no recent kubelet heartbeat in events)
# Recovery: ssh in, sudo systemctl restart kubelet
# Or: just terminate the instance, let ASG/Karpenter replace it

# CASE B: Disk full (DiskPressure)
sudo df -h
# Common culprits: container logs, ECR images, /tmp
# Quick fix: docker image prune, journalctl --vacuum-size=200M
# Long fix: log rotation, /var/lib/docker on larger disk

# CASE C: Memory pressure (MemoryPressure)
# A pod ate all memory; kubelet evicts but pod restarts
# Look at: kubectl top pods --all-namespaces --sort-by=memory
# Fix: memory limit on offending pod, or kill pod

# CASE D: Network — kubelet can't reach API server
# Check: aws-node DaemonSet (CNI) healthy?
# kubectl get ds -n kube-system aws-node
# If many nodes NotReady simultaneously, suspect VPC/network plane

# CASE E: Hardware failure
# dmesg | grep -i error
# AWS instance status checks failed
# Terminate; ASG/Karpenter replaces

# Step 4: Drain pods if recovery slow
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
# This respects PDBs

# Step 5: Replace if needed
# Managed node group: aws eks describe-nodegroup
# Karpenter: deletion triggers replacement automatically
kubectl delete node <node> # tells K8s the node is gone
# Then terminate in AWS console / API
```

■ **Tip:** Tell the EKS recovery story from your resume. Be ready to whiteboard the SG ingress rule fix you found.

Scenario

Q13. Your team has been deploying with kubectl apply for 3 years. You want to migrate to GitOps with Argo CD. How do you sell it AND execute it without breaking things?

A: (1) Build the case: quantify pain. How long does a current rollback take? How many people have prod kubeconfig? What's the audit trail? Compare to GitOps: rollback = git revert (1 min), no kubeconfig spread, audit trail = git log. (2) Pilot: pick one low-risk service (internal tool, dev environment). Set up Argo CD, migrate the service, demonstrate dev-to-prod promotion via PR. Run for 2 weeks. (3) Show results: deploy frequency up, rollback time down, drift detection caught X manual changes that would have caused incidents. (4) Train: workshops on Argo CD usage. Documentation. Pair sessions. (5) Strangler pattern for migration: new services go straight to GitOps. Old services migrated in waves. (6) Keep an escape hatch initially: kubectl access still works in emergencies, but discouraged. Audit every direct kubectl edit. Over time, restrict cluster admin. (7) After all teams onboarded (6-9 months): make GitOps the default, restrict direct cluster access to break-glass procedure only. (8) Trade-offs to acknowledge: Argo CD is one more system to operate. Need to learn Helm/Kustomize. Secret management gets more complex initially. But the wins compound.

■ **Tip:** From your resume: 'Migrated CI/CD pipelines from CircleCI to GitHub/CloudBees and deployed OpenShift clusters with automated onboarding via Argo CD, improving scalability with ~20% faster deployments and ~30% less downtime.' Exact story.

Scenario

Q14. How would you debug a sudden 50% drop in cluster throughput with no obvious cause?

A: Systematic. (1) Confirm: dashboards across multiple metrics agree — is it real or a metric problem? (2) Time correlation: when exactly did it start? Any deploys, config changes, AWS region events, dependency changes at that time? (3) Layer-by-layer: from edge inward. (a) Edge: CloudFront/ALB error rate, latency. (b) Ingress: APISIX/nginx-ingress error rate, target group health. (c) Service: pod readiness, restarts, OOMKills. (d) Pod: CPU throttling, memory pressure, GC pauses, profile data. (e) Dependencies: DB latency, cache hit rate, external API latency. (4) Common 50%-drop causes: one bad pod serving half traffic (uneven scheduling), CFS throttling after a CPU limits change, one AZ degraded (1/3 of traffic affected, looks like ~33% drop), GC tuning regression, downstream service capping at half, connection pool exhausted, DB connection limit. (5) Use traces: pick a slow trace, identify the longest span — that's likely the root. (6) If it's recent change: revert first, diagnose later. (7) If it's organic load growth: scale and assess capacity.

■ **Tip:** Mention CFS throttling explicitly. Senior signal.

Scenario

Q15. How do you handle a vendor SaaS dependency going down (e.g., GitHub down, Slack down) during business hours?

A: Three angles. (1) Acknowledge what you control: communicate internally that you've seen the outage. Avoid duplicated reports. Status page like statuspage.io. (2) Adapt: what work CAN proceed without GitHub? Local dev. Local testing. Anything queued can wait. What can't: deploys, PR reviews. Decide: pause deploys (safer) or use backup path (Jenkins backup, manual oc/kubectl with audit). (3) Communicate to leadership: 'GitHub is down, ETA from their status page is X. We've paused deploys for safety. Customer-facing systems are unaffected (we don't deploy on critical path). Will resume when restored.' (4) Plan for next time: identify SaaS SPOFs. Where does GitHub's outage hurt us most? Tighten dependencies: mirror images to multiple registries, GitOps repo backed up to GitLab/Bitbucket too, runbook for manual operations during SaaS outages. (5) Don't panic-engineer during the outage. Wait for restoration unless customer-facing is actually broken.

■ **Tip:** Senior maturity: 'don't add complexity during an outage you don't control.' Most outages resolve in 1-4 hours.

Round 11: Behavioral / HR / Hiring Manager (45-60 min)

Final round. Tests culture fit, leadership maturity, communication, and growth potential. NVIDIA values: 'one team' (Jensen's philosophy), excellence (intellectual rigor), speed of light (no wasted time), innovation. Use STAR rigorously: Situation, Task, Action, Result. Have 5-7 polished stories from your 14 years at Cisco that you can adapt to any question.

Behavioral

Q1. Tell me about a time you disagreed with your manager.

A: Use STAR. Pick a real conflict where you had data and showed respect. Sample frame: 'During the EKS migration at Cisco IQ, my manager wanted to delay the External Secrets Operator rollout to focus on raw compute migration. I disagreed — I felt secrets were a higher risk path. I gathered data: we had 12 services with hard-coded secrets in OpenShift manifests, and the migration plan would propagate them into EKS without External Secrets. I presented a one-pager: risk (compliance audit miss), cost (additional 2 weeks for ESO setup), and alternative (parallel track). My manager pushed back on timeline. I proposed: I'll personally do the ESO work in parallel, no impact to the compute migration. He agreed. Outcome: we shipped both on time, passed the compliance audit, and the ESO pattern became the standard for all future migrations. Lesson: bring data, propose alternatives, don't dig in on opinion alone.'

■ **Tip:** Show respect, present data, end with measurable outcome. Don't make your manager look bad.

Behavioral

Q2. Tell me about a time you mentored someone.

A: Sample: 'On the IQ platform team, a newer SRE was struggling with Terragrunt's dependency model. PRs were taking 2-3 review cycles because of duplicate state lookups and missing module pins. I spent two paired sessions walking through our pattern — dependency blocks, locals inheritance, version pinning. Then I built a skeletal `new-cluster-onboarding` template repo that codified our conventions. I had her use it for her next deliverable. PR was clean first try. After that, I made the template a team standard, documented it in the runbook, and held a brown-bag for the rest of the team. Result: PR cycle time for new infrastructure dropped about 50%. Lesson: mentoring works best when the takeaway is a reusable artifact, not just words.'

■ **Tip:** From your resume: AI-augmented engineering work, knowledge-base templates. Adapt that into a mentoring story.

Behavioral

Q3. Tell me about a time you made a mistake in production.

A: Vulnerability + ownership + lesson. Sample: 'Earlier in my Cisco IQ time, I pushed a Terragrunt change that updated a security group ingress rule for an EKS node group. I tested in dev. It looked clean. I applied to prod. About 20 minutes later, an app team reported their pods couldn't reach the cluster API server — Fargate CoreDNS was isolated. Root cause: my change tightened ingress in a way that prevented Fargate-launched CoreDNS pods from reaching the cluster primary security group. I had missed it because dev didn't use Fargate. Recovery: I reverted within 5 minutes once I confirmed the connection. Then I spent 4 hours documenting the actual SG topology — 29 sections of resource inventory — and pushed 5 PRs to codify the missing rule plus drift detection. Lesson: dev parity with prod matters; also, any change to networking should require explicit testing across all consuming workload types. Now we have a checklist for that. The 29-section inventory is in our runbook.'

■ **Tip:** This is literally your resume's EKS cluster recovery story. Tell it with vulnerability.

Behavioral

Q4. Tell me about a time you had to learn something quickly.

A: Sample: 'When we started the GenAI pipelines for renewals and sentiment analysis, LangChain was new to me — I'd worked with LLMs at a lower level but not the orchestration framework. The product team had

committed to a Q-end demo, and I had 3 weeks to deliver the platform side. I broke it down: week 1, read LangChain docs cover to cover and built three POC chains. Week 2, integrated with our Kubernetes/OpenStack environment, secured with Vault. Week 3, hardened for production — added LangSmith for tracing, set up CI/CD. The demo shipped. Following month, I wrote our internal LangChain deployment pattern doc. Lesson: when learning under deadline, build something within 48 hours — concrete artifacts surface gaps faster than reading. Then iterate.'

■ **Tip:** Resume: 'Designed and deployed GenAI pipelines (LangChain, LangSmith) on Kubernetes/OpenStack.' Tell this story.

Behavioral

Q5. Tell me about a time you had to influence without authority.

A: Sample: 'During the OpenShift-to-EKS migration, three app teams owned their own deploy patterns — some used Helm, some Kustomize, some raw manifests. I needed them to standardize on a Helm chart pattern for the migration to scale. I had no authority over them. My approach: I built a reference chart for the highest-volume service team. Showed them: their deploy went from 200 lines of YAML to 30 lines of values.yaml. PR review time dropped. I asked them to use it for their next 3 services and report back. They did. They became advocates. The other teams asked them — not me — how it worked. Within 6 months, 80% of services were on the standard chart pattern. Lesson: influence is downstream of value. Demonstrate value to one team; they sell it for you.'

■ **Tip:** Senior engineering management 101. 'Show, don't tell' framing.

Behavioral

Q6. Tell me about your biggest professional accomplishment.

A: Pick the marquee project that has measurable impact and senior-scope ownership. 'The IQ Platform's CXTM buildout at Cisco. I owned the AWS infrastructure design and implementation end-to-end: VPC topology, EKS cluster, Karpenter autoscaling, ArgoCD multi-account rollout, External Secrets, cross-account NLB-to-ALB routing, observability stack. Spanned three regions (US, EU, APAC), 2 quarters of execution. The platform now hosts ~30 services for the CXTM product line. Impact: deployment consistency standardized, onboarding friction reduced, ArgoCD-driven GitOps replaced 3 different per-team deploy flavors, and the infrastructure pattern is now reusable for any new cluster onboarding. Beyond the tech, what I'm proudest of: the entire pattern is codified — Terragrunt modules, runbook, OPA policies, documentation — so the team isn't dependent on me to extend it. That's senior engineering for me: build leverage, not single-points-of-failure.'

■ **Tip:** Senior signal: end with 'I built leverage, not just shipped a thing.' That's the level-up to staff/principal thinking.

Behavioral

Q7. Why do you want to leave Cisco?

A: Already covered in Round 1 but expect at HR too. Refine: 'Cisco has been excellent — 14 years of growth from customer support engineer to technical leader. I've owned major platform builds, including the IQ Platform's EKS migration and the CXTM buildout. What's drawing me to NVIDIA is a step-change in scope: the infrastructure that runs every modern AI inference workload. The AI Inference Operations role specifically — Dynamo and NIXL — sits exactly at the intersection of my Cisco IQ work (Kubernetes platform, secure delivery, observability, GenAI pipelines) and where the industry is going. It's not that Cisco is wrong; it's that NVIDIA is more right for the next chapter of my career.'

■ **Tip:** Always positive about current employer. Always specific about why NEW employer (not just 'big company'). Bonus: NAME the product (Dynamo, NIXL).

Behavioral**Q8. What's your biggest weakness?**

A: Real weakness + how you manage it. NOT 'I work too hard.' Sample: 'I tend to over-invest in writing things down — runbooks, documentation, postmortems. The strength is repeatability. The weakness is sometimes I spend an hour documenting something that could have been a 5-minute follow-up to the team. I've gotten better by: timeboxing docs writing (30-min cap unless it's a major artifact), and asking 'what's the smallest doc that prevents this issue again?' rather than the most complete one. I still err on documenting more than less, but I do it faster.'

■ **Tip:** *Specific, real, demonstrably managed. Don't say something generic that sounds like it came from a list.*

Behavioral**Q9. Describe your leadership style.**

A: 'Practical and outcomes-focused. Three principles: (1) Lead by writing — clear docs, clear PRs, clear architecture diagrams. People follow patterns when they can read them. (2) Build leverage — reusable modules, automation, templates. My job is to make the team faster, not to be the bottleneck. (3) Default to written async — over Slack or meetings. Especially with global teams (the IQ Platform team is across US, EU, India), async-by-default keeps everyone in sync without burning their evenings. I'm hands-on technical — I write Terraform, I review PRs, I take incident pages. I think senior leadership means being technical AND organizational; either alone isn't enough.'

■ **Tip:** *Senior signal: 'I'm hands-on technical AND organizational.' That's how Senior+ engineers are differentiated.*

Behavioral**Q10. How do you handle a teammate who isn't pulling their weight?**

A: 'Specific concern. First step: assume best intent and ask. Maybe they're blocked on something I don't see. I'd 1:1 with them — 'I noticed X is stalled, what's getting in the way?' Often it's a real obstacle (unclear requirements, missing access, blocked on a dependency). If it's that, I help unblock. If after unblocking the pattern continues, I'd raise to my manager with specifics — not 'they're slacking' but 'these 3 deliverables slipped by X days each, here's what I've tried.' Let management handle the people side. If I'M the manager: more direct conversation about expectations, support plan, and timeline. People don't usually want to underperform; usually it's a system issue. Patience and clarity, not punishment.'

■ **Tip:** *Senior maturity: 'people don't want to underperform.' That framing matters.*

Behavioral**Q11. Tell me about a time you missed a deadline.**

A: Real story + ownership. 'During the IQ Platform GenAI build, I'd committed to a CI/CD migration from CircleCI to GitHub/CloudBees in 4 weeks. Mid-week 2, I hit a wall: CloudBees plugin compatibility issues with our Jenkins shared library. To fix properly required 2 more weeks. I had two choices: cut scope or miss the deadline. I went to my manager early — at end of week 2, not week 4. I explained: the original deadline is at risk, here's why, here are two options. Option A: I deliver a partial migration on schedule with documented gaps. Option B: I deliver the full migration 2 weeks late. We picked B because Option A would have created tech debt. We shipped 2 weeks late but with the right architecture. Lesson: when slippage is inevitable, communicate early. The cost of a 2-week slip caught at week 2 is much smaller than caught at week 4.'

■ **Tip:** *Communicating early is the senior virtue. Not 'I worked overtime and shipped on time' — that's a junior brag.*

Behavioral**Q12. How do you stay current technically?**

A: 'Layered. Daily: feeds for the tech I use — KubeWeekly, AWS What's New, CNCF blog, Hacker News. I skim, bookmark deep dives. Weekly: hands-on with a tool I haven't used. Recent example: Cilium and eBPF — I built a small POC to understand it. Quarterly: certification or major learning project — I added the Terraform Associate this year, and I'm working through Anthropic's prompt engineering and the OpenTelemetry spec deeply right now. Per project: I treat every project as a chance to use something new — Karpenter when it was new, External Secrets when it was new, LangChain when it was new. Cisco gave me a wide surface area to do this. Selectively:

I follow specific engineers and orgs whose decisions I respect — Charity Majors, Brendan Gregg, the SRE Weekly, the Go Time podcast.'

■ **Tip:** *Specific is better than generic. 'I read books' is weak; 'I built a Cilium POC last quarter' is strong.*

Behavioral

Q13. Why should we hire you over other senior candidates?

A: Specific differentiators. 'Three reasons. First, the technical overlap is exact. The JR2017386 role asks for Kubernetes, Ansible, Terraform, AWS, CI/CD, CVE handling, GitHub admin — that's literally my Cisco IQ stack. Most candidates have 60-80% of that; I have all of it, deployed in production. Second, I have AI infrastructure experience specifically: I built and operate GenAI pipelines using LangChain and LangSmith on Kubernetes, integrated with platform SRE standards. That maps to NVIDIA AI Inference Operations directly — same architecture concerns, just at NVIDIA's scale. Third, I'm at the right point in my career. I've owned multi-quarter platform builds end-to-end, but I'm not so senior that I want to stop being hands-on. I write Terraform, I review PRs, I take incidents. NVIDIA's Senior level seems to want exactly that — technical lead, not a pure manager. That's where I am, and that's where I want to stay.'

■ **Tip:** *Show you've read the JD. Tie YOUR specifics to THEIR specifics. Don't say 'I'm a hard worker.'*

Behavioral

Q14. What do you look for in a manager?

A: 'Three things. Trust and autonomy: I want a manager who tells me the outcome and trusts me with the path. I've done my best work under managers who set context, asked 'what do you need?' and got out of the way. Technical depth: I work better with managers who can have a real technical conversation — not necessarily a hands-on coder, but someone who can reason about systems. Otherwise tradeoffs get lost. Honest feedback: I'd rather hear 'this isn't working' early than a polite review six months later. I give feedback the same way and expect the same. Career-wise: I want a manager who actively advocates for growth — projects that stretch, sponsorship for visibility, calibration support at performance review time.'

■ **Tip:** *Subtle: 'autonomy' and 'technical depth' are both code for 'I want to be senior, not micromanaged.' That's the right signal.*

Behavioral

Q15. Where do you see yourself in 5 years?

A: 'Honestly: deeper, not wider. I'm not chasing management. I want to be a staff/principal-track engineer who's deeply technical, working on the most strategic infrastructure problems at the org. In 5 years, I'd hope to be Staff at NVIDIA, owning a multi-team-spanning infrastructure initiative — something like 'inference platform reliability at scale' or 'the standard way we ship AI infrastructure.' That role exists at NVIDIA. I'd want to be in it.'

■ **Tip:** *Specific career path. NVIDIA has a strong IC track (Staff, Principal, Distinguished). Reference it.*

HR

Q16. What's your expected start date?

A: 'I'd give 2 weeks notice at Cisco. So from offer signature to first day, about 2-3 weeks. I'm flexible to align with NVIDIA's onboarding schedule.'

HR

Q17. Do you have any concerns about the role or the team?

A: 'Two things I'd want clarity on, not concerns. First, the on-call rotation: what's the frequency, how is escalation structured, what's the SLO for response? Second, the team's distribution: is this team co-located, distributed, or a mix? I've worked effectively in hybrid models but I want to understand the team's working norms. Those are the operational questions; technically, the role matches my background well.'

■ **Tip:** *Show you're thinking professionally about how the work will happen. Don't bring up real concerns at this stage.*

HR

Q18. Tell me about a benefit or compensation element that's important to you.

A: 'Equity refresh and the technical learning budget. The base and bonus matter, but at this level my total comp is dominated by RSUs. NVIDIA's equity program is strong, and an annual refresh is what I'd be looking for. The technical learning budget — for certifications, conferences, books — is the other one. I've grown by staying current, and a budget for that is a strong signal that the company invests in engineering.'

■ **Tip:** *Avoid asking about PTO or flexible hours in HR — sounds like you're looking to work less. Equity and growth are the right asks.*

HR

Q19. Do you have any questions for me?

A: Always 3-5 ready. HR-appropriate: (1) 'What's the typical performance review cadence — half-year, year?' (2) 'What's the equity vesting schedule? Standard 4-year-with-1-year-cliff or something different?' (3) 'How does NVIDIA structure equity refreshes — annual, performance-tied?' (4) 'What's typical for a senior engineer's first 90 days here — buddy program, onboarding plan, what does success look like?' (5) 'When can I expect to hear next steps?'

■ **Tip:** *ALWAYS have 5+ questions. Never say 'no I'm good.' Even after a long process, ask 1-2.*

HR

Q20. What would make you turn down an offer from NVIDIA?

A: Honest. 'Three things. One: comp significantly below market — I've done the research, I'd want to be in the range we've discussed. Two: a role significantly different from what was advertised — if the actual day-to-day isn't the AI Inference Operations DevOps work, I'd be hesitant. Three: a team where the working style feels broken — siloed teams, poor PR review culture, no on-call discipline. I'd want to talk with future teammates as part of the interview to gauge that.' But realistically, if compensation aligns and the role matches, I'd take it.

■ **Tip:** *Honest framing. Don't be a pushover; you have value. But signal you're motivated to make this work.*

Production Debug Workflows

These are the muscle-memory playbooks for production incidents. Memorize the structure; the specifics vary. When you're paged at 3am, you don't have time to think — you have time to execute. Reference these during scenario rounds (R10) and incident response questions (R7).

Pod Pending

Scheduler can't place pod

```
kubectl describe pod <pod> -n <ns> → read Events at bottom
→ check requests vs node allocatable (kubectl describe nodes)
→ taints / tolerations (kubectl describe nodes | grep -i taint)
→ nodeSelector / affinity (kubectl get pod -o yaml | grep -A 5 selector)
→ PVC binding (kubectl get pvc -n <ns>)
→ ResourceQuota (kubectl describe quota -n <ns>)
→ Karpenter / Cluster Autoscaler logs (look for ICE, IAM, AWS errors)
→ Fix: scale cluster, fix selectors, fix PVC, broaden node pool
```

CrashLoopBackOff

App starts then dies

```
kubectl logs <pod> -n <ns> --previous ← THE crash output
→ kubectl describe pod → check Exit Code (137=OOM, 1=app, 139=segv, 143=SIGTERM)
→ env vars / configmap / secret refs available? (kubectl describe configmap/secret)
→ livenessProbe too aggressive? slow boot need startupProbe?
→ dependency reachable from pod? (kubectl exec -- nc -vz db 5432)
→ OOM? bump memory limit OR fix leak
→ Fix: rollback if recent deploy, else address root cause
```

ImagePullBackOff

Registry / auth / network

```
kubectl describe pod <pod> → look at Failed: image pull errors
→ image name + tag spelled right? (typo is #1 cause)
→ image actually exists in registry? (aws ecr describe-images)
→ imagePullSecrets / IAM access from node? (IRSA, instance profile)
→ network: NAT / VPC endpoints for ECR reachable?
→ rate limit? (DockerHub anonymous = 100/6hr)
→ architecture match? (arm64 image on amd64 node, or vice versa)
→ Fix: correct tag, fix IAM, add VPC endpoint, use ECR pull-through cache
```

Argo CD OutOfSync (didn't change Git)

Live ≠ desired

```
argocd app diff <app> → see exact field-level differences
→ controller fields (HPA replicas, mutating webhooks)? use ignoreDifferences
→ manual change via kubectl? decide: backport to Git OR sync to revert
→ default values changed in K8s upgrade? update manifests
→ Fix: ignoreDifferences for controller-owned, sync for everything else
→ Long-term: enable self-heal for non-prod, audit who has kubectl edit
```

Failed terraform apply mid-run

Partial state

DO NOT rerun blindly

- read error: which resource, which step?
- state lock still held? (terraform force-unlock only if no apply running)
- terraform plan → see what's still tracked
- AWS console: what was actually created? deleted? partial?
- Decide forward-fix (apply again after fixing config) vs rollback (revert PR)
- state inconsistent? terraform state list/rm/import (with state backup)
- Never edit state by hand in prod

GitHub Actions can't assume AWS role

OIDC trust mismatch

Check workflow has 'permissions: id-token: write'

- IAM role trust policy: principal = github OIDC provider?
- trust 'sub' claim matches: repo:org/repo:ref:refs/heads/main?
- trust 'aud' = sts.amazonaws.com?
- role ARN typed correctly in workflow?
- CloudTrail event: AssumeRoleWithWebIdentity failure reason?
- environment-protected secret accessible from this branch/env?

Users see 503 but pods are healthy

Routing problem

Check ingress / ALB / target group health: AWS console or kubectl describe ingress

- Service has endpoints? (kubectl get endpoints <svc>)
- readiness probe path/port matches app reality?
- security group on ALB → node SG → app port allowed?
- target group health check path/port correct?
- ALB logs (S3): which target group? what response?
- Fix: realign readiness, fix SGs, fix target group config

AWS bill spike

Cost incident

Cost Explorer: group by Service, then by Tag

- top offenders: EC2, NAT, EBS, data transfer, RDS?
- correlate with deploys / launches / new env
- drill into NAT: data transfer hidden but huge
- drill into EBS: unattached volumes? unused snapshots?
- CloudWatch: orphaned log groups never expiring?
- Karpenter logs: lots of provisioning churn?
- Fix: VPC endpoints for S3/ECR, lifecycle policies, rightsize, Spot, budgets+alerts

Quick-Reference Cheatsheet

Final morning review. Read THIS, don't try to learn anything new the day-of. Frameworks, commands, mental models — the things to have on the tip of your tongue.

Answer Frameworks (use these verbatim)

STAR (behavioral)	Situation → Task → Action → Result
TROUBLE (debug)	Trigger → Reproduce → Observe → Unblock → Build-fix → Learn → Eliminate recurrence
ARCH (system design)	Actors → Requirements → Components → High availability → Risks/tradeoffs
SECURE (cloud security)	Scope → Encrypt → Control access → Update/patch → Rotate secrets → Evidence/logging
Incident	Detect → Triage → Mitigate → Communicate → Resolve → Postmortem
Tool comparison	What it does → strengths → weaknesses → when I'd use it

Kubernetes / kubectl essentials

Pod state	<code>kubectl get pods -n <ns> -o wide</code>
Pod details + events	<code>kubectl describe pod <pod> -n <ns></code>
Logs (current)	<code>kubectl logs <pod> -n <ns> -c <container></code>
Logs (crashed instance)	<code>kubectl logs <pod> -n <ns> --previous</code>
Stream logs	<code>kubectl logs -f <pod> -n <ns></code>
Exec into pod	<code>kubectl exec -it <pod> -n <ns> -- sh</code>
Ephemeral debug	<code>kubectl debug -it <pod> --image=busybox --target=<container></code>
Events sorted	<code>kubectl get events -n <ns> --sort-by=.lastTimestamp</code>
Rollout status	<code>kubectl rollout status deploy/<name> -n <ns></code>
Rollback	<code>kubectl rollout undo deploy/<name> -n <ns></code>
Top pods (CPU/mem)	<code>kubectl top pods -A --sort-by=memory</code>
Node status	<code>kubectl get nodes / kubectl describe node <name></code>
Service endpoints	<code>kubectl get endpoints <svc> -n <ns></code>
Cordon / drain	<code>kubectl cordon <node> / kubectl drain <node> --ignore-daemonsets</code>
Apply with diff	<code>kubectl diff -f manifest.yaml / kubectl apply -f manifest.yaml</code>
Port forward	<code>kubectl port-forward svc/<svc> 8080:80</code>

Linux debugging — fundamentals

Top by mem / CPU	<code>ps -eo pid,user,%mem,%cpu,comm --sort=-%mem head</code>
Who's on port?	<code>ss -lntp grep '<port>' / lsof -i :<port></code>
Top largest files	<code>find / -type f -size +100M 2>/dev/null xargs du -h sort -rh head</code>
Deleted-open files (disk-full mystery)	<code>lsof +L1 → df vs du mismatch</code>
Disk I/O	<code>iostat -xz 1 (await/util > 90% = saturated)</code>
Memory pressure	<code>cat /proc/pressure/{cpu,io,memory} (PSI metrics)</code>
Load average	<code>uptime (divide by nproc to interpret)</code>
Run-state vs blocked	<code>vmstat 1 5 (r=running, b=blocked-on-IO)</code>

RSS vs PSS vs VSZ	RSS=real RAM, PSS=shared-divided, VSZ=mapped-virtual (often huge, ignore)
Network connections	ss -tnp state established / ss -s (summary)
DNS resolution path	dig +short <name> / cat /etc/resolv.conf / nslookup <name>
Trace syscalls	strace -e openat,read,write -p <pid>
Open files for process	ls -la /proc/<pid>/fd/
Kernel msgs	dmesg -T tail -50 / journalctl -u <unit> -f

Terraform / Terragrunt

Init / validate / plan / apply	fmt → validate → init → plan -out=tf.plan → apply tf.plan
State management	S3 + DynamoDB lock; encrypt; version; separate per blast radius
Modules vs workspaces	Modules = reusable code; workspaces = state separation (avoid for prod)
count vs for_each	for_each ALWAYS unless purely numeric scale (no identity)
Import existing resource	terraform import OR import { to=... id=... } block (1.5+)
Prevent destruction	lifecycle { prevent_destroy = true } + service-level deletion protection
Secrets in TF	Reference ARN, ignore_changes on values; state itself is sensitive
Drift detection	Scheduled terraform plan in CI; AWS Config; alert on diff
Failed mid-apply	Don't rerun blindly; plan, check reality, decide forward/rollback
Terragrunt key benefits	DRY backend config; dependency blocks; auto-init; per-env state

AWS — memory hooks for networking & identity

VPC	Neighborhood (isolated network)
Subnet	Street (CIDR slice, single AZ)
Route Table	GPS routes (per-subnet)
Internet Gateway	Public exit
NAT Gateway	Private outbound exit (in PUBLIC subnet)
Security Group	Stateful firewall on ENIs (allow only)
NACL	Stateless subnet firewall (allow + deny)
VPC Endpoint	Private road to AWS service (no NAT cost)
ALB	Layer 7 — understands URLs, host/path routing, TLS terminate, WAF
NLB	Layer 4 — TCP/UDP, static IP, ultra-low latency, preserves source IP
IAM Role	Who can do what (assumable; short-lived)
IRSA	Pod-level AWS identity via K8s SA + OIDC
EKS Pod Identity	Newer simpler IRSA (no OIDC plumbing)
Cross-account access	Role in target trusts principal in source via sts:AssumeRole
OIDC federation (CI)	GitHub Actions → STS → short-lived AWS creds, no static keys

CI/CD & GitOps — patterns

Pipeline stages	Lint → unit test → SAST/dep scan → build image → sign → push → GitOps PR → Argo sync → smoke
Build once, promote	Same image digest from dev through prod; never rebuild for prod
Argo CD sync states	Synced (matches Git) + Healthy (resources ready) — both required

Argo app-of-apps	Root app manages child apps; ApplicationSet generates many from template
Sync policies	manual / automated; +prune (deletes); +selfHeal (reverts drift)
Rollback in GitOps	git revert + sync (NOT helm rollback in cluster)
Secrets in CI/CD	OIDC federation; never long-lived keys; mask in logs; pre-commit secret scan
Self-hosted runners	Ephemeral, isolated by repo, no untrusted PR with secrets, private subnet
CVE handling	Scan → triage (severity + exploitability) → rebuild → re-scan → roll out → verify zero affected

SRE / Observability mental models

Four Golden Signals	Latency, Traffic, Errors, Saturation
RED method	Rate, Errors, Duration (services)
USE method	Utilization, Saturation, Errors (resources)
SLI / SLO / SLA	Indicator (measured) / Objective (internal target) / Agreement (contract)
Error budget	100% - SLO = allowed unreliability; freeze on exhaustion
Burn rate alerts	Multi-window (5m AND 1h both high) catches outage + slow degradation
Liveness probe rule	Only use if you can describe the deadlock — otherwise omit
Readiness probe	Controls traffic eligibility; failures DO NOT restart pod
Startup probe	Suspends liveness/readiness during slow boot (use for ML loading)
Postmortem rules	Blameless · timeline · contributing factors (plural) · action items with owners
Cardinality bomb	High-cardinality labels (user_id, request_id) explode time-series
Toil reduction	Manual + repetitive + automatable = automate or eliminate at source
Graceful degradation	Serve 80% well over 100% poorly; circuit breakers; cache fallback

Python — production patterns

Counter / freq	from collections import Counter; Counter(it).most_common(N)
Default dict	from collections import defaultdict; defaultdict(list)
Stack / queue	list (stack), collections.deque (both ends, O(1))
Min-heap	import heapq; heappush(h, x); heappop(h); negate for max-heap
Top K	heapq.nlargest(k, it), heapq.nsmallest(k, it)
Binary search	import bisect; bisect.bisect_left(a, x); bisect.insort(a, x)
Cache function	from functools import lru_cache, cache; @cache (3.9+)
Running sum	import itertools; itertools.accumulate(it)
Combos / perms	itertools.combinations / permutations / product
Streaming JSON	for line in sys.stdin: json.loads(line) — bounded memory
Retry with backoff	exponential * 2^attempt + jitter (random.uniform 0..delay)
LRU cache	OrderedDict + Lock; or functools.lru_cache (single-threaded)
YAML safe	yaml.safe_load (NEVER yaml.load — RCE risk)
Walrus operator	if (m := re.match(p, s)): use(m.group(1))
Type hints	Always in production code; List, Dict, Optional, Tuple

NVIDIA-specific signals (drop these naturally)

nvidia-device-plugin	DaemonSet exposing GPUs as nvidia.com/gpu schedulable resource
-----------------------------	--

dcgm-exporter	Prometheus exporter for GPU utilization, memory, temp, ECC
MIG (Multi-Instance GPU)	H100/A100 partitioning — 2x40GB or 7x10GB slices on one GPU
Triton Inference Server	NVIDIA's open-source model serving stack (covers your DevOps surface)
Dynamo + NIXL	NVIDIA AI Inference platforms — the products of this team
GPUDirect / RDMA / RoCE	Low-latency GPU-to-GPU networking (InfiniBand or RoCE)
NCCL	NVIDIA collective comm library for multi-GPU training
TensorRT	Optimized inference runtime; quantization (FP16/INT8)
Karpenter for GPU pools	Standard pattern: NodePool per GPU type with taints
Buildx multi-arch	linux/amd64 + linux/arm64 (Grace Hopper / Grace Blackwell are ARM)

Interview day discipline

Repeat the question	Buys thinking time, confirms understanding
Think out loud	Reasoning is graded, not just the answer
Ask clarifying questions	Especially for system design — never assume requirements
When stuck	State what you DO know; partial credit > silence
Coding edge cases	Empty input? Single item? Negative? Duplicates? Overflow?
Mitigate before diagnose	For any incident scenario — restore service FIRST
Cite real stories	Your Cisco IQ work: EKS recovery, ArgoCD multi-account, OpenShift→EKS
Ask 3-5 questions back	Never 'no I'm good' — that's the single biggest red flag
Behavioral end strong	Specific differentiators tied to JD; not 'I'm a hard worker'
After interview	24hr thank-you note referencing something specific from the chat

The single rule: every story you tell should be REAL. You have 14 years of Cisco platform engineering. OpenShift→EKS, CXTM buildout, ArgoCD multi-account, EKS cluster recovery, GenAI pipelines — these are your gold. Use them. Good luck.